智锻代码·开源鸿蒙全球AI Agent代码生成挑战赛-技术报告

- 技术报告（基于 Jarvis-Agent 的 jsec 与 jc2r 实现）
  - 目录
  - 系统架构设计
    - 系统关系总览
    - 系统关系说明
    - 系统依赖关系总结
    - 设计原则
    - Agent 系统架构设计
    - CodeAgent 系统架构设计
    - jarvis-sec 系统架构设计
    - jarvis-c2rust 系统架构设计
  - 核心算法与策略
    - 代码安全问题检测算法设计
    - C/C++到Rust转换策略和决策树
    - unsafe使用决策机制
    - 渐进式代码演进路径规划

技术报告（基于 Jarvis-Agent 的 jsec 与 jc2r 实现）

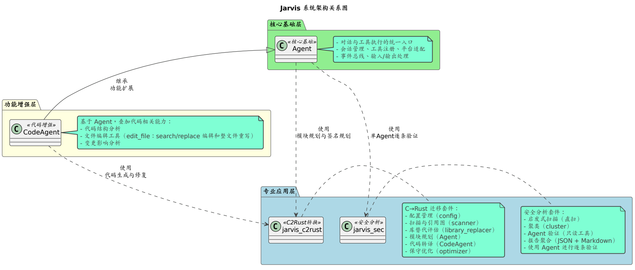
目录

- a. 系统架构设计
- b. 核心算法与策略
- c. 功能实现说明
- d. 测试方案设计

系统架构设计

Jarvis 系统采用分层架构设计，以通用 Agent 为核心基础，通过继承的方式构建了 CodeAgent 增强系统，并在此基础上实现了 jarvis-sec（jsec）和 jarvis-c2rust（jc2r）两个专业应用系统。

系统关系总览



系统关系说明

1. 核心基础层：Agent（通用Agent）

定位：Jarvis 的核心执行实体，提供对话、工具执行、会话管理等基础能力。

核心能力：

- 对话与工具执行的统一入口
- 会话管理（SessionManager）、工具注册（ToolRegistry）、平台适配（PlatformRegistry）
- 事件总线（EventBus）、输入/输出处理链
- 支持非交互模式等场景

扩展点：

- 通过 OutputHandler 协议扩展输出处理能力
- 通过 ToolRegistry 扩展工具能力
- 通过事件总线实现旁路扩展

2. 功能增强层

2.1 CodeAgent（代码Agent）

关系：继承自 Agent，在通用 Agent 基础上叠加代码相关能力。

增强能力：

- 代码结构分析（文件/函数/类层次结构）
- 文件编辑工具（edit_file，支持 search/replace 编辑和整文件重写）
- 变更影响分析（依赖关系、调用链分析）
- 自动迭代修复（基于构建错误）

应用场景：

- 被 jarvis-c2rust 用于代码生成与修复（代码生成实现方法）
- 独立使用：`jarvis-code-agent`（jca）命令行工具

3. 专业应用层

3.1 jarvis-sec（jsec：安全分析套件）

关系：基于 Agent 实现安全分析能力。

使用方式：

- **CLI 交互模式**：提供 `config`、`scan`、`analyze` 三个子命令，支持配置文件管理（`.jarvis/jsec/config.json`）和零参数执行扫描；`analyze` 支持从外部 JSON 文件分析安全问题（支持标准格式或非标准格式自动转换）
- **启发式扫描**：纯 Python 本地扫描，不依赖 Agent，采用模块化检查器架构
- **C/C++ 与 Rust 检查器**：按语言拆分为 C/C++ 检查器（`c_checker.py`）和 Rust 检查器（`rust_checker.py`），C/C++ 检查器包含多种启发式规则（内存管理、缓冲区操作、错误处理、线程安全、格式串风险、命令执行等），覆盖不安全 API、边界类、内存管理类、错误处理与并发类、输入/权限/敏感信息等
- **Agent 验证**：使用 Agent 进行逐条验证（只读工具：代码读取工具/脚本执行工具）
- **报告聚合**：将 Agent 验证结果聚合为 JSON + Markdown 报告

工作流程：

```
直扫（direct_scan）→ 聚类（Cluster Agent）→ 分析（Analysis Agent）→
验证（Verification Agent）→ 复核（Review Agent）→ 报告聚合
```

3.2 jarvis-c2rust（jc2r：C→Rust 迁移套件）

关系：同时使用 Agent 和 CodeAgent，实现完整的 C→Rust 迁移流水线。

使用方式：

- **Agent 用于规划**：
  - 模块规划（模块规划模块）：使用 Agent 生成 crate 模块结构（JSON）
  - 签名规划（签名规划方法）：使用 Agent 选择目标模块与 Rust 函数签名
  - 代码审查（代码审查方法）：使用 Agent 审查逻辑一致性
- **CodeAgent 用于生成与修复（分离使用）**：
  - 代码生成 Agent：每个函数使用独立的代码生成 CodeAgent（每次调用 get_generation_agent 重新创建，不复用实例），用于代码生成任务（代码生成实现方法）
  - 修复 Agent：每次修复时重新创建 CodeAgent 实例，不复用，用于修复构建错误和测试失败（构建循环方法）
  - 修复 Agent 上下文增强：修复 Agent 的上下文中自动包含原 C 实现代码，帮助 Agent 更好地理解原始实现意图，提高修复准确性
  - 记忆功能：代码生成 Agent 当前实现中未启用 force_save_memory；完成函数实现后可根据需要记录关键信息，供后续修复 Agent 和优化 Agent 使用
  - 依赖检查与实现：在实现或修复函数时，要求检查当前函数及其所有依赖函数是否已实现，对于未实现的依赖函数需一并补齐等价的Rust实现
  - 测试失败信息反馈：测试失败时获取完整的测试失败信息并通过专门的标签传递给修复Agent
  - 测试代码删除检测：基于事件订阅机制（工具调用前事件和工具调用后事件），在每次工具调用后立即检测测试代码是否被错误删除，若检测到问题则立即回退，确保测试代码不会被意外删除（工具调用前回调方法、工具调用后回调方法，优化器模块中同样实现）

CLI 交互模式：提供 **config**、**run**、**verify** 三个子命令。config 管理 `.jarvis/c2rust/config.json`（含 root_symbols、disabled_libraries、additional_notes、enable_ffi_export_validation 等）；run 为一键流水线，内部依次执行 scan → lib-replace → prepare → transpile → optimize（支持断点续跑，状态记录于 run_state.json）；verify 为可选功能对齐验证（独立子命令）。scan、lib-replace、prepare、transpile、optimize 为 run 内部的阶段步骤，非独立 CLI 子命令。

工作流程：

```
scan（scanner）→ lib-replace（library_replacer）→ prepare（模块规划，由 llm_module_agent 执行 LLM 规划并落盘）→
transpile（CodeAgent + Agent）→ optimize（optimizer + CodeAgent）；可选地执行 **verify** 对转译结果做 C 与 Rust 功能对齐分析与迭代优化。
```

- **库替代评估**：评估可用 Rust 标准库和第三方库，决定是使用现有库还是转译 C 实现
- **模块规划增强**：prepare 阶段由 llm_module_agent 执行，Agent 生成 crate 模块结构（JSON）并落盘，智能划分功能模块
- **转译器优化**：基于 CodeAgent 的代码生成，每个函数使用独立 CodeAgent 实例（每次调用重新创建）

- **优化器增强**：基于 CodeAgent 的代码优化，支持测试代码删除检测与回退

特点：

- 渐进式迁移（支持断点续跑）
- 保守与可回退（构建检测与 git_guard）
- 模块化设计（各阶段可独立使用）

系统依赖关系总结

1. **继承关系**：CodeAgent 继承自 Agent，扩展代码相关能力
2. **使用关系**：
    - jarvis-sec 使用 Agent 进行安全分析
    - jarvis-c2rust 使用 Agent 进行规划，使用 CodeAgent 进行代码生成与修复
3. **工具共享**：所有系统共享工具注册表，使用统一的工具接口（代码读取工具、文件编辑工具/文件重写工具、脚本执行工具等）

设计原则

- **分层清晰**：核心基础 → 功能增强 → 专业应用的清晰分层
- **高内聚低耦合**：各系统通过标准接口（Agent/CodeAgent API）交互，保持模块边界清晰
- **可扩展性**：通过继承（CodeAgent）实现能力扩展
- **可复用性**：专业应用系统复用通用 Agent 和 CodeAgent 的能力，避免重复实现

接下来依此对每个模块进行详细说明。

Agent 系统架构设计

本文档围绕 Jarvis 的核心执行实体 Agent，基于源码与现有文档进行结构化设计说明，覆盖模块组成、职责与接口、与外部系统/环境的交互、模块间交互流程、参数说明与典型执行过程。目标读者为本项目开发者与高级用户。

- 相关核心组件：EventBus、MemoryManager、TaskAnalyzer、FileMethodologyManager、PromptManager、SessionManager、ToolRegistry、AgentRunLoop、PlatformRegistry/BasePlatform、工具执行器 execute_tool_call、输入处理器链 builtin_input_handler/shell_input_handler/file_context_handler、文件编辑工具 edit_file（支持整文件重写）等

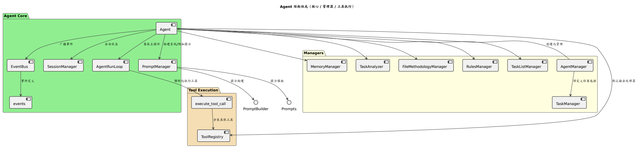
1. 设计目标与总体思路

- 轻协调、强委托：Agent 保持轻量化，侧重编排，将核心逻辑委托至独立组件（运行循环、工具注册表、平台适配层等）。
- 高解耦、可插拔：通过 Registry（ToolRegistry/PlatformRegistry）与事件总线（EventBus）实现能力可插拔与旁路扩展。
- 稳健运行：针对模型空响应、上下文超长、工具输出过大、异常回调等场景提供防御性处理。
- 易扩展与可观测：关键节点统一事件广播，支持 after_tool_call 回调动态注入。
- 多场景友好：支持非交互模式、文件上传/本地两种方法论与历史处理模式、工具筛选降噪等。

2. 模块组成

下图分两部分展示 Agent 内部与其周边模块的静态组成与依赖关系。

图 2-1：Agent 核心、管理器与工具执行


Agent 结构组成（核心 / 管理器 / 工具执行）
Agent Core
Agent
EventBus
SessionManager
AgentRunLoop
PromptManager
events
Managers
MemoryManager
TaskAnalyzer
FileMethodologyManager
RuleManager
TaskListManager
AgentManager
TaskManager
Tool Execution
execute_tool_call
ToolRegistry
PromptBuilder
Prompts
广播事件
管理会话
驱动主循环
构建系统/用户提示
生成提示
生成提示
管理任务列表
管理任务
管理工具
管理工具
管理工具

`tools/<tool_name>.py` 并通过 ToolRegistry 自动注册
    - 关键特性：
        - 支持在生成的新工具内部编排现有 Agent（通用任务编排、ARCHER 工作流、task_list_manager）和 CodeAgent（代码修改、构建验证、lint、review 等）
        - 支持对已有工具进行自举式演化（再次调用 meta_agent，为指定工具生成“_improved”版本）
        - 输入参数：
            - `tool_name`：要生成或改进的工具名，同时作为文件名和类名
            - `function_description`：清晰描述工具的目标功能、输入/输出、约束条件以及是否需要编排 Agent/CodeAgent
        - 返回值：`success/stdout/stderr` 结构，其中 `stdout` 会包含生成结果说明和新工具文件的绝对路径
    - 源码位置：`src/jarvis/jarvis_tools/meta_agent.py`
- **RulesManager**（规则管理）
    - 职责：加载、合并与激活/停用规则，为系统提示提供“已加载规则”内容；支持多来源（全局/项目/内置/中心库/配置目录）与命名规则解析（含前缀）
    - 聚焦：规则目录优先级（中心库 > 项目 > 配置）；特殊规则名（builtin_rules 对应内置 rule.md 索引）；activate_rule/deactivate_rule 与合并结果注入 PromptManager
    - 源码位置：RulesManager 模块（`src/jarvis/jarvis_agent/rules_manager.py`）

3.1.3 工具执行模块（Tool Execution）

- **execute_tool_call**（工具执行统一入口）
    - 图型：时序图（选择处理器→可选确认→处理器执行→标准返回）
    - 聚焦：返回协议（need_return/tool_prompt）；多处理器冲突拒绝策略；执行前确认机制
    - 源码位置：工具执行器模块
- **ToolRegistry**（工具注册表）
    - 图型：时序图（Agent→ToolRegistry→具体工具→返回）
    - 聚焦：单响应一次调用约束；格式容错；长输出分流策略
    - 说明：ToolRegistry 位于 `jarvis_tools` 模块，作为 Agent 的默认输出处理器

3.1.4 输入处理模块（Input Handlers）

- **builtin_input_handler**（内置输入处理器）
    - 职责：处理内置的特殊输入标记（如 `<Web>`、`<FindRelatedFiles>`、`<Dev>`、`<Fix>`、`<Check>`、`<rule:xxx>`、`<Pin>` 等），并追加相应的提示词
    - 处理流程：查找特殊标记 → 从替换映射表获取模板 → 替换标记为模板内容或追加到附加提示
    - **Pin 标记功能**：
        - Pin 标记（`'<Pin>'`）用于固定用户输入内容，确保重要信息在对话过程中不被遗忘
        - 处理逻辑：Pin 标记最后处理，移除标记后的内容会追加到 `agent.pin_content` 属性中
        - 自动追加：固定的内容会在对话摘要（`_handle_history_with_summary`）和历史记录（`_handle_history_with_file_upload`）中自动追加，确保重要信息持续可见
        - 追加格式：在摘要中以“## 用户的原始需求和要求”标题追加，在历史记录中以“## 用户强调的任务目标和关键信息”标题追加
    - 源码位置：builtin_input_handler模块
- **shell_input_handler**（Shell 输入处理器）
    - 职责：处理 Shell 命令（以 `!` 开头）
    - 源码位置：Shell输入处理器模块
- **file_context_handler**（文件上下文处理器）
    - 职责：处理文件引用（如 `@file.txt`），自动加载文件内容到上下文
    - 源码位置：文件上下文处理器模块
- **UserInteractionHandler**（用户交互封装）
    - 图型：管线式活动图（按序处理，遇 need_return 提前返回；多行输入签名兼容）
    - 聚焦：提前返回对主循环的影响；交互层可替换性（CLI→TUI/WebUI）；多行输入与确认回调封装
    - 源码位置：UserInteractionHandler模块

3.1.5 输出处理模块（Output Handlers）

- **OutputHandlerProtocol**（输出处理器协议）
    - 职责：定义输出处理器的接口协议
    - 源码位置：OutputHandlerProtocol模块 **3.1.7 分享管理模块（Share Management）**
- **ShareManager**（分享管理器基类）
    - 职责：提供工具、方法论、规则分享的通用逻辑（Git 仓库管理、资源选择等）
    - 源码位置：ShareManager模块
- **MethodologyShareManager**（方法论分享管理器）
    - 职责：管理方法论的分享流程
    - 源码位置：MethodologyShareManager模块
- **ToolShareManager**（工具分享管理器）
    - 职责：管理工具的分享流程
    - 源码位置：ToolShareManager模块
- **RuleShareManager**（规则分享管理器）
    - 职责：管理规则的分享流程，将本地规则分享到中心规则仓库（需配置 central_rules_repo）
    - 源码位置：RuleShareManager模块

3.1.8 辅助模块（Utilities）

- **prompts**（提示词定义）
    - 职责：定义默认提示词模板（DEFAULT_SUMMARY_PROMPT、SUMMARY_REQUEST_PROMPT 等）
    - 源码位置：提示词定义模块
- **prompt_builder**（提示词构建器）
    - 职责：构建工具使用提示（action_prompt）
    - 源码位置：提示词构建器模块
- **protocols**（协议定义）
    - 职责：定义接口协议（OutputHandlerProtocol 等）
    - 源码位置：协议定义模块
- **utils**（工具函数）
    - 职责：提供通用工具函数（如提示拼接函数）
    - 源码位置：工具函数模块
- **config_editor**（配置编辑器）
    - 职责：提供配置文件编辑功能
    - 源码位置：ConfigEditor模块
- **language_support_info**（语言支持信息）
    - 职责：提供编程语言支持信息（文件扩展名映射、语言特性等）
    - 源码位置：语言支持信息模块
- **language_extractors**（语言提取器）
    - 职责：提供不同编程语言的代码提取功能（目录结构）
    - 源码位置：`src/jarvis/jarvis_agent/language_extractors/`
- **dialogue_recorder**（对话记录器）
    - 职责：记录和管理所有用户-模型交互历史，支持多会话、JSONL格式存储、自动清理
    - 核心功能：
        - 自动生成会话ID，每个会话对应一个JSONL文件
        - 记录所有用户输入和模型响应，包含时间戳和元数据
        - 会话文件保存在 `~/.jarvis/data/dialogues/` 目录
        - 支持自动清理旧会话，避免占用过多存储空间
        - 大模型可以读取会话文件了解完整对话历史
    - 源码位置：`src/jarvis/jarvis_utils/dialogue_recorder.py`

3.1.9 平台层（Platform Layer）

- **PlatformRegistry**（平台注册表）
    - 图型：组件+流程（目录扫描→校验→注册→创建/普通平台获取/廉价平台获取/智能平台获取）
    - 聚焦：用户目录与内置目录合并策略；必需方法校验；多平台场景化选择
    - 说明：PlatformRegistry 位于 `jarvis_platform` 模块
    - 多平台支持：
        - 获取普通平台方法：获取普通平台，用于一般任务
        - 获取廉价平台方法：获取廉价平台，用于简单任务（查询重写、关键词提取、相关性评估等）
        - 获取智能平台方法：获取智能平台，用于复杂任务（代码生成、代码转换等）
- **BasePlatform**（平台基类）
    - 说明：BasePlatform 位于 `jarvis_platform` 模块，提供统一的 LLM 平台接口

3.1.10 入口模块（Entry Points）

- **jarvis**（主入口）
    - 职责：主命令的入口点，提供完整的 CLI 接口
    - 聚焦：命令行参数解析、配置管理、AgentManager 初始化
    - 源码位置：主入口模块
- **main**（代理入口）
    - 职责：`jarvis-agent` 命令的入口点，提供简化的 Agent 启动接口
    - 聚焦：Agent 定义文件加载、Agent 实例化、任务执行
    - 源码位置：代理入口模块

3.2 模块职责要点

核心模块（Agent Core）

- **Agent**（核心协调者）
    - 初始化并组装组件（EventBus/Managers/Handlers/Platform/Session），初始化顺序：EventBus → Managers → PromptManager → 系统提示设置
    - 设置系统提示，首轮按需进行工具筛选与文件/方法论处理
    - 将主运行循环委派给 AgentRunLoop
    - 在关键节点广播事件（TASK_STARTED、BEFORE/AFTER_MODEL_CALL、BEFORE/AFTER_HISTORY_CLEAR、BEFORE/AFTER_ADDON_PROMPT、BEFORE/AFTER_SUMMARY、BEFORE_TOOL_FILTER、TOOL_FILTERED、AFTER_TOOL_CALL、INTERRUPT_TRIGGERED）
    - 动态加载 after*tool_call 回调：扫描 after_tool_call_cb_dirs 配置指定的目录，支持三种导出形式（直接回调、get**工厂、register_工厂）
- **AgentRunLoop**（主循环执行体）
    - 驱动“模型思考 → 工具执行 → 结果拼接/中断处理 → 下一轮”的迭代
    - 统一处理工具返回协议与异常兜底，支持自动完成
    - 工具提醒机制：每工具提醒轮次间隔轮（默认20）注入工具使用提示
    - 自动摘要token控制：当剩余token低于输入窗口的25%时触发摘要与历史清理，重置对话长度计数
- **SessionManager**（会话状态）
    - 管理 prompt、附加提示、会话长度计数、用户数据；负责保存/恢复/清理历史
- **PromptManager**（提示管理）
    - 构建系统提示（系统规则 + 工具使用提示），构建默认附加提示（工具规范 + 记忆提示）
- **EventBus**（事件总线）
    - 提供 subscribe/emit/unsubscribe；同步回调异常隔离，不影响主流程

管理器模块（Managers）

- **AgentManager**（Agent 工厂与生命周期管理）
  - 负责 Agent 的构造、配置注入、会话恢复
  - 处理预定义任务加载（非交互模式跳过）
  - 支持任务内容直接传入（命令行参数）或交互式输入
- **TaskManager**（任务管理）
  - 从 `.jarvis/pre-command` 或数据目录的 `pre-command` 文件加载预定义任务
  - 提供任务选择界面（支持 fzf 和手动输入）
  - 支持为任务添加补充信息
- **MemoryManager**（记忆管理）
  - 记忆标签提示注入；关键事件驱动下进行记忆整理/保存；与 save/retrieve/clear_memory 工具协作
- **TaskAnalyzer**（任务分析）
  - 任务完成阶段旁路分析与满意度收集；必要时沉淀方法论
- **FileMethodologyManager**（文件与方法论）
  - 基于平台能力选择“文件上传模式”或“本地模式”；加载/上传方法论；上下文溢出时以文件方式转移历史

工具执行模块（Tool Execution）

- **execute_tool_call**（工具执行统一入口）
  - 解析模型响应中的工具调用，仅允许单次调用；执行前确认、执行后回调、长输出处理（上传或智能截断）等
  - 多处理器冲突检测：当多个输出处理器同时可处理响应时，返回错误提示，要求模型明确选择
  - 执行确认机制：通过工具执行确认参数（`execute_tool_confirm`）控制是否在执行前进行用户确认；默认值为 False（不进行确认，直接执行）
- **ToolRegistry**（工具注册表，默认输出处理器）
  - 发现/加载/执行工具（内置、外部 .py、MCP）；解析 TOOL_CALL，执行并返回标准化结果
  - 文件编辑/重写能力（通过 edit_file 工具提供，支持search/replace编辑和整文件重写）

输入处理模块（Input Handlers）

- **输入处理器链**（builtin_input_handler/shell_input_handler/file_context_handler）
  - 按序处理用户输入，遇 need_return 提前返回
  - 内置输入处理器：处理内置命令（如提示命令、清除命令等）
  - shell_input_handler：处理 Shell 命令（以 `!` 开头）
  - file_context_handler：处理文件引用（如 `@file.txt`），自动加载文件内容到上下文
- **UserInteractionHandler**（用户交互封装）
  - 封装多行输入与确认回调，便于未来替换为 TUI
  - 多行输入兼容性：优先使用带空行打印参数的签名，失败时回退为单参数签名
  - 确认回调委派：保持与原有用户确认方法一致的行为

输出处理模块（Output Handlers）

- **OutputHandlerProtocol**（输出处理器协议）
  - 定义输出处理器的标准接口，支持扩展自定义输出处理逻辑

分享管理模块（Share Management）

- **ShareManager**（分享管理器基类）
  - 提供工具和方法论分享的通用逻辑（Git 仓库管理、资源选择、提交推送等）
- **MethodologyShareManager/ToolShareManager**（具体分享管理器）
  - 分别管理方法论和工具的分享流程，继承自 ShareManager

平台层（Platform Layer）

- **PlatformRegistry/BasePlatform**（平台/模型）
  - 屏蔽不同 LLM 服务商差异；Agent 通过统一接口 chat_until_success/set_system_prompt/upload_files 等进行交互

入口模块（Entry Points）

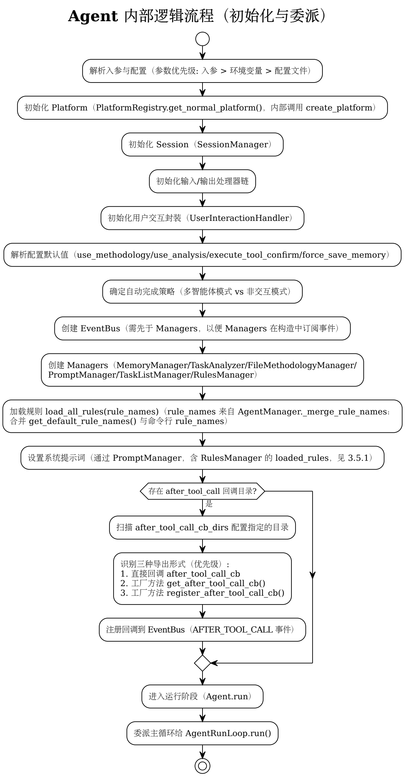
- **jarvis**（主入口）
  - 主命令的入口点，提供完整的 CLI 接口（配置管理、数据备份/恢复、Web 模式等）
- **main**（代理入口）
  - `jarvis-agent` 命令的入口点，提供简化的 Agent 启动接口，支持 Agent 定义文件加载

3.3 读者使用建议

- 先读“模块职责要点”，再看“对应图”，如需实现细节再去源码。文档图用于建立“概念模型”，不替代源码阅读。
- 按模块分类阅读：核心模块 → 管理器模块 → 工具执行模块 → 输入/输出处理模块 → Web 支持模块 → 其他辅助模块
- 重点关注模块间的依赖关系：Agent 作为协调中枢，通过 EventBus 实现解耦，通过 Registry 实现可插拔能力

3.4 Agent 设计

- 内部实现结构（PlantUML）



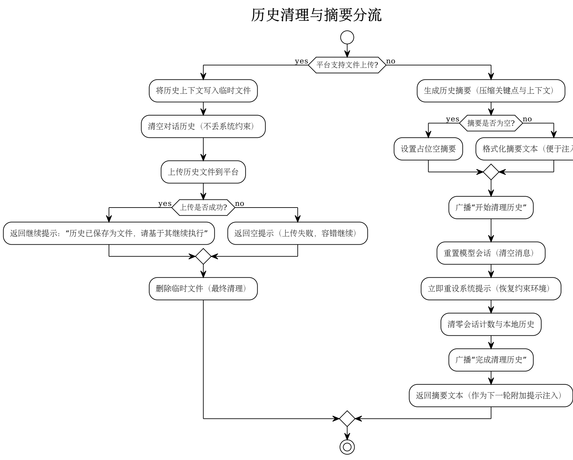
- 职责与定位：
  - 轻量协调者：初始化组件、构建系统/附加提示、委派主循环、广播事件
  - 通过 Registry 与事件总线实现可插拔能力与旁路扩展
- 核心方法：
  - **init**: 解析参数与配置；初始化 Platform/Session/Handlers/Managers（含 RulesManager）；load_all_rules(rule_names) 加载规则（rule_names 来自配置与命令行合并）；设置系统提示；加载 after_tool_call 回调
  - run/_main_loop: 进入主循环，委派 AgentRunLoop
  - _call_model/_invoke_model: 输入处理、附加提示拼接、上下文计数与模型调用（含 BEFORE/AFTER_MODEL_CALL 事件）
  - _call_tools: 工具执行委派至 execute_tool_call
  - _complete_task: 处理总结与任务完成事件，触发记忆/分析旁路
  - _filter_tools_if_needed: 工具超过阈值时使用临时模型筛选并重设系统提示
  - _summarize_and_clear_history: 上下文过长的摘要/文件上传分流与历史清理
- 关键参数影响行为：auto_complete、need_summary、use_methodology、use_analysis、execute_tool_confirm、force_save_memory、non_interactive、in_multi_agent、use_tools/files 等
- 小型结构图：

- 历史清理与摘要分流（Agent）

总结流程中的 Git diff 传递优化：

- **缓存机制**：AgentRunLoop 在总结触发前获取并缓存 git diff（存储在 `_git_diff` 属性中），避免在总结时重复获取
- **智能获取时机**：
  - 自动总结触发时（token 超限或对话轮次超限）：在调用 `_summarize_and_clear_history()` 前获取 git diff
  - 主动总结标记触发时（`<!!!SUMMARY!!!>`）：同样在总结前获取 git diff
  - 仅对 CodeAgent 类型获取（检查 `start_commit` 属性是否存在且非空）
- **Token 限制处理**：
  - 通过 `_check_diff_token_limit` 方法检查 diff 内容的 token 数量

    - 限制在输入窗口的 10%（`max_diff_tokens = int(max_input_tokens * 0.1)`）
    - 超出限制时进行智能截断，保留前面的行并添加截断提示信息
- **总结集成**：
    - Agent 的 `_handle_history_with_summary` 方法从 AgentRunLoop 获取 git diff
    - 获取策略：AgentRunLoop 在总结触发时（自动总结或主动总结标记）会先获取并缓存 git diff，然后在 `_handle_history_with_summary` 中优先使用已缓存的 diff（`get_cached_git_diff()`），如果缓存不存在则调用 `get_git_diff()` 重新获取
    - 将 git diff 格式化为代码块添加到摘要中：`## 代码变更摘要\n```\n{git_diff_info}\n```\`
- **错误处理**：获取失败时返回错误信息字符串，不影响主流程继续执行
- **Agent 引用存储**：Agent 存储 `_agent_run_loop` 引用（在 `run()` 方法中设置），便于在总结时访问缓存的 diff

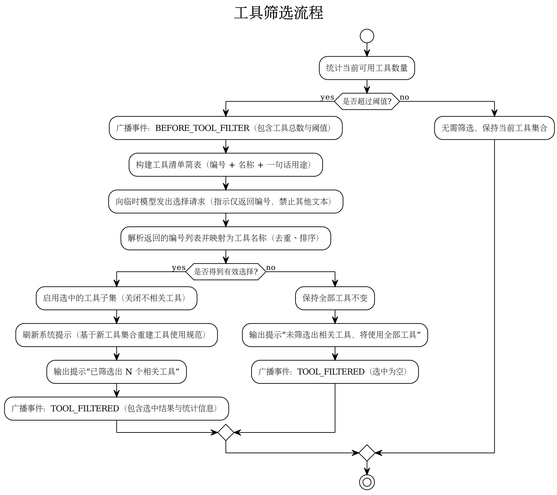


图示说明

- 分流依据：根据平台是否支持文件上传选择路径；支持时将历史写为文件上传；不支持则生成摘要
- 关键约束：清理历史后必须重设系统提示，避免丢失行为规范与工具约束

- 失败回退：上传失败返回空提示；摘要为空则使用占位文本；流程不中断

- 工具筛选流程（Agent._filter_tools_if_needed）



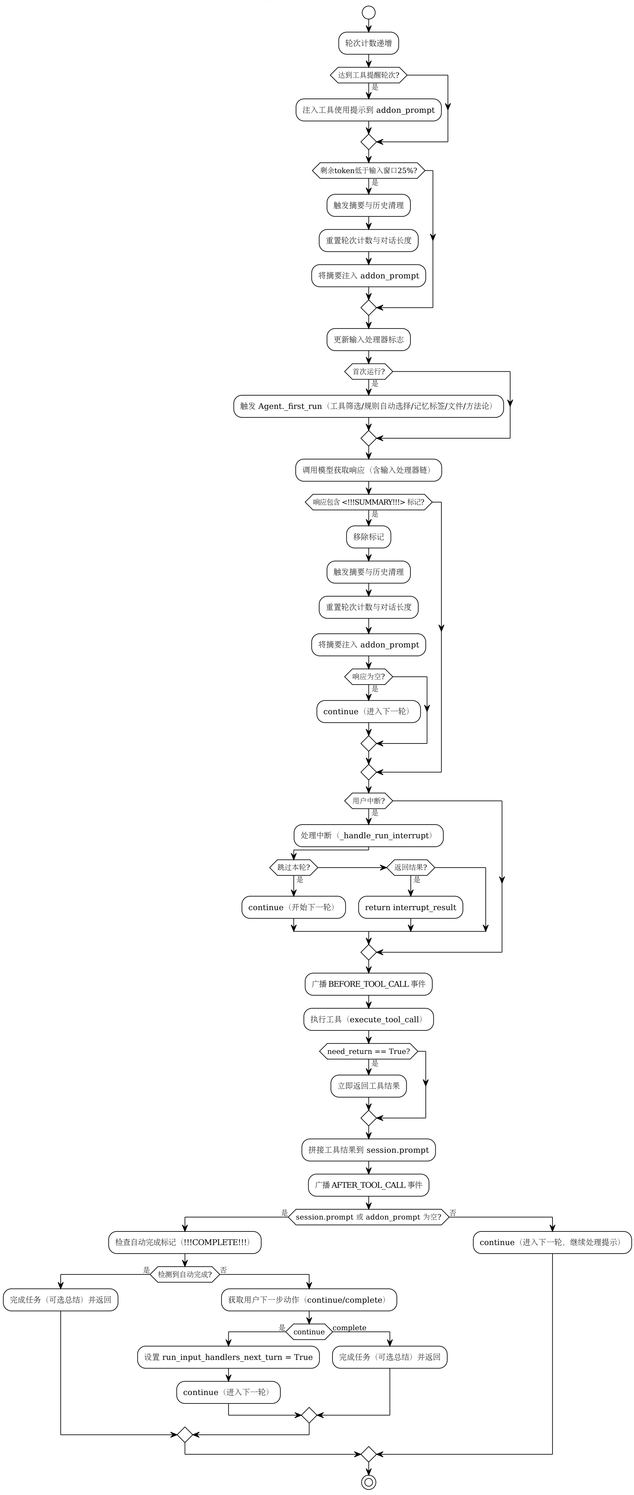
图示说明

- 触发条件：仅当可用工具数量超过阈值时进行筛选；否则保持原集合
- 选择方式：向临时模型请求"只返回编号"，再映射为工具名；启用子集后需刷新系统提示以降低决策复杂度
- 容错路径：未选出任何工具时维持原集合并继续任务

3.5 AgentRunLoop 设计

- 职责：承载主运行循环；控制迭代、工具执行、拼接提示、处理中断与完成
- 核心流程：
    - 首轮初始化（由 Agent._first_run 触发）后循环执行：
        1. 轮次计数递增（conversation_rounds，用于工具提醒）
        2. 工具提醒机制：每工具提醒轮次间隔轮（默认20）注入工具使用提示
        3. 自动摘要检查：当剩余token低于输入窗口的25%时触发摘要与历史清理，重置对话长度计数
            - **Git diff 集成优化**：在触发总结前，如果 Agent 是 CodeAgent 类型（有 `start_commit` 属性），自动获取并缓存 git diff 信息
        4. 更新输入处理器标志（run_input_handlers_next_turn）
        5. 首次运行处理（Agent._first_run：工具筛选、规则自动选择、记忆标签提示注入、文件/方法论处理）
        6. _call_model → 获取响应（含输入处理器链处理）
        7. 检查响应中的 <!!!SUMMARY!!!> 标记（`ot('!!!SUMMARY!!!')` 是等价的封装形式）：如果检测到该标记，触发摘要与历史清理，移除标记后继续处理响应
            - **Git diff 集成优化**：主动总结标记触发时，同样会获取并缓存 git diff 信息（仅对 CodeAgent 类型）
        8. _handle_run_interrupt → 处理用户中断（INTERRUPT_TRIGGERED）
        9. execute_tool_call → 执行工具（若识别到 TOOL_CALL）
        10. join_prompts → 拼接工具结果到 session.prompt
        11. 自动完成检测（!!!COMPLETE!!!）→ _complete_task
        12. 获取下一步用户动作（continue/complete）→ 继续或完成
- 事件：在工具与模型调用关键节点广播 BEFORE/AFTER_TOOL_CALL
- 返回协议：当工具返回 need_return=True，立即返回当前上下文；否则继续循环
- 关键参数：
    - `conversation_rounds`：当前对话轮次计数（用于工具提醒）
    - `tool_reminder_rounds`：工具提醒轮次间隔（环境变量 JARVIS_TOOL_REMINDER_ROUNDS，默认20）
    - `summary_remaining_token_threshold`：自动摘要触发阈值（剩余token低于输入窗口的25%，基于剩余token数量）
    - `_git_diff`：缓存的 git diff 内容，避免重复获取
- **Git diff 传递优化机制**：
    - **缓存策略**：AgentRunLoop 在总结触发时（调用 `_summarize_and_clear_history()` 之前）获取 git diff 并存储在 `_git_diff` 属性中，然后在 `_handle_history_with_summary` 中复用，避免重复计算
    - **智能获取**：仅在 CodeAgent 类型且存在 `start_commit` 时获取 git diff，普通 Agent 跳过此步骤
    - **Token 限制处理**：通过 `_check_diff_token_limit` 方法检查 diff 内容的 token 数量，限制在输入窗口的 10%（`max_diff_tokens = int(max_input_tokens * 0.1)`，代码注释说明"预留一部分token用于其他内容，使用10%作为diff的限制"），超出时进行截断处理
    - **获取时机**：在两种场景下获取 git diff：
        1. 自动总结触发时（token 超限或对话轮次超限）：在调用 `_summarize_and_clear_history()` 之前获取并缓存
        2. 主动总结标记触发时（`<!!!SUMMARY!!!>`）：在调用 `_summarize_and_clear_history()` 之前获取并缓存
    - **总结时获取策略**：在 `_handle_history_with_summary` 方法中，优先使用已缓存的 diff（`get_cached_git_diff()`），如果缓存不存在则调用 `get_git_diff()` 重新获取
    - **错误处理**：获取失败时返回错误信息字符串，不影响主流程继续执行
    - **Agent 引用存储**：Agent 存储 `_agent_run_loop` 引用，便于在总结时访问缓存的 diff
- 主动总结机制：
    - 模型可以在响应中输出 `<!!!SUMMARY!!!>` 标记（`ot('!!!SUMMARY!!!')` 是等价的封装形式）来主动触发总结
    - 系统检测到标记后：移除标记、触发 `_summarize_and_clear_history()`、重置计数器、将摘要注入下一轮的附加提示
    - 适用于部分任务已完成、之前的上下文价值不大的场景，允许模型主动清理历史以开始新的任务阶段

AgentRunLoop 核心决策路径（聚焦关键分支）



3.5.1 Rule 设计说明

Rule 系统用于将“行为约束、编码规范、流程定义”等以规则文本形式注入 Agent 的系统提示或单轮输入，使模型在对话中持续遵循或按需加载规则。

设计目标与定位

- **统一规则入口**：规则内容通过 RulesManager 加载与合并，由 PromptManager 写入系统提示的 `<loaded_rules>` 块，与系统提示词、工具提示等一起下发给模型。
- **多来源、可扩展**：支持项目/全局/中心库的 `rules` 目录、内置规则（`builtin/rules/` 及 `builtin/rules/rule.md`）及配置的规则目录；支持中心规则仓库（Git）的克隆与每日更新检查。
- **按需激活与运行时注入**：启动时通过 Agent 参数 `rule_names`（逗号分隔）指定要加载的命名规则；支持通过 `load_rule` 工具按文件路径加载并渲染规则，以及通过输入标记 `<rule:规则名>` 在当轮将规则内容注入用户输入。

规则来源与优先级

- **默认规则与按需激活**：`load_all_rules(rule_names)` 根据传入的 `rule_names`（逗号分隔）激活对应规则；未指定时返回空。常用规则名为 `builtin_rules`（`builtin/rules/rule.md`），调用方（如 Agent）可按需传入以加载。
- **命名规则解析（get_named_rule）**：规则名可带前缀以指定来源：`builtin:`、`project:`、`global:`、`central:`、`config:`；无前缀时按 `builtin/rules/rule.md` 索引、内置规则查找。
- **规则目录优先级（_get_all_rules_dirs）**：中心规则仓库（若配置）> 项目 `.jarvis/rules` > 配置的规则目录；同名规则以优先级高的为准。

RulesManager 职责与状态

- **加载与缓存**：从各来源读取规则内容，经 Jinja2 渲染（支持 current_dir、git_root_dir、jarvis_src_dir、jarvis_data_dir、rule_file_dir 等变量）后写入 `_loaded_rules`（name → content）。
- **激活与合并**：`rule_names` 中指定的规则通过 `activate_rule(name)` 加入 `_active_rules`；`deactivate_rule(name)` 可动态移除；`_merge_active_rules()` 将当前激活规则按名排序后拼接为 `_merged_rules`。
- **对外接口**：`load_all_rules(rule_names)` 返回 `(merged_rules, loaded_rule_names)`，供 Agent 初始化时得到 `loaded_rules`（字符串）与 `loaded_rule_names`；`get_named_rule(rule_name)` 供工具与输入处理器按名取内容；`get_all_available_rule_names()`、`get_all_rules_with_status()` 支持列表与状态展示。

规则注入流程

- **启动时**：Agent 构造时传入 `rule_names` → `_init_managers(rule_names)` → `RulesManager.load_all_rules(rule_names)` → 得到 `self.loaded_rules`（合并字符串）、`self.loaded_rule_names`；`_setup_system_prompt()` 中 PromptManager 将 `loaded_rules` 包在 `<loaded_rules>...</loaded_rules>` 中拼入系统提示。
- **运行时（输入侧）**：builtin_input_handler 识别用户输入中的 `'<rule:规则名>'`，调用 `_get_rule_content(rule_name)`（内部使用 RulesManager.get_named_rule）获取内容，将标记替换为 `<rule>...</rule>` 及分隔符后写入当轮输入，实现单轮按名注入。
- **运行时（工具侧）**：`load_rule` 工具接收 `file_path`，按路径读文件并用 `render_rule_template` 渲染后返回内容；不依赖规则名，适合项目内任意路径的规则文件。若存在 load_rule 工具，PromptManager 会在系统提示中追加 `<rule_usage_guide>`，引导模型在涉及规范、流程时使用该工具。

内置输入标记与规则

- `<rule:规则名>`：在用户输入中书写该标记，会在当轮被替换为对应规则的完整内容（含可选路径注释），便于临时引入某条命名规则。
- `<ListRule>`：触发后调用 `rules_manager.get_all_rules_with_status()`，以表格形式输出所有可用规则及其状态（已激活/未激活）、预览与路径，便于查看与选规则。

规则文件与规范

- 项目规则建议放在 `.jarvis/rules/` 下（可按子目录分类）；全局规则在 `~/.jarvis/rules/`；内置规则在 Jarvis 的 `builtin/rules/`，通过 `builtin/rules/rule.md` 及 `get_builtin_rule` 暴露。
- 规则内容支持 Markdown 与 Jinja2 模板；新增规则可参考 `add_rule` 规范（如 builtin/rules/tool_config/add_rule.md），区分项目规则与全局规则、目录与命名约定。
- **与 Skills 生态兼容**：Jarvis 的 rule 格式与 Cursor 的 skills 格式兼容，可直接将 skills 生态中的规则作为 Jarvis 规则使用，实现无缝

继承。

规则自动选择流程

- **触发时机**：Agent 首次运行（`_first_run`）时，若 `enable_auto_rule_select=True` 且 `session.prompt` 非空，则调用 `auto_select_and_load_rules(task_description)`。
- **前置条件**：`_has_user_specified_rules()` 为 False 时执行；用户已通过命令行 `rule_names` 或输入标记 `<rule:xxx>` 指定规则则跳过。
- **两阶段流程**（RulesManager.select_rule_by_task + _filter_rules_by_content）：
  1. **规则选择**：调用 `get_all_available_rule_names()` 获取可用规则索引，构造编号列表，通过临时模型根据任务描述选择最匹配的规则（最多 5 个），返回序号并映射为规则名。
  2. **内容过滤**：对选中规则加载内容，通过 `_filter_rules_by_content` 让模型根据任务与规则内容过滤不相关规则，得到最终规则列表。
- **激活**：对过滤后的规则依次调用 `activate_rule(name)` 激活，合并到 `_merged_rules` 并刷新系统提示。

小结

- Rule 系统通过 RulesManager 统一多来源规则的加载、激活与合并，经 PromptManager 注入系统提示，并通过 `<rule:xxx>` 与 load_rule 工具支持按名/按路径的运行时注入，与 ListRule 一起提供可观测与可扩展的规则使用方式。

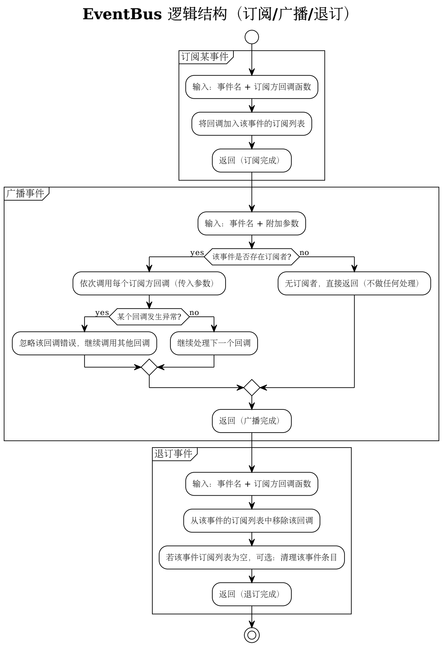
3.6 SessionManager 设计

读者要点

- 会话语义：清理历史仅重置对话与计数，但保留并立即重新应用系统提示，确保约束持续生效
- 保存/恢复作用域：基于平台名与模型名生成唯一文件名（格式：`saved_session_{agent_name}_{platform_name}_{model_name}.json`），避免跨平台/模型污染；恢复后自动删除文件
- 使用场景：长对话需释放上下文时的重置；跨运行的会话持久化与恢复
- 风险与约束：清理后必须重新设置系统提示；恢复失败或文件缺失时应平稳回退，不影响主流程
- 数据存储：当前提示字段（当前提示）、附加提示字段（附加提示）、会话长度计数字段（会话长度计数）、用户自定义数据字段（用户自定义数据字典）

以下为 **EventBus** 的逻辑结构（事件总线为 Agent 核心组件，与 SessionManager 同属“模块功能说明”中的核心/管理器类，此处一并编排）：

- 内部逻辑结构（PlantUML）



图示说明

- 目的：以“事件名”为索引，支持外部模块在关键节点进行旁路扩展（统计、记忆、分析等），不干扰主流程
- 异常处理：广播过程中单个订阅回调失败被忽略并继续执行其他订阅者，保证主流程稳定
- 边界：EventBus 不做业务判断与流程控制，仅负责调用订阅者；是否保存记忆/执行分析由订阅者自行决定

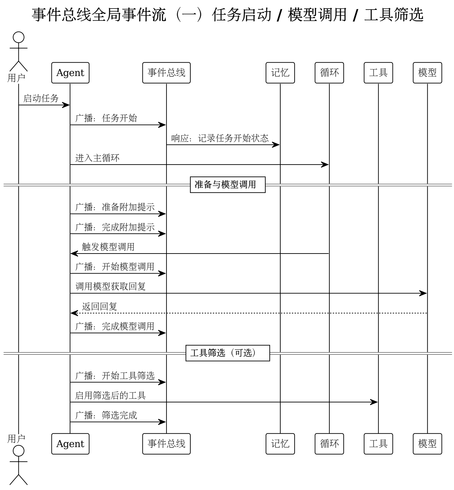
- 使用建议：为每类关键节点定义清晰事件名（如 BEFORE_MODEL_CALL、AFTER_TOOL_CALL 等），订阅者内部做好容错与幂等

- API：subscribe(callback)、emit(event, **kwargs)、unsubscribe(callback)

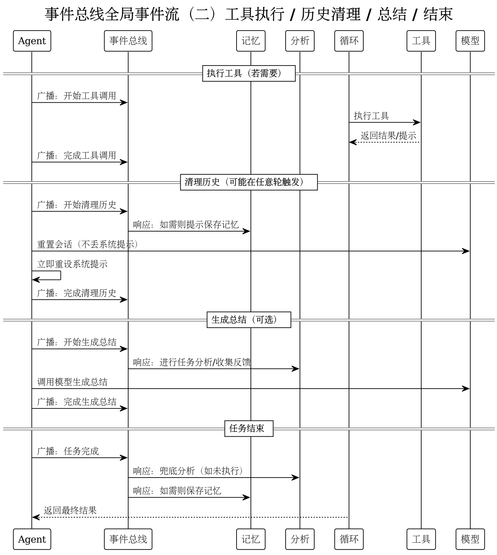
- 特性：同步广播、回调异常隔离，便于旁路扩展（记忆保存、任务分析、统计）

- 事件总线全局事件流（总览图，分两部分展示）

图：事件流（一）任务启动、模型调用与工具筛选



图：事件流（二）工具执行、历史清理、总结与任务结束



图示说明

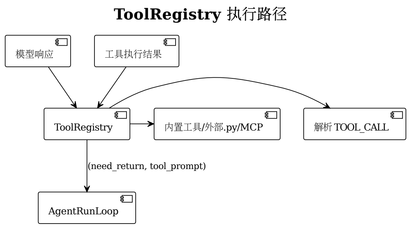
- 图使用“广播/响应/调用/返回”等通俗术语；不展示内部函数名与具体实现
- “可选”表示仅在特定条件触发（如工具筛选、生成总结）
- 实际事件名在代码中对应 TASK_STARTED、BEFORE/AFTER_MODEL_CALL、BEFORE/AFTER_TOOL_CALL、BEFORE/AFTER_HISTORY_CLEAR、BEFORE/AFTER_SUMMARY 等；图强调流程含义，避免细节干扰

3.7 execute_tool_call 设计

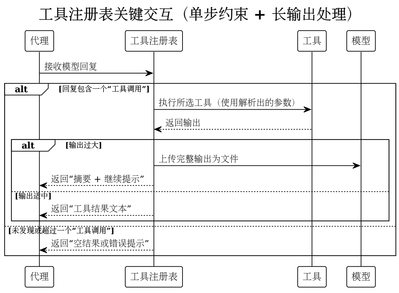
- 职责：统一解析模型响应中的工具调用，选择并执行合适的输出处理器，返回标准协议
- 核心流程：
  1. 遍历 Agent 的输出处理器列表，筛选可处理响应的处理器（can_handle）
  2. 多处理器冲突检测：若多个处理器同时可处理，返回错误提示，要求模型明确选择
  3. 无处理器匹配：返回空结果，继续循环
  4. 执行前确认：若工具执行确认参数为 True，进行用户确认；用户拒绝则返回空结果
  5. 执行处理器：调用处理器处理方法
  6. 返回标准协议：`(need_return: bool, tool_prompt: Any)`
- 返回协议：
  - `need_return=True`：工具要求立即结束本轮，直接返回结果给用户
  - `need_return=False`：工具结果将拼接回上下文，继续下一轮循环
  - 工具提示字段：工具执行结果文本或返回对象
- 异常处理：工具执行异常被捕获并返回错误信息，不影响主循环稳定

3.8 ToolRegistry 设计

- 角色：默认输出处理器，解析并执行 TOOL_CALL
- 加载来源：内置工具、外部 .py 工具、MCP 工具（外部进程）
- 执行协议：
  - 单步约束：一次响应仅允许一个调用块，检测到多个则拒绝执行
  - 结束标签容错：缺失结束标签时尝试自动补全并提示规范
  - 大输出处理：平台支持时上传文件并清理历史；否则智能截断（前后各 30 行）
  - 统计与记录：执行计数与最近执行工具记录（**last_executed_tool/executed_tools**）
- 微结构图：



- 关键交互时序（PlantUML）



图示说明

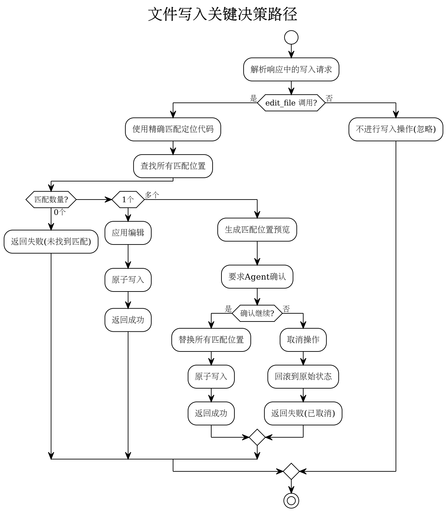
- 单步约束：每次回复仅允许一个工具调用；多个调用直接拒绝并提示规范
- 长输出处理：优先按平台能力上传文件并提供简短摘要；不支持上传时进行智能截断（保留头/尾片段）
- 返回约定：统一返回“是否需立即结束本轮 + 文本提示”，主循环据此决定直接返回或继续迭代

3.9 文件编辑工具（edit_file）设计

读者要点

- 用途：通过工具调用进行最小必要变更（edit_file）或整文件替换（edit_file 通过空 search 参数实现）
- 编辑模式：
  - edit_file：基于 search/replace 的普通文本编辑（使用精确字符串匹配）
    - **重要**：search 必须提供足够的上下文来唯一定位目标位置，避免匹配到错误的位置
    - **多处匹配智能确认**：当 search 文本存在多处匹配时，自动生成预览并要求 Agent 确认是否继续
    - **移除 count 参数**：不再支持通过 count 参数控制替换次数，改为通过多处匹配检测和确认机制
- 安全约束：
  - 编辑工具：支持原子写入和失败回滚
  - edit_file 的整文件重写模式采用原子写并可回滚
- 选择策略：优先使用 edit_file 的 search/replace 模式，仅在大范围重构或整文件生成时使用 edit_file 的空 search 参数功能
- 结果呈现：返回成功与差异摘要；失败时明确说明原因（包括详细的错误提示和修复建议）

关键决策路径（活动图）

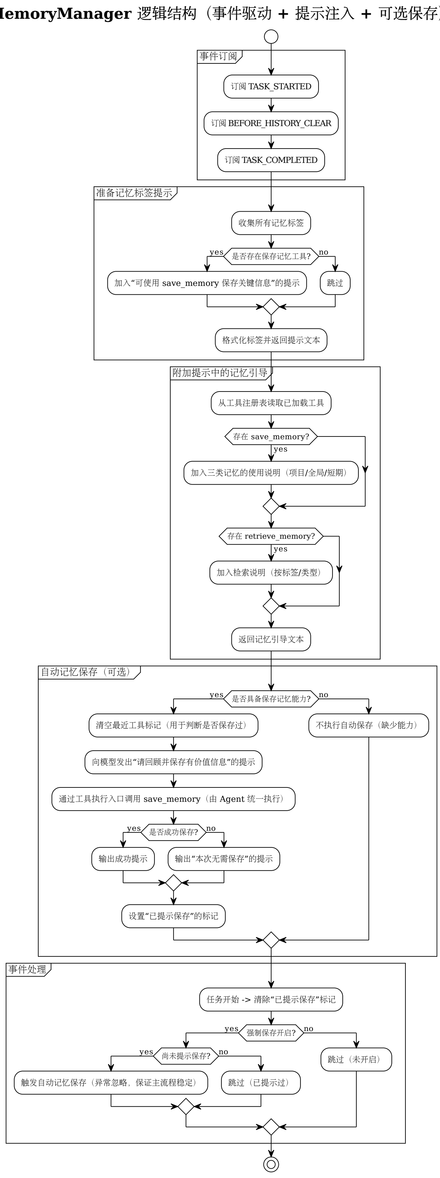


3.10 UserInteractionHandler 设计

- 职责：封装用户交互接口（多行输入与确认），便于未来替换为 TUI/WebUI
- 设计目标：抽象用户交互层，保持向后兼容，支持不同 UI 实现
- 核心方法：
  - 多行输入方法（提示字符串，空行打印标志）返回字符串：多行输入封装
    - 优先使用带空行打印参数的签名
    - 失败时回退为单参数签名（多行输入器方法）
  - 确认方法（提示字符串，默认值）返回布尔值：确认回调委派
    - 直接调用传入的确认回调，保持行为一致
- 初始化时机：在 Agent 初始化输入/输出处理器后创建，将 Agent 的确认回调指向封装后的方法

3.11 MemoryManager 设计

- 内部逻辑结构（PlantUML）



图示说明

- 事件驱动：在任务开始、历史清理前、任务完成后按需触发"提示或执行记忆保存"，不阻塞主流程
- 提示注入：将记忆标签与工具使用说明拼入每轮附加提示，促进模型主动使用记忆工具
- 能力门控：仅在存在 save_memory/retrieve_memory 工具时提供对应指引与自动保存；否则安全跳过
- 强制保存：force_save_memory 开启且尚未提示时，在 BEFORE_HISTORY_CLEAR 或 TASK_COMPLETED 触发一次自动保存

- 容错：自动保存调用异常被忽略，不影响任务输出与流程推进；状态标记避免重复提示

- 记忆类型与存储策略
- 短期记忆（short_term）
  - 存储位置：进程内存（jarvis_utils.globals.add_short_term_memory / get_short_term_memories）
  - 生命周期：仅在当前任务/进程内有效，不落盘；适合临时上下文、当前步骤的中间信息
  - 检索方式：retrieve_memory memory_types=["short_term"]，可按标签过滤
- 项目长期记忆（project_long_term）
  - 存储位置：当前项目目录 .jarvis/memory 下，JSON 文件按条目存储
  - 适用内容：项目相关的约定、配置、实现细节、架构决策等
  - 作用域：当前仓库/目录；随项目版本控制与协作共享更方便
- 全局长期记忆（global_long_term）
  - 存储位置：数据目录 get_data_dir()/memory/global_long_term 下，JSON 文件按条目存储
  - 适用内容：通用方法论、常用命令、用户偏好、跨项目知识与最佳实践
  - 作用域：同一用户在本机的所有项目/任务通用

数据模型（统一结构）

- 字段：id、type（memory_type）、tags、content、created_at、updated_at（可选）、merged_from（整理后可选）

- 命名与ID：save_memory 按微秒级时间戳生成唯一 ID（例如 20251101_072947_388226）

- 记忆整理（MemoryOrganizer）
- 目标：合并标签高度重叠的长期记忆，消除冗余、提升可检索性
- 适用类型：project_long_term、global_long_term（不处理 short_term）
- 合并策略：

    - 基于标签重叠度（min_overlap ≥ 2），按重叠数量从高到低分组
    - 调用 LLM 将同组记忆合并为一个综合记忆（最近时间权重更高）
    - 生成 JSON 格式的，包含 content 与 tags；解析后写为新记忆，并删除原始条目
- 使用方法（CLI）：
    - 整理（模拟运行）：jarvis-memory-organizer organize –type project_long_term –dry-run
    - 整理（指定重叠数）：jarvis-memory-organizer organize –type global_long_term –min-overlap 3
    - 导出：jarvis-memory-organizer export output.json -t project_long_term -t global_long_term –tag API
    - 导入：jarvis-memory-organizer import memories.json –overwrite
- 平台选择：统一使用 normal 平台与模型（PlatformRegistry.get_global_platform_registry），支持通过 -g/–llm-group 覆盖模型组
- 数据安全与回滚：
    - 新记忆创建后再删除旧记忆文件，失败日志汇总输出
    - 导出/导入支持类型校验、标签过滤、覆盖策略；异常明确告警且不中断其他记录
- 典型使用场景
- 在任务完成前自动提示保存本次关键经验（force_save_memory 开启）
- 在复杂项目中将架构决策与约定沉淀为 project_long_term，跨项目方法论沉淀为 global_long_term
- 在对话中临时缓存当前轮上下文与关键结论为 short_term，便于下一步工具调用

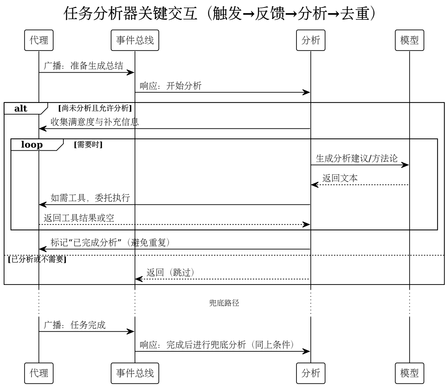
- 定期使用 MemoryOrganizer 合并重复/冗余的长期记忆，保持知识库整洁与高质

3.12 TaskAnalyzer 设计

读者要点

- 触发时机：生成总结前（BEFORE_SUMMARY）优先触发；若无总结需求，则在 TASK_COMPLETED 兜底触发
- 目标产出：基于任务过程沉淀方法论/改进建议，并收集“是否满意”的反馈
- 对主流程影响：旁路执行；回调异常或失败不影响任务完成；完成后设置去重标记避免重复分析
- 核心方法：
    - 任务分析方法（满意度反馈字符串）：执行任务分析循环
        - 构建分析提示（任务分析提示模板 + 满意度反馈）
        - 循环处理模型调用与工具执行（支持工具调用以完成分析任务）
        - 处理用户中断（采集补充信息后继续或退出）
    - 收集满意度反馈方法（自动完成标志）返回字符串：收集用户满意度
        - 仅在非自动完成且启用分析时收集
        - 询问用户是否满意，不满意时收集详细反馈
- 去重机制：通过分析完成标记和任务分析完成用户数据避免重复分析

关键交互（PlantUML）



图示说明

- 触发：生成总结前优先；无总结需求时在任务完成阶段兜底触发一次
- 产出：方法论与改进建议；同时收集满意度反馈
- 去重：设置“已完成分析”标记，避免重复分析
- 容错：分析过程中模型或工具出错忽略返回，不影响任务完成

输出与约束

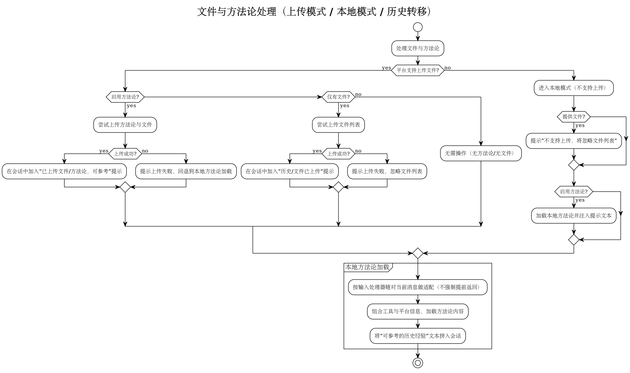
- 输出：方法论/改进建议（写入会话上下文，由上层继续整合），满意度反馈记录
- 约束：仅在 use_analysis 启用时执行；触发一次后设置去重标记
- 容错：模型/工具异常不影响主流程；中断时优先采集用户补充信息再继续分析

3.13 FileMethodologyManager 设计

- 模式选择：
    - 文件上传模式：平台支持 upload_files 时，方法论与历史以文件上传
    - 本地模式：不支持上传时，加载本地方法论库并管理上下文

- 历史处理：handle_history_with_file_upload 在上下文溢出时转移历史并返回继续提示

- 内部逻辑流程（PlantUML）

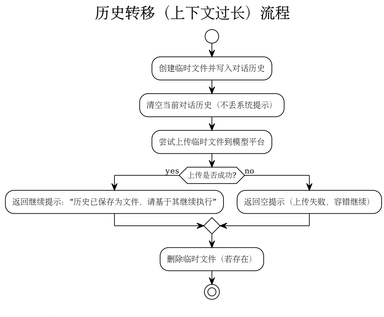


图示说明

- 模式选择：优先“上传模式”（平台支持时），否则“本地模式”加载方法论与经验
- 提示注入：成功上传或成功加载后，将“可参考经验/已上传文件”文本拼入会话，便于引用
- 失败回退：上传失败不阻断流程；方法论加载失败时回退为常规对话
- 输入处理器：本地方法论加载前对当前消息做轻量适配（如从文件/终端输入抽取上下文），不强制提前返回

- 无操作路径：既不启用方法论也无文件时，本轮不处理，保持最小影响

- 历史转移（上下文过长）流程



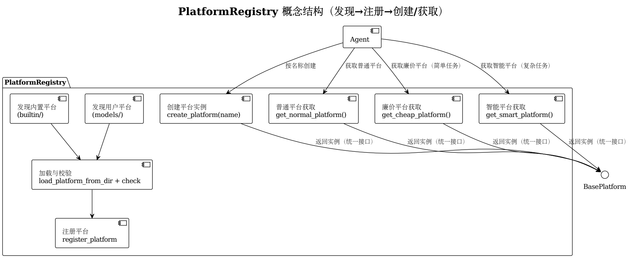
3.14 PlatformRegistry / BasePlatform 设计

- PlatformRegistry：动态创建平台实例，选择“普通平台/模型”，兼容不同厂商
- BasePlatform：统一接口（chat_until_success、set_system_prompt、upload_files、reset、set_model_name/group、name/platform_name）
- 行为：Agent 通过该层以统一方式与不同 LLM 平台交互，并按平台能力决定是否支持文件上传等增强特性

读者要点

- 边界与职责：统一发现与注册平台实现；向上提供“创建平台实例/获取普通平台”的简化接口
- 能力归一：通过 BasePlatform 接口屏蔽不同厂商差异（聊天/系统提示/文件上传等）
- 生命周期：启动时加载用户目录与内置目录的实现；运行时按名称创建实例或返回“普通平台”
- 失败回退：未找到平台或创建失败时进行明确告警并返回 None，不阻断上层流程

可视化（组件 + 关键流程）



PlatformRegistry 多平台支持说明：

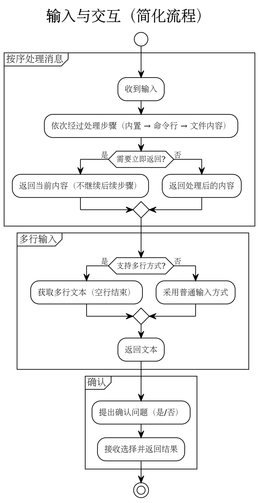
- **get_normal_platform()**：获取普通平台，用于一般任务场景
- **get_cheap_platform()**：获取廉价平台，用于对模型能力要求一般的任务（如查询重写、关键词提取、相关性评估、网页读取等）
- **get_smart_platform()**：获取智能平台，用于复杂任务场景（如代码生成、代码转换等）

平台选择策略：

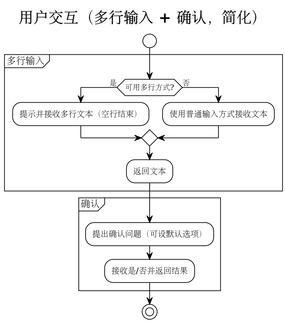
- 系统根据任务场景自动选择合适的平台
- 如果未配置对应的平台（cheap_platform/smart_platform），则回退到 platform 的值
- 支持在模型组（llm_groups，对象格式）中为每个组单独配置 cheap/smart 平台和模型，通过 normal_llm、cheap_llm、smart_llm 引用 llms 中定义的配置

3.15 输入处理器链与用户交互封装

- 内部逻辑流程（PlantUML）



- 输入处理器：builtin_input_handler、shell_input_handler、file_context_handler（按序处理，返回 need_return 标志控制是否提前返回）
- UserInteractionHandler：
  - 多行输入：兼容函数签名 func(tip, print_on_empty) 与 func(tip)
  - 确认交互：封装 confirm 回调，便于替换为 TUI/GUI/WebUI
- 用户交互封装（UserInteractionHandler）内部逻辑结构



- 目标：抽象用户交互（多行输入与确认），便于未来替换为 TUI/WebUI
- 兼容策略：优先使用带 print_on_empty 的多参签名，失败时回退为单参签名；confirm 委派保持一致行为

3.16 事件与回调扩展

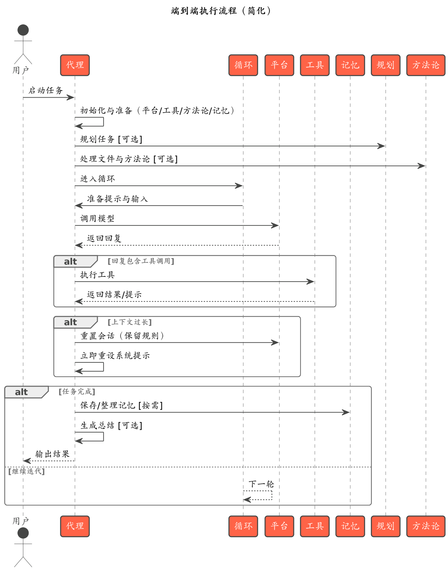
- 事件常量：TASK_STARTED/COMPLETED、BEFORE/AFTER_MODEL_CALL、BEFORE/AFTER_HISTORY_CLEAR、BEFORE/AFTER_SUMMARY、BEFORE_TOOL_FILTER/TOOL_FILTERED、AFTER_TOOL_CALL、INTERRUPT_TRIGGERED、BEFORE/AFTER_ADDON_PROMPT
- 动态回调：扫描 after_tool_call_cb_dirs 配置指定的目录；支持 after_tool_call_cb、get_after_tool_call_cb()、register_after_tool_call_cb() 三种导出形式；回调包装隔离异常

4. 与外部系统/环境的交互

- LLM 平台与模型
  - 通过 PlatformRegistry 动态创建 BasePlatform 实例，统一设置模型名称/组；Agent 使用 chat_until_success/set_system_prompt 调用
  - 支持文件上传：当模型支持 upload_files，历史与大输出以文件上传方式释放/节省上下文
- 工具生态
  - ToolRegistry 加载内置工具、外部 .py 工具与 MCP 工具；MCP 工具通过外部进程提供能力，支持跨应用集成
- 文件与方法论仓库
  - 加载本地方法论库；可与中心方法论库通过 Git 同步，实现团队共享最佳实践
- 记忆系统
  - 三层记忆：短期（内存）、项目长期（.jarvis/memory）、全局长期（~/.jarvis/data/memory/global_long_term）
  - 通过 save_memory/retrieve_memory/clear_memory 工具使用
- CLI 与环境变量/配置
  - 通过 jarvis_utils.config 获取默认配置（如计划开关、筛选阈值、after_tool_call 回调扫描目录等）
- 动态回调注入
  - 读取 after_tool_call_cb_dirs 配置指定目录下的 Python 文件，动态注册 after_tool_call 回调（三种导出方式优先级约定），用于旁路增强工具调用后处理
- 可观测性
  - show_agent_startup_stats 启动统计：输出方法论数量、工具可用数/总数、全局/项目记忆数量、工作目录等

5. 模块间交互流程（PlantUML）

下图展示一次典型端到端执行过程的时序，涵盖初始化、首轮处理、模型调用、工具执行与收尾。



6. 参数与配置说明

以下参数来自 Agent.**init**。默认值或行为参考 jarvis_utils.config 与内部回退逻辑。除特别标注外，布尔型参数可通过入参覆盖配置默认值。平台与模型由配置（如 get_normal_platform_name）决定，非 Agent 构造参数。

- system_prompt: 系统提示词，定义 Agent 行为准则（必要）
- name: Agent 名称，默认 “Jarvis”，用于全局登记与交互提示
- description: Agent 描述信息
- summary_prompt: 任务总结提示词；为空时回退 DEFAULT_SUMMARY_PROMPT 或 SUMMARY_REQUEST_PROMPT
- auto_complete: 自动完成开关；非交互模式默认开启；多智能体模式下仅在显式 True 时开启
- output_handler: 输出处理器列表；默认 [ToolRegistry]（包含 edit_file 等工具，支持整文件重写）
- use_tools: 指定允许使用的工具名白名单；为空时加载默认集合；当工具过多时 Agent 首轮可触发 AI 筛选并动态缩减
- execute_tool_confirm: 执行工具前是否进行用户确认（可由配置 is_execute_tool_confirm 决定）
- need_summary: 是否在完成阶段生成总结
- multiline_inputer: 多行输入函数；由 UserInteractionHandler 进行向后兼容封装
- use_methodology: 是否启用方法论引导；默认从配置读取（is_use_methodology）
- use_analysis: 是否启用任务完成后的分析（TaskAnalyzer）；默认从配置读取（is_use_analysis）
- force_save_memory: 是否在关键节点强制提示/执行记忆保存；默认从配置读取（is_force_save_memory）
- files: 需要处理或上传的文件列表（触发 FileMethodologyManager 处理）
- confirm_callback: 确认回调，签名 (tip: str, default: bool) -> bool；默认 CLI user_confirm
- in_multi_agent: 多智能体运行标志；用于控制自动完成（子 Agent 默认非交互自动完成）
- agent_type: “normal” 或 “code”；“code” 时构造 CodeAgent（转发构造参数）
- rule_names: 规则名称列表（逗号分隔），用于加载指定规则（见 3.5.1 Rule 设计说明）
- allow_savesession: 是否允许 SaveSession/RestoreSession 命令；默认 False，仅 jvs/jca 主程序传入 True
- optimize_system_prompt: 若为 True，在第一次 run() 时根据用户输入优化系统提示词
- non_interactive: 是否非交互模式（优先级最高，覆盖环境变量与配置）

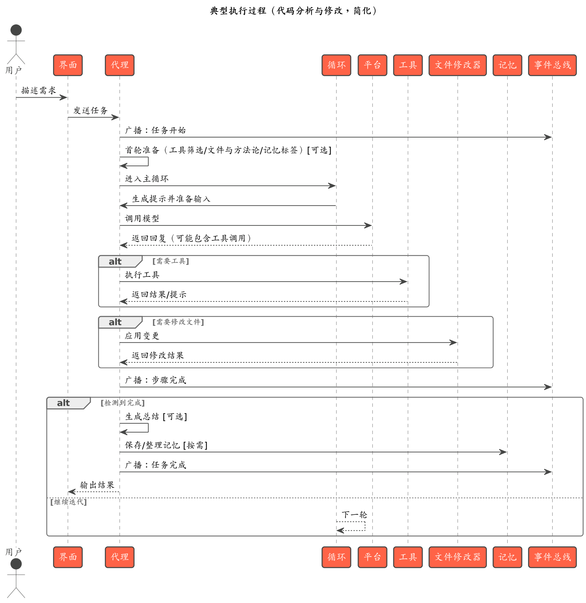
行为与默认策略补充

- 非交互与自动完成：
  - 非交互模式（或子 Agent）默认自动完成为 true；多智能体模式时除非显式 True 否则不自动完成
- 工具筛选：
  - 可用工具数超过阈值（get_tool_filter_threshold）时，使用临时模型产生选择编号，更新 ToolRegistry.use_tools，并重置系统提示
- 多平台场景化选择：
  - 系统根据任务场景自动选择合适的LLM平台：
    - **cheap平台**：用于对模型能力要求一般的任务（查询重写、关键词提取、相关性评估、网页读取等）
    - **normal平台**：用于一般任务场景（默认）
    - **smart平台**：用于复杂任务场景（代码生成、代码转换等）
  - 配置项：cheap_platform、cheap_model、smart_platform、smart_model（顶层配置），或通过 llm_groups（对象格式）中的 normal_llm、cheap_llm、smart_llm 引用 llms 中定义的配置
  - 如果未配置对应的平台/模型，则回退到 platform/model 的值
  - 支持在模型组（llm_groups，对象格式）中为每个组单独配置，通过引用 llms 中定义的配置实现
- 主动总结标记：
  - 模型可以在响应中输出 `<!!!SUMMARY!!!>` 标记（`ot('!!!SUMMARY!!!')` 是等价的封装形式）来主动触发总结并清空历史
  - 适用于部分任务已完成、之前的上下文价值不大的场景
  - 系统检测到标记后会：移除标记、触发总结、清空历史、重置计数器、将摘要注入下一轮的附加提示
  - 在总结流程中，系统会自动集成 Git diff 信息（如果可用），帮助模型了解代码变更历史
- 大输出处理：
  - 平台支持上传：生成摘要并清理历史后，上传大输出文件，返回“摘要 + 调用上下文”提示继续
  - 平台不支持：智能截断前 30/后 30 行，中间以占位提示
- 历史与上下文：
  - conversation_length 由 get_context_token_count 计数；超过阈值时走摘要或文件上传流程（FileMethodologyManager 处理）
- 动态回调：
  - 扫描 after_tool_call_cb_dirs 配置指定的目录，支持 after_tool_call_cb、get_after_tool_call_cb()、register_after_tool_call_cb() 三种导出形式；注入 AFTER_TOOL_CALL 回调

7. 典型执行过程（端到端）

以“分析代码并修改某个函数”为例（伪场景）：

1. CLI 将用户需求交给 Agent.run
2. Agent 已完成初始化（在 run 调用前）：已加载 Platform、ToolRegistry、系统提示等；run 被调用时进入主循环
3. Agent 注册与运行状态管理：
   - 注册逻辑：普通 Agent 在 run 方法开始时注册到全局注册表；CodeAgent 跳过注册（由 CodeAgent.run 自行管理）
   - 运行状态标记：调用 set_current_agent 标记 agent 开始运行
4. 首轮处理：
   - FileMethodologyManager：若平台支持上传，尝试上传 files 与方法论；否则本地加载
   - MemoryManager：准备记忆标签提示并注入上下文
   - 工具筛选（可选）：当工具过多时用临时模型筛选相关工具子集，更新系统提示
5. 进入 AgentRunLoop：
   - 生成 addon prompt（包含工具使用规则、记忆提示、是否需要 !!!COMPLETE!!! 标记、主动总结标记 <!!!SUMMARY!!!> 说明）
   - 调用 BasePlatform.chat_until_success 获取响应
   - 检查响应中的 <!!!SUMMARY!!!> 标记（`ot('!!!SUMMARY!!!')` 是等价的封装形式）：如果检测到，触发总结并清空历史，移除标记后继续处理
     - **Git diff 优化**：在总结前，如果是 CodeAgent 类型，自动获取并缓存 git diff（从 `start_commit` 到当前 commit）
   - 若响应包含 TOOL_CALL，交由 ToolRegistry 解析并执行对应工具（如 read_code 或 edit_file）
   - 工具输出拼接回上下文（utils.join_prompts），广播 AFTER_TOOL_CALL；need_return=True 时直接返回结果
   - 若用户中断（INTERRUPT_TRIGGERED），采集补充输入，决定是否继续或跳过当前轮
   - 检查自动完成：检测到 !!!COMPLETE!!! 标记（`ot('!!!COMPLETE!!!')` 是等价的封装形式）进入收尾
6. 收尾：
   - 通过事件驱动执行 TaskAnalyzer 分析、MemoryManager 记忆保存（受 force_save_memory 控制）
   - need_summary=True 时生成总结，总结中自动集成 Git diff 信息（如果可用）
     - **Git diff 传递机制**：
       - AgentRunLoop 在总结触发时（调用 `_summarize_and_clear_history()` 之前）先获取并缓存 git diff
       - Agent 的 `_handle_history_with_summary` 方法从 AgentRunLoop 获取 git diff：优先使用已缓存的 diff（`get_cached_git_diff()`），如果缓存不存在则调用 `get_git_diff()` 重新获取
       - 通过 `_check_diff_token_limit` 方法限制 diff 大小（不超过输入窗口的 10%）
       - 将 git diff 格式化为代码块添加到摘要中：`## 代码变更摘要\n```\n{git_diff_info}\n```\`
       - 获取失败时返回错误信息，不影响主流程
     - **任务摘要增强**：在任务摘要中添加详细任务信息（通过 task_list_manager 获取任务列表摘要）
     - **用户固定内容追加**：如果存在 pin_content（通过 Pin 标记固定的用户输入内容），会在对话摘要和历史记录中追加，确保重要信息不被遗忘
   - 对话记录器自动保存会话历史到 `~/.jarvis/data/dialogues/` 目录
   - Agent 反注册与清理：
     - 普通 Agent：在 run 方法的 finally 块中反注册（调用 delete_agent）
     - CodeAgent：在 CodeAgent.run 的 finally 块中自行反注册（调用 delete_agent）
     - 运行状态清理：调用 clear_current_agent 清除运行状态标记
   - 返回最终结果至 CLI



该过程对长输出、上下文长度与外部失败具备防御性回退策略，保证流程可持续推进。

8. 可靠性与容错设计

- 模型空响应回退为空串并告警，避免 None/空字符串导致逻辑断流
- 摘要与完成阶段同样进行空响应防御
- 事件回调异常隔离，避免影响主流程
- 工具调用格式容错：ToolRegistry 对缺失结束标签的 TOOL_CALL 尝试自动补全并提示规范
- 长输出安全处理：优先文件上传，其次智能截断，抑制上下文溢出
- 历史清理后自动重置系统提示，保持约束环境持续生效

9. 扩展与二次开发建议

- 工具扩展：在内置路径或 ~/.jarvis/tools 下新增 .py 工具；跨进程/应用集成优先采用 MCP 工具
- 平台扩展：在 jarvis_platform 下新增 BasePlatform 子类，通过 PlatformRegistry 自动发现
- 旁路增强：通过 after_tool_call_cb_dirs 配置注入 AFTER_TOOL_CALL 回调，实现统计/审计等旁路能力
- 方法论共享：建立中心方法论库（Git），团队同步沉淀最佳实践，提升协作效率
- 子 Agent：利用 _build_child_agent_params 继承父 Agent 能力，构建递归执行的子任务体系
- UI 替换：UserInteractionHandler 与 OutputHandlerProtocol 的抽象便于替换为 TUI/GUI/WebUI

10. TaskListManager 运行流程详解

TaskListManager 是 Jarvis 系统中用于管理复杂任务分拆、执行和状态跟踪的核心工具，支持多任务动态管理、依赖关系处理、任务验证和状态自动管理。本节详细描述其运行流程和关键机制。

10.1 核心操作流程

操作类型：

1. `add_tasks`：批量添加任务（推荐在 PLAN 阶段使用）
2. `execute_task`：执行任务（自动创建子 Agent）
3. `update_task`：更新任务状态和信息
4. `get_task_detail`：获取任务详情
5. `get_task_list_summary`：获取任务列表摘要

10.2 任务执行流程

执行前检查（`_handle_execute_task`）：

1. **任务存在性验证**：检查任务是否存在于任务列表中
2. **参数完整性检查**：验证 `additional_info` 参数是否非空（强制要求）
3. **依赖关系验证（`_validate_dependencies_status`）：**
   - 检查所有依赖任务是否存在
   - 验证依赖任务是否已完成（状态必须为 `completed`）
   - 如果有依赖任务失败或未完成，拒绝执行并返回详细错误信息
4. **任务状态检查**：确保任务状态为 `pending`（只有待执行的任务可以执行）
5. **运行中任务互斥检查**：
   - 扫描任务列表，查找所有状态为 `running` 的任务
   - 如果存在运行中的任务，拒绝执行新任务，并返回运行中任务的详细信息（任务ID和名称）
   - 防止多任务并发执行导致的冲突和资源竞争

任务执行（执行检查通过后）：

1. **状态更新为 running**：将任务状态从 `pending` 更新为 `running`
2. **任务内容构建（`_build_task_content`）：**
   - 合并任务描述和附加信息（`additional_info`）
   - 构建标准化的任务内容格式，包含任务名称、描述、预期输出等
3. **背景信息构建（`_build_task_background`）：**
   - 收集任务列表的全局背景信息（`background`）
   - 收集已完成依赖任务的输出作为上下文
   - 构建完整的背景信息，供子 Agent 使用
4. **任务执行器选择**：
   - 根据任务类型（`main` / `sub`）选择执行方式
   - `main` 类型：在主 Agent 中直接执行（适用于简单任务）
   - `sub` 类型：创建新的通用 Agent 子实例执行（适用于复杂任务）
5. **任务执行**：调用对应 Agent 的 `run` 方法执行任务
6. **输出长度控制**：
   - 根据剩余 token 数量动态计算最大输出长度
   - 对过长输出进行智能截断（前缀80%，后缀20%）
   - 避免上下文溢出

任务完成后处理：

1. **自动状态更新**：
   - 任务执行成功：自动将状态更新为 `completed`
   - 任务执行失败：自动将状态更新为 `failed`
2. **任务验证**（针对 `main` 类型任务）：
   - 对于标记为 `completed` 的 `main` 类型任务，自动触发验证流程
   - 创建独立的验证 Agent，执行只读验证
   - 验证失败时提供详细的错误报告和修复建议

10.3 任务验证机制

验证流程（`_verify_task_completion`）：

1. **验证 Agent 创建（`_create_verification_agent`）：**
   - 创建独立的验证 Agent，配置为只读模式
   - 仅提供 `read_code`、`execute_script`、`read_file` 等只读工具
   - 禁止任何修改操作，确保验证过程的纯净性
2. **验证任务构建**：
   - 构建详细的验证任务描述，包含任务内容和背景信息
   - 要求验证 Agent 逐条验证预期输出：
     - 将预期输出解析为多条具体条目
     - 对每一条条目单独验证是否有对应的代码、文件或其他实际产物
     - 检查产物是否完整、是否符合描述
   - 明确验证标准：所有条目都验证通过才判定为整体通过
3. **验证结果解析**：
   - 从验证 Agent 的输出中提取结构化验证状态
   - 支持多种格式：`**验证状态**: [PASSED/FAILED]`、`**最终结论**: [VERIFICATION_PASSED/VERIFICATION_FAILED]`
   - 回退到关键词匹配：查找“验证通过”、“验证失败”等关键词
4. **验证失败处理**：
   - 保存完整的验证结果作为失败原因（不再提取“详细说明”片段，避免信息丢失）
   - 提供迭代次数、最后一次验证报告等详细信息
   - 在失败摘要中明确指出：允许提供修复建议，但禁止自动修复
5. **验证成功处理**：
   - 标记任务为真正完成
   - 在任务摘要中添加后续任务必要性评估提示
   - 提醒用户评估是否仍需执行后续依赖任务

10.4 任务状态更新流程

状态更新操作（`update_task` action）：

1. **权限验证**：检查调用者是否有权限更新任务
2. **状态一致性检查**：

    - 对于已完成的任务，避免重复验证主任务完成状态
    - 防止不必要的验证迭代
3. **自动验证触发**（针对 `main` 类型任务）：
    - 当 `main` 类型任务状态更新为 `completed` 时，自动触发验证
    - 确保任务真正完成，防止脏状态
4. **验证失败处理**：
    - 如果验证失败，将任务状态更新为 `failed`
    - 记录详细的失败原因和验证报告
    - 提供清晰的错误信息和修复建议

10.5 任务列表完成前检测

完成前检测逻辑（集成在 `run_loop` 中）：

1. **触发时机**：
    - 用户输入 `exit` 、 `quit` 等退出命令时
    - Agent 判断任务完成准备结束会话时
2. **检测逻辑**：
    - 扫描任务列表，查找所有未完成的任务（状态为 `pending` 或 `running` ）
    - 如果存在未完成任务，向用户展示详细提示
3. **提示信息**：
    - 显示未完成任务的数量和详细列表
    - 提供操作建议：继续执行任务、放弃任务、或强制退出
    - 防止意外结束会话导致任务丢失

10.6 错误处理和容错机制

依赖验证错误：

- `DependencyNotFoundError` ：依赖任务不存在
- `DependencyNotCompletedError` ：依赖任务未完成
- `DependencyFailedError` ：依赖任务失败
- 所有错误都返回详细的错误信息，帮助用户快速定位问题

验证异常处理：

- 验证过程中的任何异常都被捕获并记录
- 异常不会导致主流程崩溃
- 验证失败时提供详细的异常信息

输出长度控制：

- 动态计算基于剩余 token 的最大输出长度
- 回退策略：剩余 token → 输入窗口的 2/3 → 默认值（10000 字符）
- 智能截断：保留前缀 80% 和后缀 20%，中间部分用 `\n\n... [截断 XXX 字符] ...\n\n` 标记

10.7 任务类型选择建议

任务类型：

- `main` ：简单任务（1-3 步、单文件、主 Agent 可直接完成）
    - 由主 Agent 直接执行，无需创建子 Agent
    - 执行完成后自动触发验证，确保质量
    - 适用于：简单文件修改、配置更新、简单查询等
- `sub` ：复杂任务（多步骤、多文件、需要专门规划）
    - 自动创建子 Agent 执行
    - 子 Agent 继承父 Agent 的工具和配置
    - 适用于：复杂功能实现、多模块重构、系统设计等

关键原则：

- 简单任务必须使用 `main` ，避免过度拆分
- 复杂任务使用 `sub`
- 不要将本可以一次完成的任务拆分为多个子任务

10.8 最新改进总结

基于最近的提交（从 4dcab8d0 到 HEAD），TaskListManager 主要改进包括：

1. **任务验证逻辑优化**：
    - 移除验证结果中详细说明的提取逻辑，直接使用完整验证结果，避免信息丢失
    - 明确区分验证时的修复建议与禁止修复行为
    - 增强验证失败时的详细错误报告，包含迭代次数和最后一次验证报告
2. **任务执行安全性增强**：
    - 新增运行中任务互斥检查，防止并发执行冲突
    - 避免对已完成的任务重复验证主任务完成状态
    - 在任务列表完成前增加检测逻辑，防止意外退出
3. **代码重构与可维护性**：
    - 重构任务内容构建逻辑，提取 `_build_task_content` 和 `_build_task_background` 公共方法
    - 统一验证流程，在 `update_task_status` 中也支持自动验证
    - 优化字符串引号使用，提升代码一致性
4. **用户体验改进**：
    - 增加后续任务必要性评估提示
    - 提供更清晰的错误信息和操作建议
    - 改进任务状态展示和进度跟踪

10.9 配置与参数

任务定义参数：

- `task_name` ：任务名称（必填）
- `task_desc` ：任务描述（必填，必须包含约束条件、必须要求、禁止事项、验证标准）
- `expected_output` ：预期输出（必填，必须使用分条列出的结构化格式）
- `agent_type` ：Agent 类型（必填， `main` / `sub` ）
- `dependencies` ：依赖任务列表（可选，任务名称或任务ID）

全局参数：

- `main_goal` ：任务列表的核心目标（创建任务列表时必填）
- `background` ：所有子任务的公共背景信息（必须包含全局约束条件、必须要求、禁止事项、验证标准）
- `additional_info` ：任务执行时的附加信息（执行任务时必填，不能为空）

输出长度控制：

- 基于剩余 token 动态计算（默认使用剩余 token 的 2/3）
- 默认最大输出长度：10000 字符
- 截断策略：前缀 80%，后缀 20%

10.10 典型使用场景

1. **大型项目重构**：
    - 使用 `add_tasks` 批量创建重构任务
    - 设置任务间的依赖关系
    - 按依赖顺序逐个执行任务
    - 自动验证每个任务的完成情况
2. **模块化开发**：
    - 将大型功能拆分为多个独立模块
    - 每个模块作为独立任务
    - 利用依赖关系确保开发顺序
    - 验证每个模块的功能完整性
3. **数据处理流水线**：
    - 按数据处理步骤创建任务（采集→清洗→转换→存储）
    - 设置严格的依赖关系
    - 自动跟踪每个步骤的执行状态
    - 验证数据质量
4. **迁移项目**：
    - 按迁移步骤创建任务（分析→规划→转换→验证）
    - 使用 `sub` 类型任务处理复杂代码转换任务
    - 利用验证机制确保迁移质量
    - 跟踪迁移进度

CodeAgent 系统架构设计

本章节描述 CodeAgent 作为 Agent 的子类，在继承 Agent 核心能力的基础上，为“代码工程”场景提供的增强能力与流程封装。内容基于源码进行结构化说明，覆盖模块组成、职责与接口、关键交互流程、CLI 入口与参数说明、可靠性与扩展建议。

- 关联组件与工具：Git 提交工作流工具、工具注册生态（由 Agent 承载）、事件系统（AFTER_TOOL_CALL）、lint 工具建议（lint.py）、文件变更后处理工具（after_change.py）、内置规则系统（builtin_rules.py）、配置系统（jarvis_utils.config）、输出交互（PrettyOutput）
- 源码位置：`src/jarvis/jarvis_code_agent/code_agent.py::CodeAgent`

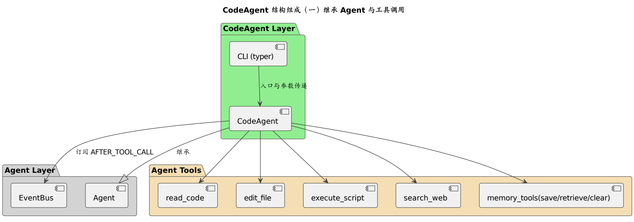
1. 设计目标与总体思路

- 继承 Agent：CodeAgent 直接继承 Agent 类，通过父类初始化方法初始化父类，获得 Agent 的所有核心能力（模型交互、工具调用、会话管理等）。
- 场景聚焦：围绕“代码分析与修改”的端到端流程，提供仓库管理、补丁预览、提交确认、静态检查引导等增值能力。
- 功能扩展：在 Agent 基础上扩展代码相关能力（上下文管理、影响分析、构建验证等），通过重写和扩展方法实现。
- 稳健与可观测：强调 git 配置/仓库状态检查、换行符策略、错误回退；大变更摘要化预览，避免上下文膨胀。

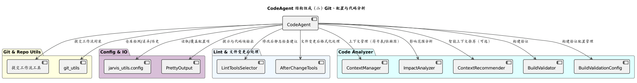
2. 模块组成（PlantUML）

下图分两部分展示 CodeAgent 与其协作组件的静态组成与依赖关系。

图：CodeAgent 结构组成（一）继承关系与工具层



图：CodeAgent 结构组成（二）Git / 配置 / 代码分析器



关键点

- CodeAgent 直接继承 Agent，通过父类初始化方法初始化父类，获得 Agent 的所有核心能力（模型交互、工具调用、会话管理等）。
- CodeAgent 在 Agent 基础上扩展代码相关能力，通过重写和扩展方法实现。
- AFTER_TOOL_CALL 用于在工具执行后进行旁路增强（展示 diff、提交、静态扫描引导、影响分析、构建验证、文件格式化）。
- Git 工具链与配置/输出等均为 CodeAgent 的扩展能力。
- 代码分析器模块提供代码结构分析、依赖关系管理、影响范围分析和智能上下文推荐等能力。
- 内置规则系统：规则以 Markdown 文件形式组织，支持动态加载，已优化为适合大模型的指令性格式。
- 文件变更后处理：支持根据文件类型自动执行格式化工具，确保代码风格一致。

3. CodeAgent 核心功能与扩展能力

读者要点

- CodeAgent 继承 Agent 的所有核心能力：
  - 模型交互：通过模型属性访问 LLM 模型
  - 工具调用：通过工具调用方法调用工具
  - 会话管理：通过会话属性管理对话历史
  - 事件系统：通过事件总线属性订阅和发布事件
  - 运行入口：通过 `self.run(input)` 启动任务（继承自 Agent）
- CodeAgent 的扩展能力（在 Agent 基础上新增）：
  - **模型平台**：重写 `_init_model()`，使用 **smart 平台**（get_smart_platform），适用于代码生成等复杂场景。
  - **需求/场景分类**：首次运行时根据用户输入调用 `classify_user_request` 进行场景分类，按场景切换系统提示词（scenarios.yaml + get_system_prompt(scenario)），使系统提示与任务类型匹配。
  - **内置命令处理**：优先识别并执行内置快捷命令（如 @ 触发的操作），避免不必要的工具调用，提升响应效率
  - **目录树优化**：智能截断目录树输出（max_tree_lines=200），防止大型项目上下文膨胀，确保关键信息优先展示
  - **会话管理统一架构**：基于 SessionRestorable 基类实现会话持久化，支持会话恢复与状态一致性检查（如 Git Commit 验证）
  - 环境与仓库管理：发现仓库根、更新 .gitignore、处理未提交修改、统一换行符敏感策略（含 Windows 建议）。
  - **项目概况注入**：首轮 run 时调用 `get_project_overview(root_dir)` 将项目概况拼接到用户输入前，并注入首轮操作规范提示（如 PATCH 优先、禁止 sed 等），便于模型理解项目结构。
  - 代码分析器模块：
    - 上下文管理：维护符号表和依赖图，提供代码上下文查询能力
    - 影响范围分析：分析编辑的影响范围，识别受影响文件、符号、测试等
    - 智能上下文推荐：根据用户输入推荐相关代码上下文（可通过配置 `is_enable_intent_recognition` 启用/关闭意图识别与推荐）
    - 构建验证：自动检测构建系统并执行构建验证
    - 静态分析：根据文件类型自动选择和执行 lint 工具（已重构为命令模板机制）
  - 内置规则系统：
    - 规则文件化：规则以 Markdown 文件形式组织在 `rules/` 和 `test_rules/` 目录
    - 动态加载：启动时自动扫描并加载所有规则文件
    - 指令性格式：规则已优化为适合大模型的指令性格式，使用“必须”、“禁止”等明确指令
    - 规则分类：通用开发实践规则（TDD、代码审查、重构等）和语言特定测试规则
  - **文件编辑**：edit_file 支持 search/replace 与**整文件重写**（空 search 参数）；大范围重写时建议配合空 search 并提前备份。
  - 文件变更后处理：
    - 自动格式化：根据文件类型自动执行对应的格式化工具（ruff、prettier、rustfmt等）
    - 可配置：支持通过配置文件自定义格式化命令模板
    - 多语言支持：支持 Python、JavaScript、Rust、Go、Java、C/C++ 等多种语言
    - 提交确认后可执行后处理：handle_commit_confirmation 的 post_process 会调用 post_process_manager 对已修改文件执行格式化等
  - **代码审查（Review）与迭代修复**：主流程结束后可选执行多轮“审查 → 修复”循环（`_review_and_fix`）。使用独立 Review Agent 审查本次修改的 git diff，输出结构化 JSON（ok/issues/summary）；若不通过则根据 issues 触发修复（修复阶段可生成总结并再次审查）。支持 `disable_review`、`review_max_iterations`（0 表示不设上限）。
  - 提交工作流：run 支持 **prefix/suffix** 参数用于提交信息生成；主流程与 review 后调用 **handle_uncommitted_changes** 处理未提交变更；自动/交互式 commit、提交历史展示与接受/重置；提交确认后执行后处理（格式化等）。
  - 差异与预览：按文件输出 diff，**按文件补丁预览**（build_per_file_patch_preview）；支持**增强 diff 可视化**（visualize_diff_enhanced、可视化模式与行号显示可配置）；针对删除/重命名/大变更进行适配与摘要化处理；**补丁应用前可选用户确认**（is_confirm_before_apply_patch），未确认时可输入自定义回复作为附加提示。
  - 大量代码删除防护：非交互模式下自动检测大量代码删除，询问大模型判断是否合理（ask_llm_about_large_deletion），防止误删重要代码。
  - 统计与提示：记录代码行增删、修改次数；根据文件类型生成 lint 建议与静态扫描引导。
  - **任务结束前可选分析**：CodeAgent 初始化时关闭 Agent 的 use_analysis，在 run 结束前（最终提交确认之前）根据配置与用户确认可选调用 `analysis()` 进行任务分析与方法论沉淀。
  - CLI 入口：非交互约束、单实例锁（按仓库维度）、会话恢复、参数同步配置。

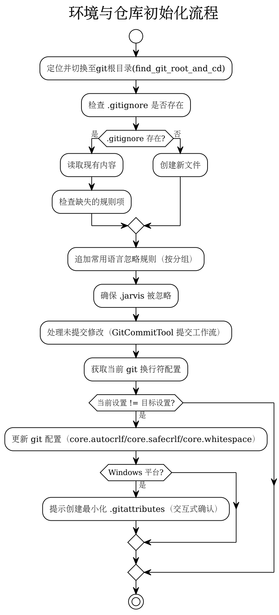
3.1 CodeAgent 初始化流程

- 初始化步骤（按顺序）：
  1. 设置基础属性：
     - 设置根目录为当前工作目录
     - 保存工具组等配置参数
  2. 初始化上下文管理器（ContextManager）：
     - 创建 ContextManager 实例，传入项目根目录
     - 维护符号表（SymbolTable）和依赖图（DependencyGraph）
     - 提供代码上下文查询能力（查找定义、引用、依赖关系等）
  3. Git 配置检查（Git配置检查方法）：检查用户名和用户邮箱是否已设置
     - 严格模式（默认）：任一缺失则退出，提示配置命令
     - 警告模式（JARVIS_GIT_CHECK_MODE=warn）：仅提示警告，继续运行
  4. 构建工具白名单：
     - 基础工具列表：
       - `execute_script`：脚本执行工具，用于执行 shell/脚本命令
       - `read_code`：代码读取工具，读取源代码文件的指定行号范围
       - `edit_file`：文件编辑工具，基于 search/replace 的普通文本编辑
       - `load_rule`：规则加载工具，加载相关规则和最佳实践
       - `virtual_tty`：虚拟终端工具，支持交互式操作
       - `search_web`：网络搜索工具，检索外部信息
       - `read_webpage`：网页内容读取工具，读取网页内容
       - `save_memory`：记忆保存工具，保存项目约定和架构决策
       - `retrieve_memory`：记忆召回工具，检索相关记忆
       - `clear_memory`：记忆删除工具，清理记忆
       - `methodology`：方法论工具，加载和使用方法论
       - `task_list_manager`：任务列表管理工具（可选，通过 `enable_task_list_manager` 参数控制）
     - 追加工具：通过追加工具参数（逗号分隔）追加，自动去重
  5. 读取规则文件（按优先级顺序加载，动态加载机制）：
     - **规则文件组织**：规则以 Markdown 文件形式组织，统一存储在 `.jarvis` 目录下：
       - 项目规则：`.jarvis/rules/` 目录（用户自定义规则）
       - 内置规则：通过 `RulesManager` 统一管理，支持命名规则（named rules）和文件规则
       - 规则文件路径已统一到 `.jarvis` 目录，便于管理和版本控制
     - **动态加载机制**：`RulesManager` 在初始化时自动扫描规则目录，支持通过规则名称获取规则内容
     - **规则格式优化**：所有规则文件已改写为适合大模型理解和使用的指令性格式：
       - 使用“必须”、“禁止”等明确指令性语言
       - 提供详细的执行要求和检查清单
       - 包含代码示例（正确 vs 错误实践）
       - 结构化的格式便于AI解析和执行
     - **外部规则加载**（按优先级顺序）：
       - 项目规则：`.jarvis/rules/`（若存在，优先级最高）
       - 全局规则：`{get_data_dir()}/rules`（若存在）
       - 配置的规则目录：通过 `get_rules_load_dirs()` 获取配置的规则加载目录（若存在）
       - **中心规则仓库**（优先级最高）：如果配置了 `JARVIS_CENTRAL_RULES_REPO`，从中心规则仓库加载规则
         - 支持本地目录路径或 Git 仓库 URL
         - 首次运行时自动克隆到 `{get_data_dir()}/central_rules_repo`
         - 每日自动更新（通过 `daily_check_git_updates` 机制）
         - 中心规则仓库中的规则优先级最高，会覆盖同名规则
     - 规则文件加载：从 rules 目录中按文件名加载规则文件，支持从多个目录加载同名规则（按优先级覆盖）
     - 按顺序拼接为单一 `<rules>` 区块，追加到系统提示
     - **规则分享功能**：支持通过 `RuleShareManager` 将本地规则分享到中心规则仓库
       - 只能分享默认数据目录（`{get_data_dir()}/rules`）中的规则
       - 不能分享配置的规则目录和项目规则目录中的规则
       - 分享时会将规则文件移动到中心仓库（而非复制），原文件会被删除
       - 自动检测并排除已存在于中心仓库的规则
  6. 调用父类 Agent 初始化（父类初始化方法）：
     - 注入系统提示（代码工程师工作准则 + 规则块）
     - 设置工具白名单（使用工具参数）
     - 默认禁用方法论引导（使用方法论参数为False，可通过关键字参数覆盖）

            - 默认禁用任务分析（使用分析参数为False，可通过关键字参数覆盖）
            - 关闭自动完成（自动完成参数为False）
            - 启用规划（规划参数，默认从配置读取）
            - 初始化模型、会话、处理器等 Agent 核心组件
        7. 建立 CodeAgent 自关联：
            - 设置代码代理属性为自身，便于工具通过属性获取方法获取 CodeAgent 实例
        8. 初始化上下文推荐器（上下文推荐器，可选）：
            - 若模型已初始化且可用（通过模型属性访问）：
                - 创建上下文推荐器实例，传入上下文管理器和 LLM 模型
                - 用于任务开始时的智能上下文推荐
            - 若初始化失败：仅记录警告，不影响主流程（跳过上下文推荐功能）
        9. 订阅工具调用后事件：通过事件总线订阅方法注册工具调用后回调
    - 系统提示内容：
        - 代码工程师角色定位与核心原则
        - 工作流程（项目分析 → 需求分析 → 代码分析 → 方案设计 → 实施修改）
        - 工具使用规范（优先使用文件搜索、内容搜索、read_code）
        - 文件编辑工具使用规范（edit_file：支持search/replace和整文件重写）
        - 子任务与子 CodeAgent 使用建议
        - 全局规则（数据目录 rules）与项目规则（.jarvis/rules）合并为 `<rules>` 区块

3.2 环境与仓库初始化

- 初始化流程（环境初始化方法，在运行方法开始时调用）：
    1. 查找 Git 根目录（查找Git根目录方法）：调用查找Git根目录并切换目录方法，切换到仓库根并更新根目录属性
    2. 更新 .gitignore（更新Git忽略文件方法）：
        - 检查并确保配置目录被忽略
        - 追加常用语言忽略规则（按语言分组：通用、Python、Rust、Node、Go、Java、C/C++、.NET 等）
        - 仅追加缺失项，不覆盖现有规则；保留注释与空行
    3. 处理未提交修改（处理Git变更方法）：
        - 调用Git提交工具执行提交工作流（使用前缀/后缀参数）
        - 支持交互式确认与临时提交
    4. 配置换行符设置（配置行尾设置方法）：
        - 目标设置：自动换行转换关闭、安全换行检查关闭、空白字符处理为行尾回车
        - 仅在当前设置与目标不一致时修改
        - Windows 平台额外提示：建议创建最小化Git属性文件（交互式确认）



说明

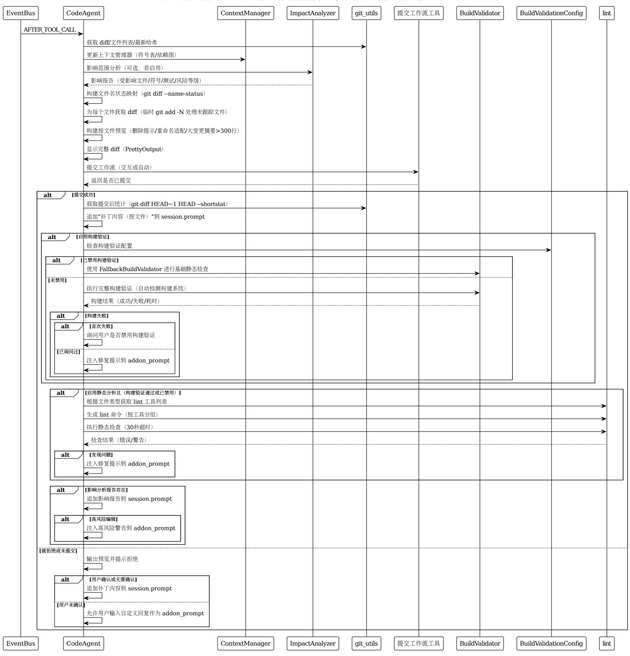
- 保守策略：仅在必要时变更 git 配置；Windows 平台提供 .gitattributes 建议，以避免 CRLF/LF 差异导致的大 diff。
- .gitignore 更新：按语言分组追加规则，保留现有内容与格式；新建文件时仅包含缺失项。
- 未提交修改处理：使用 GitCommitTool 统一处理，支持交互式确认与临时提交；失败不阻断流程。

3.3 工具执行后旁路增强（差异/提交/统计/静态检查/影响分析/构建验证）

- 回调入口：工具调用后回调方法（订阅工具调用后事件）
- 核心流程：
    1. 获取差异信息：
        - 调用获取差异方法获取完整差异文本
        - 调用获取差异文件列表方法获取修改文件列表
        - 调用获取最新提交哈希方法记录起始提交哈希
    2. 更新上下文管理器（更新修改文件上下文方法）：
        - 对每个修改的文件，更新符号表和依赖图
        - 调用上下文管理器更新文件上下文方法重新分析文件内容
        - 提取符号（函数、类、变量等）并更新符号表
        - 分析导入依赖并更新依赖图
    3. 影响范围分析（分析编辑影响方法，可选）：
        - 若启用影响分析（启用影响分析参数），解析Git差异为编辑操作列表
        - 使用影响分析器分析编辑的影响范围：
            - 识别受影响的符号（函数、类等）
            - 查找符号引用位置
            - 分析依赖链影响
            - 检测接口变更（函数签名、参数等）
            - 查找相关测试文件
        - 生成影响报告（ImpactReport），包含：
            - 受影响文件列表
            - 受影响符号列表
            - 相关测试文件
            - 接口变更详情
            - 风险等级评估（LOW/MEDIUM/HIGH）
            - 修复建议
        - 高风险编辑时，在 addon_prompt 中注入警告提示
    4. 构建按文件补丁预览（构建按文件补丁预览方法）：
        - 构建文件名状态映射（构建名称状态映射方法）：通过Git差异名称状态命令识别文件状态（新增/修改/删除/重命名/复制）
        - 为每个文件获取差异（获取文件差异方法）：使用Git临时暂存命令临时暂存未跟踪文件以展示差异
        - 特殊处理策略：
            - 删除文件（D）：不展示 diff，仅输出删除提示（附带删除行数若可用）
            - 重命名/复制（R/C）：使用新路径作为键，记录状态映射
            - 大变更（新增+删除 > 300 行）：仅输出统计行数，避免上下文过长
            - 其它文件：输出该文件的 diff 代码块（使用 `git diff --numstat` 获取行数统计）
            - 无法获取 diff：输出友好提示（“变更已记录（无可展示的文本差异）”）
    5. 提交工作流：
        - 显示完整差异（格式化输出打印方法，语言类型为差异）
        - 调用处理提交工作流方法执行提交（交互或自动）
    6. 统计记录（提交成功后）：
        - 代码行数变化：通过 Git 差异统计命令获取并记录
        - 修改次数与提交计数：在提交与显示提交历史时更新
    7. 构建验证（处理构建验证方法，可选）：
        - 若启用构建验证（启用构建验证参数）：
            - 检查项目配置（构建验证配置）：是否已禁用构建验证
            - 若已禁用：使用回退构建验证器进行基础静态检查
            - 若未禁用：使用构建验证器进行完整构建验证
                - 自动检测构建系统（Rust/Cargo、Python构建配置、Node.js包管理、Java构建工具、Go、CMake、Makefile 等）
                - 执行构建命令（如Cargo构建、Python构建、npm构建等）
                - 捕获构建输出和错误信息
                - 返回构建结果（构建结果对象），包含成功/失败状态、耗时、构建系统类型等
            - 构建失败处理：
                - 首次失败：询问用户是否禁用构建验证（适用于特殊环境场景）
                - 已询问过：直接显示错误并注入修复提示到附加提示
            - 构建成功：显示构建系统类型和耗时
    8. 文件变更后处理（文件变更后处理工具，可选）：
        - 若启用文件变更后处理（通过 `after_change.py` 模块配置）：
            - 根据文件扩展名自动选择对应的格式化工具：
                - Python：ruff format
                - JavaScript/TypeScript：prettier
                - Rust：rustfmt
                - Go：gofmt
                - Java：google-java-format
                - C/C++：clang-format
                - HTML/CSS：prettier
                - JSON/YAML：prettier
                - Markdown：prettier
                - SQL：sqlfluff format
                - Shell：shfmt

            - XML：xmllint
        - 支持从数据目录的 `after_change_tools.yaml` 配置文件加载自定义命令模板
        - 命令模板支持占位符：`{file_path}`（完整路径）、`{file_name}`（文件名）
        - 自动执行格式化命令，确保代码风格一致
    9. 静态分析（处理静态分析方法，可选）：
        - 若启用静态分析（启用静态分析参数）且构建验证通过（或已禁用）：
            - 根据文件类型获取静态检查工具列表（获取静态检查工具方法）：
                - 支持多种语言：Python（ruff、mypy）、Rust（cargo clippy、rustfmt）、Go（go vet）、C/C++（clang-tidy）、JavaScript/TypeScript（eslint、tsc）、Java（pmd）等
                - 支持从数据目录的静态检查工具配置文件加载自定义配置
                - **重构优化**：lint 工具配置已重构为命令模板机制，支持更灵活的配置方式
            - 生成静态检查命令（获取文件的静态检查命令方法）：
                - 按工具分组，项目级工具（如 cargo clippy）每个项目只执行一次
                - 文件级工具（如 ruff、eslint）每个文件都执行
                - 自动查找配置文件（如ESLint配置、Prettier配置等）
                - 使用命令模板机制，支持占位符替换
            - 执行静态检查（运行静态分析方法）：
                - 按工具分组批量执行
                - 30秒超时限制
                - 只记录有错误或警告的结果（返回码不为0）
            - 格式化结果（格式化静态检查结果方法）：
                - 输出工具名、文件路径、命令、错误信息
                - 限制输出长度（1000字符），避免过长
            - 发现问题时：注入修复提示到附加提示，要求模型修复所有问题
    10. 会话上下文更新：
    - 提交成功：将“补丁内容（按文件）”追加到代理会话提示字段
    - 影响分析报告：若生成了影响报告，追加到会话上下文
    - 构建验证结果：追加构建验证结果（成功/失败）
    - 静态分析结果：追加静态分析结果（通过/发现问题）
    - 文件变更后处理结果：追加格式化执行结果（如适用）
    1. 用户确认机制（提交被拒绝时）：
    - 输出预览与拒绝提示
    - 若应用补丁前确认方法返回 False 或用户确认，将补丁内容追加到会话上下文
    - 否则允许用户输入自定义回复作为附加提示

工具执行后旁路增强（差异预览/提交/统计/静态检查/影响分析/构建验证）

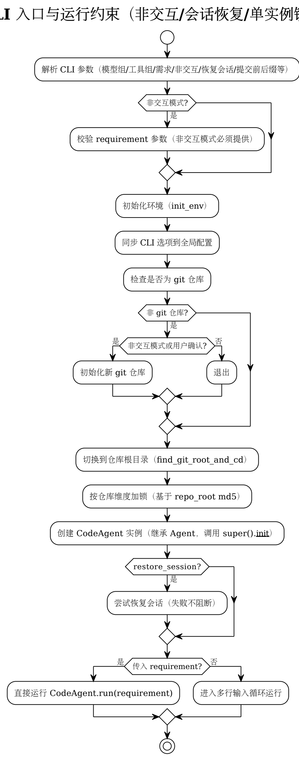


说明

- 上下文管理器更新：
    - 每次文件修改后，自动更新符号表和依赖图，保持代码上下文的最新状态
    - 支持多语言（Python、Rust、Go、C/C++、JavaScript/TypeScript 等）的符号提取和依赖分析
    - 符号表包含函数、类、方法、变量等，支持查找定义和引用
    - 依赖图维护文件间的导入依赖关系，支持查找依赖和被依赖文件
- 影响范围分析：
    - 自动解析 git diff 为编辑操作，识别受影响的符号和文件
    - 分析符号引用链和依赖链，找出所有可能受影响的代码位置
    - 检测接口变更（函数签名、参数变化等），评估风险等级
    - 自动查找相关测试文件，提供修复建议
    - 高风险编辑时自动注入警告提示，提醒模型注意影响范围
- 构建验证：
    - 自动检测构建系统（Rust/Cargo、Python、Node.js、Java、Go、CMake、Makefile 等）
    - 执行构建命令并捕获错误，提供详细的构建失败信息
    - 支持项目级配置（BuildValidationConfig），允许禁用构建验证（适用于特殊环境）
    - 构建失败时提供禁用选项，避免在特殊环境中反复失败
    - 已禁用构建验证时，使用 FallbackBuildValidator 进行基础静态检查
- 静态分析：
    - 根据文件类型自动选择 lint 工具（ruff、mypy、clippy、eslint 等）
    - 支持项目级工具（如 cargo clippy）和文件级工具（如 ruff）
    - 自动查找配置文件（.eslintrc、.prettierrc 等）
    - 按工具分组批量执行，30秒超时限制
    - 发现问题时自动注入修复提示，要求模型修复所有问题
- 差异预览策略：
    - 删除文件：不展示 diff，仅输出删除提示（附带删除行数若可用）
    - 大变更（新增+删除 > 300 行）：仅输出统计行数，避免上下文过长
    - 其它文件：输出该文件的 diff 代码块（使用临时 `git add -N` 处理未跟踪文件）
    - 无法获取 diff：输出友好提示
- 统计记录：
    - 插入/删除行数（基于 `git diff --shortstat`）及修改、提交相关计数
- 用户确认机制：
    - 提交被拒绝时，根据应用补丁前确认配置决定是否要求用户确认
    - 支持用户输入自定义回复作为附加提示，便于后续继续处理

3.4 CLI 入口与运行约束

- CLI 入口：命令行接口函数（使用Typer框架装饰）
- 核心流程：
    1. 参数解析与预处理：
        - 解析 CLI 参数（模型组/工具组/需求/非交互/恢复会话/提交前后缀等）
    2. 环境初始化：
        - 调用 `init_env` 初始化配置与欢迎信息
        - 同步 CLI 选项到全局配置（llm_group/tool_group/restore_session/non_interactive）
    3. Git 仓库校验与初始化：
        - 检查是否为 git 仓库（`git rev-parse --git-dir`）
        - 若非仓库：交互或非交互模式下提示初始化新仓库
    4. 仓库根目录定位与单实例锁：
        - 调用 `find_git_root_and_cd` 切换到仓库根
        - 按仓库维度加锁：基于 `repo_root` 的 md5 哈希生成锁文件名（`code_agent_{md5}.lock`）
        - 回退策略：加锁失败时回退到全局锁（`code_agent.lock`）
    5. CodeAgent 构造与初始化：
        - 创建 CodeAgent 实例（传入 llm_group/tool_group/append_tools/non_interactive）
        - CodeAgent 内部执行：git 配置检查、工具白名单构建、规则加载、调用父类 Agent 初始化（super().**init**）、事件订阅
    6. 会话恢复（可选）：
        - 若 `restore_session=True`，调用 `agent.restore_session()`
        - 失败不影响继续运行
    7. 任务执行：
        - 若传入 `requirement`：直接调用 `agent.run(requirement, prefix, suffix)`
        - 否则：进入多行输入循环，持续处理用户输入



约束与行为

- 非交互模式：必须通过 –requirement 传入任务；多行输入不可用；执行脚本超时受限（由平台环境约束）。
- 单实例锁：按仓库维度加锁（基于 repo_root 的 md5），避免跨仓库互斥；失败时回退到全局锁。
- 会话恢复：从存档文件恢复；失败不影响继续运行。
- Git 仓库要求：必须在 git 仓库中运行；非仓库时支持交互式或非交互自动初始化。

3.5 代码分析器模块（code_analyzer）

CodeAgent 集成了代码分析器模块，提供代码结构分析、依赖关系管理、影响范围分析和智能上下文推荐等功能。

- 源码位置：`src/jarvis/jarvis_code_agent/code_analyzer/`
- 核心组件：
  - ContextManager：上下文管理器，维护符号表和依赖图
  - ImpactAnalyzer：影响范围分析器
  - ContextRecommender：智能上下文推荐器（基于 LLM）
  - DependencyAnalyzer：依赖分析器（基础类）
  - BuildValidator：构建验证器
  - SymbolExtractor：符号提取器（支持多语言）

3.5.1 ContextManager（上下文管理器）

- 职责：维护项目的符号表和依赖图，提供代码上下文查询能力
- 核心数据结构：
  - SymbolTable：符号表，存储函数、类、方法、变量等符号定义
  - DependencyGraph：依赖图，维护文件间的导入依赖关系
  - 文件缓存：缓存文件内容，避免重复读取
- 主要方法：
  - 更新文件上下文方法（文件路径，内容）：更新单个文件的符号表和依赖信息
  - 获取编辑上下文方法（文件路径，起始行，结束行）：获取编辑位置的上下文信息
  - 查找引用方法（符号名，文件路径）：查找符号的所有引用位置
  - 查找定义方法（符号名，文件路径）：查找符号的定义位置
- 语言支持：
  - 通过 LanguageRegistry 自动检测语言类型
  - 支持 Python、Rust、Go、C/C++、JavaScript/TypeScript 等
  - 每种语言有对应的 SymbolExtractor 和 DependencyAnalyzer 实现
- 使用场景：
  - 文件修改后自动更新上下文
  - 影响范围分析时查找符号引用
  - 智能上下文推荐时提供相关符号和文件

3.5.2 ImpactAnalyzer（影响范围分析器）

- 职责：分析代码编辑的影响范围，识别可能受影响的文件、函数、测试等
- 核心功能：
  - 解析 git diff 为编辑操作列表（parse_git_diff_to_edits）
  - 识别受影响的符号（函数、类等）
  - 查找符号引用位置（通过 ContextManager）
  - 分析依赖链影响（传递闭包）
  - 检测接口变更（函数签名、参数变化等）
  - 查找相关测试文件（TestDiscoverer）
  - 评估风险等级（LOW/MEDIUM/HIGH）
  - 生成修复建议
- 影响类型（ImpactType）：
  - REFERENCE：符号引用
  - DEPENDENT：依赖的符号
  - TEST：测试文件
  - INTERFACE_CHANGE：接口变更
  - DEPENDENCY_CHAIN：依赖链
- 输出（ImpactReport）：
  - 受影响文件列表
  - 受影响符号列表
  - 相关测试文件
  - 接口变更详情
  - 风险等级
  - 修复建议
- 使用场景：
  - 工具执行后自动分析编辑影响
  - 高风险编辑时提醒模型注意影响范围
  - 提供修复建议和测试建议

3.5.3 ContextRecommender（智能上下文推荐器）

- 职责：使用 LLM 进行语义理解，根据用户输入推荐相关的代码符号位置
- 核心功能：
  - 使用 LLM 从用户输入中提取关键词（_extract_keywords_with_llm）
  - 基于关键词进行符号名称匹配和文本搜索（_search_symbols_by_keywords、_search_text_by_keywords）
  - 使用 LLM 从候选符号中筛选关联度高的条目（_select_relevant_symbols_with_llm）
  - 在 LLM 交互时提供项目概况信息（代码统计、Git 文件信息、最近提交）
- 推荐策略：
  - 基于关键词提取：使用 LLM 从用户输入中提取关键词
  - 基于关键词搜索：在符号表和代码文本中搜索匹配的符号
  - 基于 LLM 筛选：使用 LLM 从候选符号中挑选最相关的条目
- 输出（ContextRecommendation）：
  - 推荐符号列表（最多10个），每个符号包含：
    - 符号名称和类型
    - 文件路径
    - 行号位置
- 使用场景：
  - 任务开始时根据用户输入推荐相关代码位置
  - 帮助模型快速定位需要关注的符号位置
  - 减少模型需要搜索和阅读的代码量

3.5.4 BuildValidator（构建验证器）

- 职责：自动检测构建系统并执行构建验证，确保代码修改后项目仍能正常构建
- 支持的构建系统：
  - Rust：Cargo（cargo build）
  - Python：setup.py、pyproject.toml（python -m build）
  - Node.js：npm、yarn、pnpm（npm run build）
  - Java：Maven（mvn compile）、Gradle（./gradlew build）
  - Go：go build
  - C/C++：CMake（cmake + make）、Makefile（make）
- 构建系统检测（BuildSystemDetector）：
  - 按优先级检测构建系统配置文件
  - 支持多构建系统项目（选择第一个检测到的）
- 构建验证流程：
  - 检测构建系统类型
  - 执行构建命令（带超时限制）
  - 捕获构建输出和错误信息

        - 返回构建结果（BuildResult）
    - 构建结果（BuildResult）：
        - success：是否成功
        - build_system：构建系统类型
        - duration：耗时（秒）
        - output：构建输出
        - error_message：错误信息（失败时）
    - 配置管理（BuildValidationConfig）：
        - 项目级配置：`.jarvis/build_validation_config.yaml`
        - 支持禁用构建验证（适用于特殊环境）
        - 记录禁用原因和询问状态
    - 兜底验证器（FallbackBuildValidator）：
        - 当构建验证被禁用时使用
        - 进行基础静态检查（语法检查等）
    - 使用场景：
        - 代码修改后自动验证构建
        - 构建失败时提供错误信息和修复提示
        - 特殊环境（如容器）中可禁用构建验证

3.5.5 静态分析工具（lint.py）

- 职责：根据文件类型自动选择和执行 lint 工具
- 支持的语言和工具：
    - Python：ruff、mypy、pylint、flake8、black
    - Rust：cargo clippy、rustfmt
    - Go：go vet、golint
    - C/C++：clang-tidy、cppcheck
    - JavaScript/TypeScript：eslint、tsc
    - Java：pmd、checkstyle
    - 其他：PHP（phpstan）、Ruby（rubocop）、Swift（swiftlint）等
- 工具配置：
    - 默认配置：LINT_TOOLS 字典（文件扩展名/文件名 -> 工具列表）
    - 自定义配置：支持从 `{get_data_dir()}/lint_tools.yaml` 加载
- 命令生成（get_lint_commands_for_files）：
    - 按文件类型获取工具列表
    - 生成 lint 命令（支持配置文件查找）
    - 区分项目级工具（如 cargo clippy）和文件级工具（如 ruff）
    - 项目级工具每个项目只执行一次
- 配置文件查找（find_config_file）：
    - 支持从项目根目录和文件所在目录向上查找
    - 支持多种配置文件名（如 .eslintrc.js、.eslintrc.json 等）
    - 区分必需配置和可选配置
- 执行策略：
    - 按工具分组批量执行
    - 30秒超时限制
    - 只记录有错误或警告的结果
    - 限制输出长度（1000字符）
- 使用场景：
    - 代码修改后自动执行静态检查
    - 发现问题时注入修复提示
    - 帮助模型修复代码质量问题

3.6 核心工具能力（Agent 工具调用）

- 代码读取：read_code
    - 用途：读取源代码文件的指定行号范围，并为每行添加行号后返回。
    - 功能特性：
        - 简单直接：读取指定行号范围的文本内容，为每行添加行号标注
        - 参数：支持 `path`（文件路径）、`start_line`（起始行号，默认1）、`end_line`（结束行号，-1表示文件末尾）
        - Token 限制：自动估算输出 token 数，超过限制（2/3最大窗口）时提示分批读取
        - 多文件支持：支持同时读取多个文件，自动合并同一文件的多个范围请求
        - 上下文感知：集成 ContextManager，自动提供代码上下文信息（当前作用域、使用的符号、相关文件等）
    - 最佳实践：遵循“先读后写”，优先按范围读取（start_line/end_line）以避免上下文膨胀。
- 文件编辑工具：edit_file
    - edit_file：基于 search/replace 的普通文本编辑，使用精确字符串匹配，适合已知原始代码片段、需精确替换的场景
        - **重要：search 必须提供足够的上下文来唯一定位目标位置**，避免匹配到错误的位置。建议包含：
            - 目标代码的前后几行上下文（至少包含目标代码所在函数的签名或关键标识）
            - 目标代码附近的唯一标识符（如函数名、变量名、注释等）
            - 避免使用过短的 search 文本（如单个单词、短字符串），除非能确保唯一性
        - **多处匹配智能确认与预览**：
            - 当 search 文本在文件中存在多处匹配时，工具会自动检测并生成匹配位置预览
            - 预览信息包含每个匹配位置的行号、列号以及前后各3行的上下文代码
            - 如果存在多处匹配，工具会要求 Agent 确认是否继续（通过 Agent 的确认回调）
            - Agent 可以查看预览信息后决定是否继续替换所有匹配，或取消操作
            - 如果确认继续，会替换所有匹配位置；如果取消，文件内容会回滚到原始状态
        - **移除 count 参数**：不再支持通过 count 参数控制替换次数，改为通过多处匹配检测和确认机制来确保替换的准确性
        - **增强的错误提示**：当搜索失败时，提供详细的错误信息和修复建议，包括：
            - 明确说明未找到精确匹配的文本
            - 建议使用 `read_code` 工具重新读取文件查看当前内容
            - 提示文件可能已被更新，需要根据实际内容调整 search 文本
            - 建议在 search 中包含足够的上下文，确保能唯一匹配到目标位置
    - 建议：
        - 简单 search/replace 使用 edit_file
        - edit_file 通过空 search 参数可实现整文件重写（适用于大范围生成或重构，支持原子写与回滚）
    - 文件编辑工具（edit_file）：

        - edit_file：基于 search/replace 的普通文本编辑，使用精确字符串匹配

        - 原子写入与回滚：
            - 编辑前创建备份文件（`.bak`）
            - 写入失败时自动回滚到备份
            - 写入成功后删除备份文件
        - 错误处理：
            - 匹配失败：明确提示未找到匹配，建议检查代码上下文
            - 部分失败：支持部分补丁成功，返回详细错误摘要
            - 多处匹配：自动生成预览并要求确认，避免误替换
    - 使用示例：

        - edit_file 使用示例：

          ```
          edit_file({
            "files": [{
              "file_path": "src/main.py",
              "diffs": [{
                "search": "def hello():\n    print('Hello')",
                "replace": "def hello():\n    print('Hello, World!')"
              }]
            }]
          })
          ```

          注意：search 文本应包含足够的上下文（如函数签名），确保能唯一定位目标位置。

    - 最佳实践：
        - 优先使用 edit_file 进行简单的 search/replace
        - **重要**：search 文本必须包含足够的上下文，确保能唯一匹配到目标位置，避免误匹配
    - 实现位置：
        - 读取工具：`jarvis_tools/read_code.py::ReadCodeTool`
        - 编辑工具：`jarvis_tools/edit_file.py`
    - 技术细节：
        - Token 估算：基于输出内容估算 token 数，超过限制时提示分批读取
- 命令执行（静态检测等）：execute_script
    - 用途：执行 shell/脚本命令，用于 lint、静态分析、单元测试、构建等工程化操作。
    - 约束：非交互环境有超时限制；避免输出过长，建议配合 rg/grep 等做过滤；不支持交互式命令。
    - 提示：集中在所有修改完成后统一进行静态检查，一次性调用相关工具，避免多次分散调用。
- Web 搜索：search_web
    - 用途：检索外部信息（API/错误/最佳实践/安全建议），辅助方案设计与问题定位。
    - 最佳实践：明确查询上下文与预期结果；对关键结论进行复核，并按需写入记忆或备注。
- 记忆工具：save_memory / retrieve_memory / clear_memory
    - 用途：沉淀项目约定、架构决策、常用命令与方法论；支持按类型与标签检索与清理。
    - 建议：在任务完成或历史清理前根据需要进行保存与整理。

说明

- 上述能力均通过 Agent 的工具调用接口执行；CodeAgent 通过工具白名单进行启用与收敛，并在 AFTER_TOOL_CALL 事件中进行旁路增强（差异预览、提交与统计、静态检查引导）。

3.7 大量代码删除防护

- 功能概述：在非交互模式下，自动检测大量代码删除并询问大模型判断是否合理，防止误删重要代码。
- 触发条件：
    - 仅在非交互模式（`non_interactive=True`）下生效
    - 在调用 `handle_commit_workflow()` 之前进行检测
    - 检测阈值：默认净删除行数超过 200 行（可通过 `detect_large_code_deletion(threshold)` 参数调整）
- 检测流程：
    1. 调用 `detect_large_code_deletion()` 获取代码删除统计信息
    2. 如果检测到大量删除（返回非 None），获取统计信息：
        - `insertions`：新增行数
        - `deletions`：删除行数
        - `net_deletions`：净删除行数（删除 - 新增）
    3. 构建补丁预览（`per_file_preview`），包含所有修改文件的差异信息
- 大模型判断：
    - 调用 `_ask_llm_about_large_deletion()` 方法询问大模型
    - 提示词包含：
        - 统计信息（新增/删除/净删除行数）
        - 完整的补丁预览内容

            - 可能情况的说明（重构、简化、删除未使用代码、误删等）
        - 使用确定的协议标记：
            - 合理：`<!!!YES!!!>`
            - 不合理：`<!!!NO!!!>`
        - 解析回答：直接查找协议标记，避免因回答格式不一致导致的误判
- 处理逻辑：
    - 如果大模型回答 `<!!!YES!!!>`（认为合理）：
        - 继续执行后续提交流程（处理提交工作流方法）
    - 如果大模型回答拒绝标记（认为不合理）：
        - 调用撤销变更方法撤销所有修改
        - 输出撤销提示和补丁预览
        - 终止当前工具调用流程，不进行提交
    - 如果无法找到协议标记或询问失败：
        - 采用保守策略，默认认为不合理
        - 撤销修改并终止流程
- 容错设计：
    - 检测过程异常时返回 None，不触发防护机制
    - 大模型询问失败时采用保守策略（默认不合理）
    - 协议标记解析失败时默认认为不合理，确保安全
- 使用场景：
    - 非交互模式下的自动化代码修改
    - 防止大模型误删重要代码或功能
    - 在提交前进行二次确认，提高代码修改的安全性
- 实现位置：
    - 检测函数：`jarvis_utils/git_utils.py::detect_large_code_deletion()`
    - 判断方法：`jarvis_code_agent/code_agent.py::_ask_llm_about_large_deletion()`
    - 调用位置：`jarvis_code_agent/code_agent.py::_on_after_tool_call()`

说明

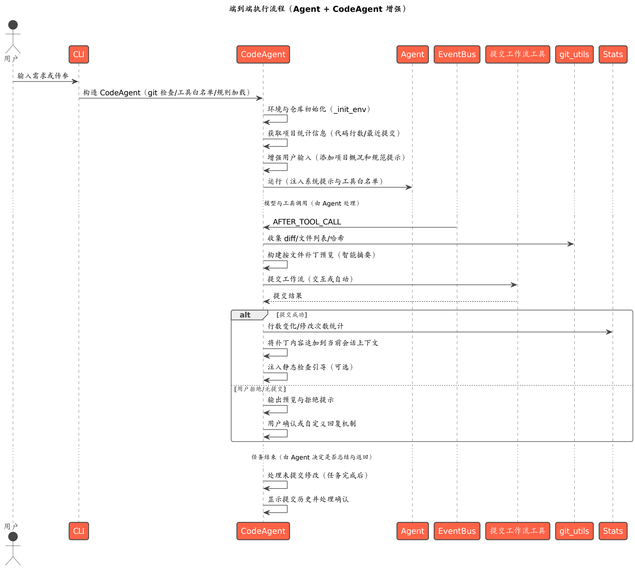
- 该防护机制仅在非交互模式下生效，交互模式下仍使用原有的用户确认机制（确认大量代码删除方法）。
- 通过确定的协议标记（`<!!!YES!!!>` / `<!!!NO!!!>`）确保解析的准确性，避免因大模型回答格式不一致导致的误判。
- 采用保守策略，在无法明确判断时默认认为不合理，确保代码安全。

3.8 代码审查与自动修复循环（Review Loop）

- 触发条件与参数：通过 CLI `--enable-review` 开启，`--review-max-iterations` 控制最大循环次数（默认 3）；在主流程完成一次修改并处理未提交变更后自动进入。
- 审查提示词模板（`_build_review_prompts`）：
    - system：审查标准（功能完整性/正确性/质量/风险），强调基于实际代码与 git diff，必要时可用 `read_code` 获取上下文。
    - user：嵌入用户需求与当前 git diff，要求逐项检查。
    - summary：要求输出 JSON，字段 `ok`、`issues[]`（type/description/location/suggestion）、`summary`。
- 审查执行流程（`_review_and_fix`）：
    1. 计算待审 diff（若当前提交等于起始提交则取未提交 diff，否则取起始到当前提交的 diff）；无 diff 时直接返回。
    2. 每轮创建 Review Agent（工具白名单仅 `execute_script` 与 `read_code`，`auto_complete=True`，关闭 methodology/analysis，名称 `CodeReview-Agent-{n}`），运行审查并解析 JSON 结果。
    3. 若 `ok=true` 则结束审查；若存在问题则逐条打印类型/描述/位置/建议（交互模式下先询问是否继续）。
    4. 超出最大迭代时：交互模式下询问是否重置计数继续，否则退出。
    5. 构建修复提示（汇总问题与建议）调用父类 `run` 进行代码修复，修复后再次处理未提交变更并进入下一轮。
- 交互/非交互差异：交互模式下每轮开始与修复前都询问用户是否继续；非交互模式直接循环直到通过或达到上限。
- 目的：在单 Agent 内形成“修改→审查→修复”闭环，提高改动正确性与覆盖度，减少人工复核成本。

4. 端到端执行流程（PlantUML）

下图展示一次典型“代码分析与修改”的端到端流程，不展开 Agent 内部细节。



说明

- CodeAgent 继承 Agent 的所有能力，直接使用 Agent 的模型交互、工具调用、会话管理等核心功能。
- AFTER_TOOL_CALL 作为主要旁路增强点，承载仓库提交、预览与统计、静态检查提示、影响分析、构建验证。
- 任务执行前：环境初始化、项目统计信息注入、用户输入增强、智能上下文推荐。
- 任务执行后：处理未提交修改、显示提交历史、处理提交确认。

5. 参数与配置说明（外层）

- CodeAgent 构造参数（**init**）
    - llm_group：模型组标识（可选）
    - need_summary：是否在完成阶段生成总结（默认 True，代码场景通常关闭）
    - append_tools：追加工具白名单（逗号分隔，自动去重）
    - tool_group：工具组标识（可选）
    - non_interactive：非交互模式（可选）
- 启动参数（传给 Agent 的外层注入）
    - system_prompt：注入“代码工程师工作准则与流程规范”，并拼接 （全局/项目规则）
    - use_tools：工具白名单（基础工具 + append_tools，自动去重）
    - auto_complete：自动完成任务，默认关闭（由 CodeAgent 控制交互流程）
    - need_summary：是否在完成阶段生成总结（代码场景通常关闭或由 CLI 控制）
    - use_methodology：禁用方法论引导（False）
    - use_analysis：禁用任务分析（False）
    - non_interactive：非交互模式（最高优先级）
- CLI 入口（typer）
    - -g/–llm-group：覆盖模型组
    - -G/–tool-group：覆盖工具组
    - -f/–config：配置文件路径
    - -r/–requirement：任务内容（非交互必须）
    - –append-tools：追加工具白名单（逗号分隔）
    - –restore-session：恢复会话状态
    - –prefix/–suffix：提交信息前后缀（prefix 用空格分隔，suffix 用换行分隔）
    - -n/–non-interactive：非交互模式
- 配置项与行为
    - 非交互、模型组、工具组、静态分析、补丁确认等通过配置读取，可被 CLI 覆盖
    - git 校验模式（JARVIS_GIT_CHECK_MODE）：strict（默认，缺失配置则退出）/warn（仅警告，继续运行）
    - 静态分析开关（is_enable_static_analysis）：控制是否在提交后注入 lint 建议
    - 构建验证开关（is_enable_build_validation）：控制是否在提交后执行构建验证
    - 构建验证超时（get_build_validation_timeout）：构建验证的超时时间（秒）
    - 影响分析开关（is_enable_impact_analysis）：控制是否在提交后执行影响范围分析
    - 意图识别开关（is_enable_intent_recognition）：控制是否启用智能上下文推荐
    - 补丁确认开关（is_confirm_before_apply_patch）：控制提交被拒绝时是否要求用户确认
    - 项目级构建验证配置（BuildValidationConfig）：
        - 配置文件：`.jarvis/build_validation_config.yaml`
        - 支持禁用构建验证（适用于特殊环境）
        - 记录禁用原因和询问状态

6. 可靠性与容错设计（外层）

- Git 配置检查：user.name / user.email 未设置时严格模式退出；warn 模式提示并继续。
- 非仓库场景：支持初始化新仓库（交互式或非交互自动）；失败明确告警并退出。
- 换行符策略：仅在与目标设置不一致时修改；Windows 提供 .gitattributes 建议（非强制，交互式确认）。
- .gitignore 更新：仅追加缺失项，保留现有内容与格式；读取失败时忽略，不影响主流程。
- 上下文管理器容错：
    - 文件读取失败时跳过更新，不影响其他文件的处理
    - 语言检测失败时跳过符号提取和依赖分析
    - 符号提取或依赖分析异常时捕获并继续，不中断主流程
    - 文件缓存机制避免重复读取，提高性能
- 影响范围分析容错：
    - git diff 解析失败时返回空列表，不影响主流程

    - 符号查找失败时跳过该符号的影响分析
    - AST 解析失败时跳过接口变更检测
    - 测试文件查找失败时仅记录警告，继续分析
    - 影响分析异常时捕获并记录日志，不中断主流程
- 构建验证容错：
    - 构建系统检测失败时跳过构建验证，不影响主流程
    - 构建命令执行超时或失败时捕获错误，提供详细错误信息
    - 构建验证配置读取失败时使用默认配置
    - 首次构建失败时询问用户是否禁用，避免反复失败
    - 已禁用构建验证时使用 FallbackBuildValidator 进行基础静态检查
- 静态分析容错：
    - lint 工具未安装时跳过该工具，不影响其他工具执行
    - lint 命令执行超时（30秒）时记录超时信息，继续执行其他工具
    - 配置文件查找失败时使用默认配置（可选配置工具）或跳过（必需配置工具）
    - lint 结果解析失败时跳过该结果，不影响其他结果
- 智能上下文推荐容错：
    - LLM 模型不可用时跳过上下文推荐，不影响主流程
    - LLM 调用失败时捕获异常并记录警告，继续执行任务
    - 关键词提取失败时返回空列表，继续推荐流程
    - 符号搜索失败时返回空结果，不影响主流程
- 差异容错：
    - HEAD 不存在时以临时索引方式（`git add -N`）统计未跟踪文件
    - 重命名/复制（R/C）以新路径记录，同时保留旧路径映射
    - 无法获取 diff 时给出友好提示（“变更已记录（无可展示的文本差异）”）
    - 使用 `errors="replace"` 处理编码问题，避免 diff 读取失败
- 大输出抑制：单文件新增+删除 > 300 行时采用统计摘要，防止上下文过长。
- 大量代码删除防护容错：
    - 检测过程异常时返回 None，不触发防护机制，继续正常提交流程
    - 大模型询问失败时采用保守策略（默认不合理），撤销修改并终止流程
    - 协议标记解析失败时默认认为不合理，确保代码安全
    - 仅在非交互模式下生效，交互模式下使用原有的用户确认机制
- 提交失败与拒绝：不阻断主流程；输出预览与建议；支持用户自定义回复作为附加提示；会话继续。
- 统计容错：shortstat 获取失败时忽略，不影响最终输出；StatsManager 调用异常被捕获。
- 单实例锁：按仓库维度加锁失败时回退到全局锁，确保至少具备互斥保护。
- 规则文件读取：全局规则与项目规则读取失败时忽略，仅使用基础系统提示。

7. 扩展与二次开发建议

- 工具扩展：通过 –append-tools 追加白名单；在内置或用户目录新增工具（具体加载由 Agent 处理）。
- 事件增强：除 AFTER_TOOL_CALL 外的扩展由 Agent 支持；CodeAgent 保持单事件订阅策略以控制复杂度。
- 代码分析器扩展：
    - 语言支持：在 `languages/` 目录下添加新语言的 SymbolExtractor 和 DependencyAnalyzer 实现
    - 符号提取：扩展符号类型（如装饰器、宏、类型别名等）
    - 依赖分析：支持更复杂的依赖关系（如动态导入、条件导入等）
    - 影响分析：扩展影响类型（如性能影响、安全影响等）
    - 测试发现：扩展测试文件命名模式和发现策略
- 上下文推荐扩展：
    - 关键词提取：优化 LLM 提示词，提高关键词提取准确性
    - 符号搜索：扩展搜索策略（如模糊匹配、正则匹配等）
    - 推荐筛选：优化 LLM 筛选逻辑，提高推荐相关性
    - 推荐策略：添加基于代码相似度、调用频率等的推荐策略
- 构建验证扩展：
    - 构建系统：添加新构建系统的支持（如 Bazel、Buck、Buck2 等）
    - 构建命令：支持自定义构建命令和参数
    - 构建环境：支持在容器或虚拟环境中执行构建
    - 增量构建：支持只验证修改的文件（如 cargo check –package）
- lint 建议：
    - 扩展获取静态检查工具映射（在静态检查模块中扩展静态检查工具字典）
    - 支持从 `{get_data_dir()}/lint_tools.yaml` 加载自定义配置
    - 基于配置开关（is_enable_static_analysis）进行自动引导与集中修复流程提示
    - 添加 lint 工具的自定义参数和配置选项
- 提交策略：自定义提交工作流（提交信息模板、分组或批量提交）；通过 prefix/suffix 参数注入自定义信息。
- 统计集成：扩展指标（语言分布、文件修改热度、影响范围统计）以增强可观测性；在工具调用后回调方法中追加统计记录点。
- 配置策略：引入项目级配置（.jarvis/config）以调节阈值（如大变更 > 300 行、影响分析深度等）与行为开关。
- 规则扩展：在全局规则文件（`{get_data_dir()}/rules`）或项目规则文件（`.jarvis/rules`）中添加项目特定规范。
- 补丁预览策略：自定义大变更阈值（当前 300 行）；扩展特殊文件状态的处理逻辑（如符号链接、二进制文件）。
- 影响分析扩展：
    - 影响范围可视化：生成影响范围的可视化图表
    - 影响评估优化：使用机器学习模型评估影响严重程度
    - 自动修复建议：基于影响分析结果生成自动修复建议

8. CodeAgent.run 方法详细流程

- 执行流程（run 方法）：
    1. 环境初始化（_init_env）：
        - 查找 git 根目录
        - 更新 .gitignore
        - 处理未提交修改（使用 prefix/suffix）
        - 配置换行符设置
    2. 记录起始提交哈希：`start_commit = get_latest_commit_hash()`
    3. 获取项目统计信息：
        - 代码统计（获取代码行数统计方法）：代码行数等统计信息
        - 最近提交（获取最近提交及文件方法）：最近 5 次提交及其修改文件列表
        - Git托管文件信息（获取Git跟踪文件信息方法）：文件列表或目录结构（文件数量超过阈值时返回目录结构）
    4. 智能上下文推荐（可选）：
        - 若启用意图识别（is_enable_intent_recognition）且上下文推荐器已初始化：
            - 调用 ContextRecommender.recommend_context(user_input) 生成上下文推荐
            - 推荐流程：
                - 使用 LLM 从用户输入中提取关键词
                - 基于关键词在符号表和代码文本中搜索候选符号
                - 使用 LLM 从候选符号中筛选关联度高的条目
                - 返回推荐符号列表（包含文件路径和行号）
            - 格式化推荐结果并追加到用户输入
    5. 增强用户输入：
        - 添加项目概况（代码统计 + Git 文件信息 + 最近提交）
        - 添加规范提示（“先分析再修改”、“优先使用 edit_file” 等）
        - 添加智能上下文推荐（若生成）
    6. 运行 Agent：`self.agent.run(enhanced_input)`
    7. 任务完成后处理：
        - 处理未提交修改（_handle_uncommitted_changes）：交互式确认并提交
        - 记录结束提交哈希：`end_commit = get_latest_commit_hash()`
        - 显示提交历史（_show_commit_history）：展示 start_commit 到 end_commit 之间的提交
        - 处理提交确认（_handle_commit_confirmation）：
            - 若用户接受提交：重置到起始提交，使用 GitCommitTool 重新提交（应用 prefix/suffix）
            - 若用户拒绝：询问是否重置到起始提交
            - 若启用强制保存记忆：在用户接受提交后触发记忆保存

9. 典型使用场景（面向代码工程）

- 小粒度修改：工具生成补丁后，自动预览 diff 并辅以提交；必要时注入静态扫描提示。
- 批量重构：多文件大变更时按文件摘要化展示；提交后生成聚合静态检查引导。
- 非交互流水线：CI 场景以 –non-interactive + –requirement 运行，受限时间内完成工具执行与提交。
- 会话恢复与断点续跑：通过 –restore-session 恢复上下文，继续在 AFTER_TOOL_CALL 旁路进行增强。
- 项目规范注入：通过全局规则（`{get_data_dir()}/rules`）或项目规则（`.jarvis/rules`）注入项目特定规范。
- 自定义工具扩展：通过 –append-tools 追加特定工具（如代码生成工具、测试工具等）。

10. 总结

- CodeAgent 专注于“代码工程”场景的外围增强：环境与仓库管理、提交工作流、差异预览与统计、静态检查引导、代码分析、影响范围分析、构建验证。
- 代码分析器模块提供强大的代码理解能力：
    - 上下文管理：维护符号表和依赖图，支持代码结构查询
    - 影响范围分析：自动分析编辑影响，识别风险并提供建议
    - 智能上下文推荐：使用 LLM 进行语义理解，推荐相关代码位置
    - 构建验证：自动检测并验证构建系统，确保代码修改后项目仍能正常构建
    - 静态分析：根据文件类型自动选择和执行 lint 工具
- 通过“启动参数 + 事件订阅”的最小侵入方式使用 Agent，避免对底层实现的耦合与依赖。
- 借助 CLI 与配置体系，兼顾交互与非交互场景，提供稳定可控、可扩展的工程化实践路径。
- 所有增强功能均具备完善的容错机制，确保单个功能失败不影响主流程。

jarvis-sec 系统架构设计

本文档基于源码目录 src/jarvis/jarvis_sec 下的实现，对“Jarvis安全分析套件（jarvis-sec）”进行结构化架构说明，覆盖模块组成、模块关系、工作流程、以及各模块内部设计。面向本项目开发者与使用者。

参考风格：与本仓库现有架构文档一致，使用 PlantUML 以通俗术语呈现角色与流程，强调职责边界与可回退策略。

- 主要源码模块：
    - cli.py（Typer 命令行入口）
    - **init**.py（主工作流：run_security_analysis，协调各阶段执行）
    - workflow.py（直扫基线、快速模式输出、Markdown 格式化）
    - report.py（统一报告聚合器：JSON + Markdown）
    - prompts.py（提示词构建：分析、验证、聚类、复核等阶段的提示词）
    - parsers.py（解析函数：从Agent输出中解析JSON格式的结果）
    - utils.py（工具函数：文件操作、候选处理等）
    - file_manager.py（文件管理器：统一的3个配置文件读写接口，断点恢复状态检查）
    - agents.py（Agent创建：分析、验证、聚类、复核Agent的创建与事件订阅）
    - clustering.py（聚类相关：聚类任务构建、执行、结果处理等）
    - analysis.py（分析相关：分析任务构建、Agent执行、结果过滤等）
    - verification.py（验证相关：验证任务构建、Agent执行、结果合并等）

    - review.py（复核相关：复核任务构建、Agent执行、结果处理等）
    - checkers/（语言检查规则集合）
        - c_checker.py（C/C++ 启发式规则）
        - rust_checker.py（Rust 启发式规则）
    - types.py（问题数据结构：Issue）
- 核心数据目录与产物（默认路径）：
    - /.jarvis/sec/
        - candidates.jsonl（只扫结果文件：保存每个原始告警的信息，包括gid）
        - clusters.jsonl（聚类信息文件：所有聚类（包括无效聚类），每个聚类包括的gids）
        - analysis.jsonl（分析结果文件：包括所有聚类，聚类中哪些问题是问题，哪些问题是误报）

1. 设计目标与总体思路

系统采用“四阶段流水线”设计：启发式扫描 → 聚类 → 复核 → 分析 → 报告。

四阶段概述

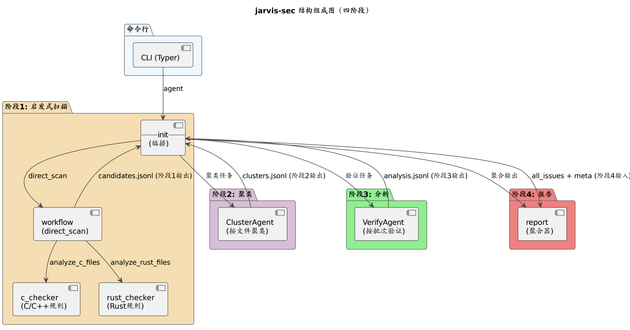
1. **启发式扫描阶段**：在不依赖外部服务的前提下，通过纯 Python + 启发式检查器提供“可复现、可离线”的安全问题直扫基线（direct_scan）。输出候选问题列表（candidates.jsonl），每个候选包含gid用于唯一标识。
2. **聚类阶段**：将候选问题按文件分组，使用单Agent对每个文件内的告警进行“验证条件一致性”聚类，生成聚类批次（clusters.jsonl）。聚类Agent可以在聚类阶段判断告警是否无效（is_invalid字段），如果标记为无效，必须提供充分的invalid_reason（包括已检查的所有调用路径、保护措施、为什么不存在触发路径等）。无效的告警将进入复核阶段。聚类时会进行完整性校验，确保所有候选的gid都被聚类。
3. **复核阶段**：对聚类Agent标记为无效的告警，由复核Agent验证其无效理由是否充分。复核Agent会检查聚类Agent是否真的考虑了所有可能的路径、调用者和边界情况。如果理由不充分，将候选重新加入验证流程；如果理由充分，确认为无效。
4. **分析阶段**：将每个聚类批次交由分析Agent执行“只读验证”，确认是否存在真实安全风险。分析Agent必须进行完整的调用路径推导，明确说明从可控输入到缺陷代码的完整调用链。如果分析Agent确认有告警，会启动验证Agent进行二次验证，只有验证通过的告警才会写入文件（analysis.jsonl）。analysis.jsonl 明确区分问题和误报，每个聚类记录包含 verified_gids（验证为问题的gid）和 false_positive_gids（验证为误报的gid）。
5. **报告阶段**：通过报告聚合器将所有确认问题聚合为 JSON + Markdown 报告，包含统计概览与详细条目。

核心设计原则

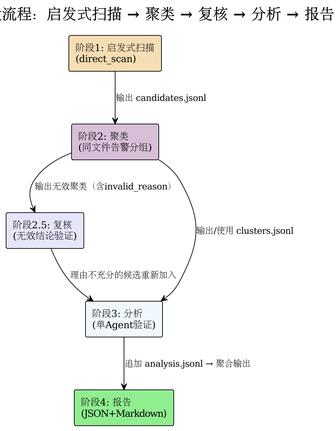
- 只读与安全：Agent 仅使用 read_code/execute_script 两类只读工具，不修改仓库；如检测到工作区变更则自动恢复（git checkout -.）。
- JSONL 产物与断点续扫：各阶段产物集中在 .jarvis/sec，通过3个清晰的配置文件（candidates.jsonl、clusters.jsonl、analysis.jsonl）支持断点续扫。系统根据这3个文件的存在性和状态自动推断恢复点，无需依赖额外的进度日志文件。
- 简洁集成：提供 direct_scan（直扫一键输出）与 run_security_analysis（完整四阶段流水线）两类入口。
- 二次验证机制：分析Agent确认告警后，验证Agent会进行二次验证，特别验证分析Agent的调用路径推导是否正确，只有验证通过的告警才会写入文件，减少误报。
- 记忆系统集成：聚类Agent、复核Agent、分析Agent和验证Agent都支持记忆工具（save_memory/retrieve_memory），可以保存和复用分析经验，提高分析效率和准确性。
- 无效结论复核机制：聚类Agent标记告警为无效时，必须提供充分的invalid_reason。复核Agent会验证这些理由是否充分，如果理由不充分，将候选重新加入验证流程，确保不会因为理由不充分而误过滤真实问题。
- 调用路径推导要求：分析Agent必须进行完整的调用路径推导，明确说明从可控输入到缺陷代码的完整调用链，以及每个调用点的校验情况，确保漏洞触发路径的真实性和可验证性。

2. 模块组成（PlantUML）

下图从“四阶段流水线”视角呈现模块静态组成与依赖关系。



四阶段流程图（简化）



要点

- 五阶段依次执行：启发式扫描 → 聚类 → 复核 → 分析 → 报告，每阶段产物作为下一阶段输入。
- 复核阶段：对聚类Agent标记为无效的告警进行复核，如果理由不充分，将候选重新加入验证流程。
- CLI 的 scan 子命令执行完整四阶段流水线，当 Agent 分析失败或无输出时自动回退到直扫基线（仅阶段1）。
- 数据产物在各阶段生成并复用，形成稳定的“分析上下文”。
- Agent 仅使用只读工具，确保分析过程不修改仓库状态。

3. 核心产物与文件约定

按四阶段分类：

阶段1：启发式扫描产物

- candidates.jsonl：只扫结果文件，保存每个原始告警的信息，包括gid。覆盖模式写入，确保文件内容是最新的。每个候选都有唯一的gid，包含所有原始扫描信息。

阶段2：聚类产物

- clusters.jsonl：聚类信息文件，所有聚类（包括无效聚类），每个聚类包括的gids。追加模式写入，支持增量保存。每个聚类记录包含 cluster_id（唯一标识，格式为 file_path|batch_index|cluster_index）、file、batch_index、cluster_index、gids、verification、is_invalid、invalid_reason 等字段。聚类时会进行完整性校验，确保所有候选的gid都被聚类。

阶段3：分析产物

- analysis.jsonl：分析结果文件，包括所有聚类，聚类中哪些问题是问题，哪些问题是误报。追加模式写入，支持增量保存。每个分析结果记录包含 cluster_id、file、batch_index、cluster_index、gids、verified_gids（验证为问题的gid列表）、false_positive_gids（验证为误报的gid列表）、issues（详细的问题列表，仅verified_gids对应的）等字段。

阶段4：报告产物

- 最终输出（字符串）：通过报告聚合器将 issues 聚合，返回“JSON + Markdown”文本（字符串，包含统计与详细条目）。

断点恢复机制

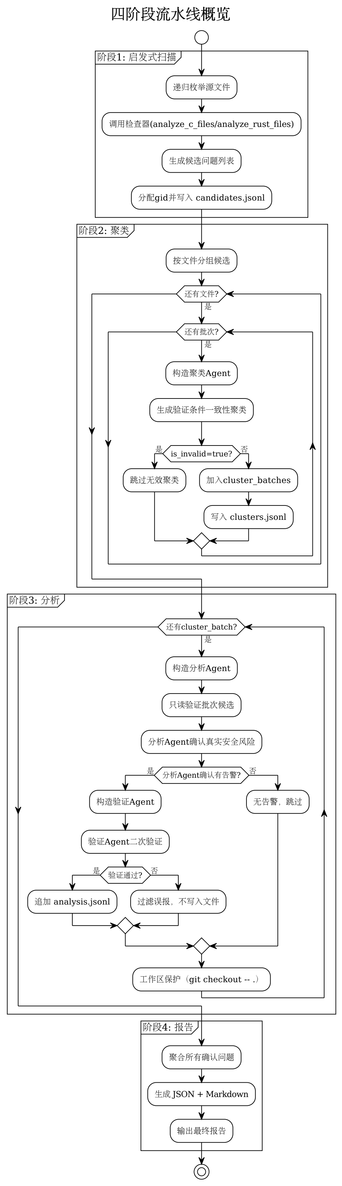
- 系统根据3个配置文件（candidates.jsonl、clusters.jsonl、analysis.jsonl）的存在性和状态自动推断断点恢复状态：
    - 只扫阶段：检查 candidates.jsonl 是否存在，如果存在则直接加载候选
    - 聚类阶段：检查 clusters.jsonl 是否存在，如果存在则加载已有聚类，并校验聚类完整性
    - 分析阶段：检查 analysis.jsonl 是否存在，如果存在则加载已有分析结果，跳过已分析的聚类

4. 命令与工作流程

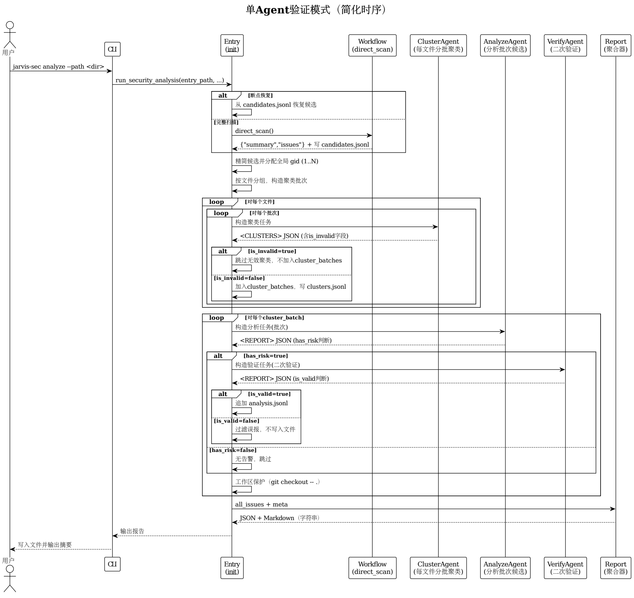
命令行子命令（CLI）

- config：管理配置文件（.jarvis/jsec/config.json），可设置 target、languages、exclude_dirs、output、cluster_limit、enable_verification、force_save_memory、llm_group 等
- scan：执行完整四阶段流水线（启发式扫描 → 聚类 → 分析 → 报告），从配置文件读取参数，输出最终报告；Agent 分析失败时自动回退到直扫基线
- analyze：从外部 JSON 文件分析安全问题，支持标准格式直接分析或非标准格式自动转换后分析

四阶段流水线（PlantUML）

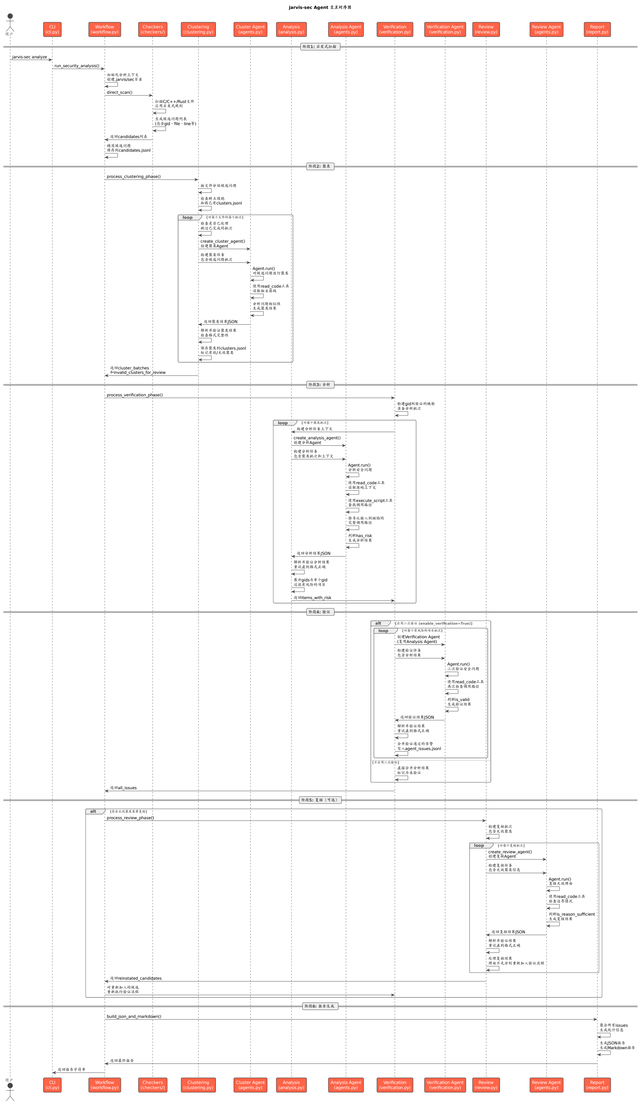


典型流水线（PlantUML）



Agent 交互时序图：

以下时序图展示了 jarvis-sec 模块中各个 Agent 的交互流程：



时序图说明：

1. **阶段1 - 启发式扫描**：Checkers 模块使用纯 Python 正则/启发式规则扫描代码，生成候选问题列表（不依赖 Agent）。
2. **阶段2 - 聚类**：
   - Cluster Agent 对候选问题进行聚类，将相似问题分组
   - 支持断点续跑，基于 clusters.jsonl 恢复已完成批次
   - 聚类结果包含有效聚类和无效聚类
3. **阶段3 - 分析**：
   - Analysis Agent 对聚类后的批次进行深入分析
   - 必须推导从可控输入到缺陷代码的完整调用路径
   - 判断每个问题是否有真实风险（has_risk）
4. **阶段4 - 验证**：
   - Verification Agent 对分析 Agent 确认的问题进行二次验证
   - 可选的验证阶段（可通过 enable_verification 关闭）
   - 验证通过的问题写入 agent_issues.jsonl
5. **阶段5 - 复核**（可选）：
   - Review Agent 对无效聚类进行复核
   - 判断无效理由是否充分（is_reason_sufficient）
   - 理由不充分的候选重新加入验证流程
6. **阶段6 - 报告生成**：
   - 聚合所有验证通过的问题
   - 生成 JSON 和 Markdown 格式的最终报告

Agent 特点：

- **严格只读**：所有 Agent 仅使用只读工具（read_code、execute_script），不修改代码仓库
- **工作区保护**：每次 Agent 运行后自动恢复工作区（git checkout – .）
- **断点续跑**：支持基于 candidates.jsonl、clusters.jsonl、analysis.jsonl 的断点恢复
- **格式验证与重试**：对 Agent 输出进行格式验证，失败时自动重试直到格式正确
- **记忆功能**：支持使用 save_memory 和 retrieve_memory 工具保存和检索分析记忆

5. 模块内部设计

按四阶段组织模块内部设计：

5.1 阶段1：启发式扫描（workflow.direct_scan）

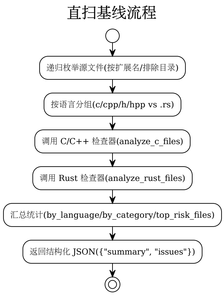
职责

- 递归枚举源文件（默认扩展：c, cpp, h, hpp, rs；排除目录：.git/build/out/target/third_party/vendor）。
- 按语言分组并调用检查器（analyze_c_files / analyze_rust_files），收集 Issue。
- 汇总统计（by_language/by_category/top_risk_files/scanned_files/scanned_root），返回结构化 JSON。

关键接口（源码参考）

- _iter_source_files：递归枚举源文件，支持扩展名过滤与目录排除
- direct_scan：主入口，调用检查器并汇总统计

流程（PlantUML）



边界与容错

- 文件读取失败时跳过并继续；排除目录可配置；检查器返回空列表不影响汇总。
- 若 ripgrep 不可用，回退为纯 Python 扫描（不影响功能）。

5.2 阶段2：聚类（init 中的聚类逻辑）

职责：将候选问题按文件分组，使用单Agent对每个文件内的告警进行“验证条件一致性”聚类，并可以在聚类阶段判断告警是否无效。如果标记为无效，必须提供充分的理由。

职责（精细拆解）

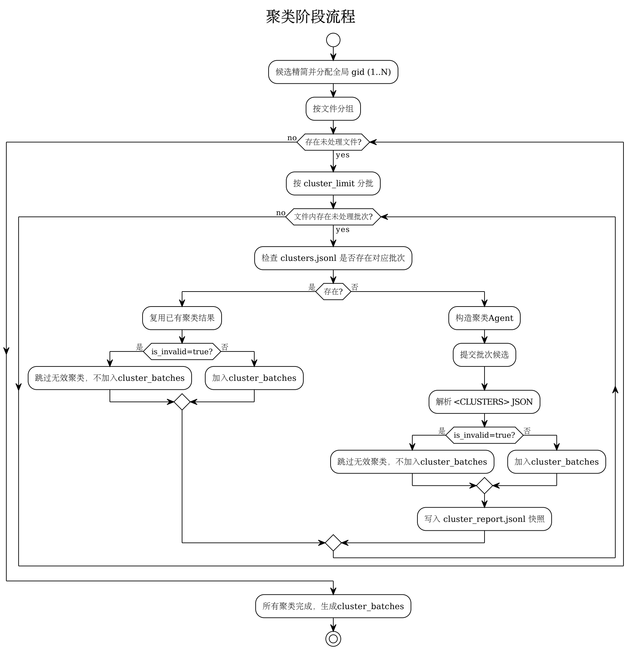
- 候选精简与全局编号：
  - 将候选精简为 compact_candidates（language/category/pattern/file/line/evidence/confidence/severity），为每条分配全局 gid（1..N）。
- 按文件分组：
  - 将候选按文件分组，每个文件内按 cluster_limit 分批（默认 50）。

- 构造聚类 Agent：
  - 以“验证条件一致性”为准聚合相近告警。
  - 工具支持：read_code/execute_script/save_memory/retrieve_memory（记忆工具）。
  - 保守判断原则：必须充分考虑所有可能的路径、调用链和边界情况。只要存在任何可能性（即使很小）导致漏洞可被触发，就不应该标记为无效。只有在完全确定、所有路径都已验证安全的情况下，才能标记为无效。
  - 摘要输出格式：仅在 ... 内输出 JSON 数组（支持 json5 语法，如尾随逗号、注释等）：
    - verification: 字符串（本簇验证条件）
    - gids: 整数数组（属于该簇的全局编号）
    - is_invalid: 布尔值（必填，true 或 false）。如果为 true，表示该聚类中的所有候选已被确认为无效/误报，将进入复核阶段。
    - invalid_reason: 字符串（当 is_invalid 为 true 时必填）。必须详细说明为什么这些候选是无效的，包括：
      - 已检查的所有调用路径和调用者
      - 已确认的保护措施和校验逻辑
      - 为什么这些保护措施在所有路径上都有效
      - 为什么不存在任何可能的触发路径
      - 必须足够详细，以便复核Agent能够验证判断
- 无效告警收集：
  - 如果聚类结果中 is_invalid 为 true，收集到 invalid_clusters_for_review 列表（包含 invalid_reason 和 members），提交给复核Agent验证。
- 断点恢复：
  - 支持断点恢复：若 clusters.jsonl 存在，优先复用已有聚类结果。如果断点记录中 is_invalid 为 true，且包含 invalid_reason，可以复用；如果缺少 invalid_reason，需要重新聚类。聚类时会进行完整性校验，确保所有候选的gid都被聚类。
- 写入聚类快照：
  - 写入聚类结果到 clusters.jsonl，每个聚类记录包含 cluster_id（唯一标识）、file、batch_index、cluster_index、gids、verification、is_invalid、invalid_reason 等字段。

关键接口（源码参考）

- _parse_clusters_from_text：解析 JSON 内容（使用 json5 解析，支持尾随逗号、注释等）
- save_cluster：写入聚类快照到 clusters.jsonl（通过 file_manager 模块）
- 聚类Agent工具：read_code/execute_script/save_memory/retrieve_memory（记忆工具）

流程（PlantUML）



边界与容错

- 聚类摘要解析失败时重试（至多 2 次）；失败批次仍标记进度并继续。
- is_invalid 字段必填，如果缺失会触发重试，不会抛出异常。
- 当 is_invalid 为 true 时，invalid_reason 字段必填，如果缺失或为空会触发重试。
- 聚类快照写入失败不阻断主流程，只打印或记录事件。
- 无效聚类（is_invalid=true）收集到 invalid_clusters_for_review，提交给复核Agent验证，不直接跳过。

5.2.1 阶段2.5：复核（init 中的复核逻辑）

职责：对聚类Agent标记为无效的告警，由复核Agent验证其无效理由是否充分。如果理由不充分，将候选重新加入验证流程。

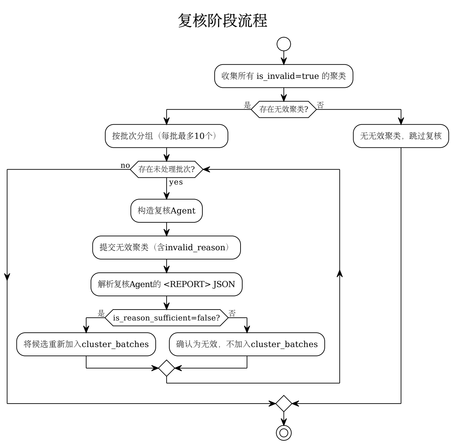
职责（精细拆解）

- 收集无效聚类：
  - 聚类阶段收集的所有 is_invalid=true 的聚类（包含 invalid_reason 和 members）进入复核阶段。
- 构造复核 Agent：
  - 工具支持：read_code/execute_script/retrieve_memory（召回记忆工具）。
  - 复核原则：
    - 必须验证聚类Agent是否真的检查了所有可能的调用路径和调用者。
    - 必须验证聚类Agent是否真的确认了所有路径都有保护措施。
    - 如果发现聚类Agent遗漏了某些路径、调用者或边界情况，必须判定为理由不充分。
    - 保守策略：有疑问时，一律判定为理由不充分，将候选重新加入验证流程。
  - 摘要输出格式：仅在 ... 内输出 JSON 数组（支持 json5 语法），每个元素：
    - gid: int（全局编号，对应无效聚类的gid）
    - is_reason_sufficient: bool（无效理由是否充分）
    - review_notes: string（复核说明，解释为什么理由充分或不充分）
- 复核结果处理：
  - 如果 is_reason_sufficient 为 false，将候选重新加入 cluster_batches，进入后续验证阶段。
  - 如果 is_reason_sufficient 为 true，确认为无效，不进入后续验证阶段。
  - 复核结果解析失败时，保守策略：将所有候选重新加入验证流程。
- 按批次复核：
  - 每批最多处理 10 个无效聚类，避免上下文过长。
- 工作区保护：
  - 每次运行复核Agent后检测 git 工作区是否有变更；如有通过 git checkout -- . 恢复。

关键接口（源码参考）

- _try_parse_summary_report：解析 JSON 内容
- 复核Agent工具：read_code/execute_script/retrieve_memory（召回记忆工具）

流程（PlantUML）



边界与容错

- 复核摘要解析失败时，保守策略：将所有候选重新加入验证流程，确保不会因为技术问题而遗漏潜在安全问题。
- 工作区保护：若检测到文件被修改，立即恢复，确保分析只读。

5.3 阶段3：分析（init 中的验证逻辑）

职责：将每个聚类批次交由分析Agent执行“只读验证”，确认是否存在真实安全风险。分析Agent必须进行完整的调用路径推导。如果分析Agent确认有告警，会启动验证Agent进行二次验证。

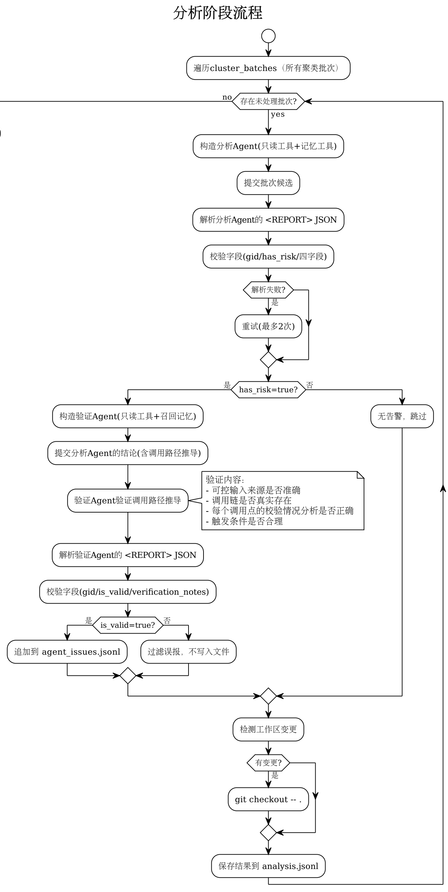
职责（精细拆解）

- 构造验证批次：
  - 基于聚类结果构造验证批次（每个批次包含同一验证条件的候选）。
- 构造分析 Agent：
  - 工具限制：read_code/execute_script/save_memory/retrieve_memory（记忆工具）。
  - **必须进行调用路径推导**：
    - 对于每个候选问题，必须明确推导从可控输入到缺陷代码的完整调用路径。
    - 调用路径推导必须包括：
      1. 识别可控输入的来源（例如：用户输入、网络数据、文件读取、命令行参数等）
      2. 追踪数据流：从输入源开始，逐步追踪数据如何传递到缺陷代码位置
      3. 识别调用链：明确列出从入口函数到缺陷代码的所有函数调用序列（例如：main() -> parse_input() -> process_data() -> vulnerable_function()）
      4. 分析每个调用点的数据校验情况：检查每个函数是否对输入进行了校验、边界检查或安全检查
      5. 确认触发条件：明确说明在什么条件下，未校验或恶意输入能够到达缺陷代码位置
    - 如果无法推导出完整的调用路径，或者所有调用路径都有充分的保护措施，则应该判定为误报。
    - 调用路径推导必须在分析过程中明确展示，不能省略或假设。
  - **调用路径追溯要求**：
    - 必须向上追溯所有可能的调用者，查看完整的调用路径，以确认风险是否真实存在。
    - 使用 read_code 和 execute_script 工具查找函数的调用者（例如：使用 grep 搜索函数名，查找所有调用该函数的位置）。
    - 对于每个调用者，必须检查其是否对输入进行了校验。
    - 如果发现任何调用路径未做校验，必须明确记录该路径。
  - 强制保存记忆：force_save_memory=True，在关键节点强制提示/执行记忆保存。
  - 记忆使用提示：提示Agent使用函数名作为tag，记忆每个函数的要点（如指针判空、输入校验、调用路径分析结果等），分析时充分利用记忆。
  - 摘要输出格式：仅在 … 内输出 JSON 数组（支持 JSON5 语法），每个元素：
    - gid: int（全局编号，>=1）
    - has_risk: bool
    - preconditions: string（触发漏洞的前置条件，当 has_risk=true 时必需）
    - trigger_path: string（漏洞的触发路径，必须包含完整的调用路径推导，包括：1）可控输入的来源；2）从输入源到缺陷代码的完整调用链（函数调用序列）；3）每个调用点的数据校验情况；4）触发条件。格式示例：“调用路径推导：函数A() -> 函数B() -> 函数C() -> 缺陷代码。数据流：输入来源 -> 传递路径。关键调用点：函数B()未做校验。”，当 has_risk=true 时必需）
    - consequences: string（漏洞被触发后可能导致的后果，当 has_risk=true 时必需）
    - suggestions: string（修复或缓解该漏洞的建议，当 has_risk=true 时必需）
- 解析与校验：
  - 解析严格校验字段与类型；如果分析Agent确认有告警（has_risk=true），不立即写入文件。
- 二次验证机制：
  - 如果分析Agent确认有告警，创建验证Agent进行二次验证。
  - 验证Agent工具：read_code/execute_script/retrieve_memory（召回记忆工具）。
  - **验证Agent职责**：
    - 验证分析Agent给出的前置条件、触发路径、后果和建议是否合理、准确。
    - 特别需要验证分析Agent的调用路径推导是否正确：
      - 验证分析Agent识别的可控输入来源是否准确
      - 验证分析Agent给出的调用链是否真实存在
      - 验证分析Agent对每个调用点的校验情况分析是否正确
      - 验证分析Agent的触发条件是否合理
    - 必要时需向上追溯调用者，查看完整的调用路径，以确认分析Agent的结论是否成立。
    - 充分利用记忆工具检索分析Agent保存的调用路径分析结果，如果发现分析Agent的结论与记忆中的信息不一致，需要仔细核实。
  - 验证Agent摘要输出格式：仅在 … 内输出 JSON 数组（支持 json5 语法），每个元素：
    - gid: int（全局编号）
    - is_valid: bool（分析Agent的结论是否正确，包括调用路径推导是否正确）
    - verification_notes: string（验证说明，解释为什么结论正确或不正确，特别说明调用路径推导的验证结果）
  - 只有验证Agent确认（is_valid=true）的告警才会写入 .jarvis/sec/agent_issues.jsonl。
  - 验证不通过的告警会被过滤，不写入文件。
- 支持重试：
  - 摘要解析失败或关键字段缺失时，最多重试 2 次。
- 工作区保护：
  - 每次运行 Agent 后检测 git 工作区是否有变更；如有通过 git checkout – . 恢复，记录 meta（workspace_restore）。
- 进度追踪：
  - 记录 batch_status 与 task_status 事件，并将每个候选标记为已处理。
  - 基于进度文件跳过已完成的候选（通过 candidate_signature 匹配）。

关键接口（源码参考）

- _try_parse_summary_report：解析 JSON 内容（使用 json5 解析，支持尾随逗号、注释等）
- _build_summary_prompt：构建分析Agent摘要提示词
- _build_verification_summary_prompt：构建验证Agent摘要提示词
- _git_restore_if_dirty：工作区保护（git checkout – .）

流程（PlantUML）



边界与容错

- 验证摘要解析失败时重试（至多 2 次）；失败批次仍标记进度并继续。
- 严禁 Agent 写操作；通过工具白名单（read_code/execute_script/save_memory/retrieve_memory）与只读约束保证安全。
- 工作区保护：若检测到文件被修改，立即恢复，确保分析只读。
- 二次验证机制：只有分析Agent和验证Agent都确认的告警才会写入文件。验证Agent特别验证分析Agent的调用路径推导是否正确，包括验证调用链是否真实存在、每个调用点的校验情况分析是否正确等，减少误报。
- 记忆系统：Agent可以使用 save_memory 保存分析要点，使用 retrieve_memory 检索已有记忆，提高分析效率和准确性。

5.4 阶段4：报告（report 模块）

职责：通过报告聚合器将所有确认问题聚合为 JSON + Markdown 报告。

职责（精细拆解）

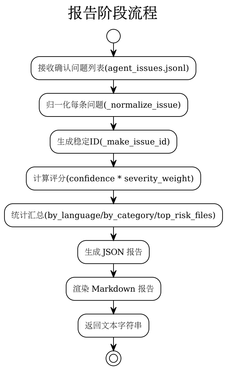
- 数据归一化：
  - _normalize_issue：归一化字段并补充缺省值（language/category/pattern/file/line/evidence/…）
  - _make_issue_id：基于文件/行/类别/模式哈希生成稳定 ID（C/R 前缀）
- 评分计算：
  - score = confidence * severity_weight（high:3.0, medium:2.0, low:1.0）
- 统计汇总：
  - summary.total：总问题数
  - summary.by_language：按语言统计（c/cpp, rust）
  - summary.by_category：按类别统计（unsafe_api, buffer_overflow, memory_mgmt, …）

    - summary.top_risk_files：按累计分排序的前 10 个风险文件
- 报告生成：
    - aggregate_issues：聚合问题列表并生成 JSON 报告（summary + issues）
    - format_markdown_report：将聚合后的 JSON 报告渲染为 Markdown
    - build_json_and_markdown：一次性生成报告文本（仅 Markdown）

关键接口（源码参考）

- aggregate_issues：聚合问题列表并生成 JSON 报告
- format_markdown_report：将聚合后的 JSON 报告渲染为 Markdown
- build_json_and_markdown：一次性生成报告文本

流程（PlantUML）



边界与容错

- 接受 Issue/dict 两种形态；缺失字段使用缺省值；评分四舍五入到 2 位小数。
- Markdown 渲染失败时返回空字符串；JSON 序列化失败抛出异常。

5.5 checkers/c_checker（C/C++ 启发式规则）

说明：本模块属于阶段1（启发式扫描）的检查器实现。

职责

- 针对 .c/.cpp/.h/.hpp 文件识别基础安全问题，输出 Issue 列表，字段包含 language/category/pattern/file/line/evidence/description/suggestion/confidence/severity。
- 支持多种规则类别：unsafe_api、buffer_overflow、memory_mgmt、error_handling、unsafe_usage、concurrency、thread_safety、input_validation、crypto、resource_leak、network_api、insecure_permissions 等。

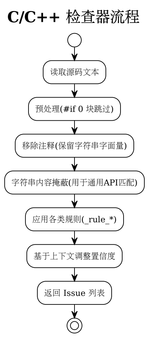
关键规则（正则/启发式）

- unsafe_api：strcpy/strcat/gets/sprintf/vsprintf 等不安全字符串 API
- boundary_funcs：memcpy/memmove/strncpy/strncat 等边界操作，需确认长度来源
- realloc_assign_back：realloc 直接覆盖原指针，可能导致内存泄漏
- malloc_no_null_check：内存分配后未见 NULL 检查
- uaf_suspect：use-after-free 线索（free 后解引用使用）
- unchecked_io：I/O/系统调用可能未检查返回值
- format_string：格式化字符串参数不是字面量
- insecure_tmpfile：tmpnam/tempnam/mktemp 不安全临时文件 API
- command_execution：system/popen/exec* 命令执行，参数非字面量
- scanf_no_width：scanf 使用 %s 但未指定最大宽度
- possible_null_deref：指针解引用附近未见 NULL 检查
- uninitialized_ptr_use：野指针使用（声明后未初始化即解引用）
- deadlock_patterns：双重加锁、锁顺序反转、缺失解锁
- double_free_and_free_non_heap：重复释放、释放非堆内存
- 其他：atoi_family、rand_insecure、strtok_nonreentrant、open_permissive_perms、alloca_unbounded、vla_usage、pthread_returns_unchecked、cond_wait_no_loop、thread_leak_no_join、inet_legacy、time_apis_not_threadsafe、getenv_unchecked 等

实现要点（准确性优化）

- 注释移除：_remove_comments_preserve_strings 移除注释（保留字符串/字符字面量），避免注释中的 API 命中导致误报。
- 字符串掩蔽：_mask_strings_preserve_len 将字符串字面量内容替换为空格，避免将字符串中的片段（如 “system(”）当作代码。
- 条件编译跳过：_strip_if0_blocks 跳过 #if 0 … #endif 块。
- 上下文检测：_has_null_check_around、_has_len_bound_around 等辅助函数检测邻近上下文，降低误报。
- 置信度调整：基于上下文线索（NULL 检查、边界检查、SAFETY 注释、测试上下文等）动态调整置信度。

流程（PlantUML）



边界与容错

- 文件读取失败时返回空列表；正则匹配失败不影响其他规则；置信度始终落在 [0.4, 0.95] 区间。
- 头文件声明行（typedef/extern）跳过，避免将函数原型误报为调用。

5.6 checkers/rust_checker（Rust 启发式规则）

说明：本模块属于阶段1（启发式扫描）的检查器实现。

职责

- 针对 .rs 文件识别基础安全问题，输出 Issue 列表，字段包含 language/category/pattern/file/line/evidence/description/suggestion/confidence/severity。

关键规则（正则/启发式）

- unsafe：存在不安全代码块/标识（检测 SAFETY 注释与测试上下文降低置信度）
- raw_pointer：出现原始指针（*mut*/const）
- mem::transmute：不安全类型转换
- mem::forget：跳过 Drop 导致资源泄漏风险
- MaybeUninit/assume_init：初始化与读取顺序问题
- unwrap/expect：错误处理不充分，可能 panic
- extern “C”：FFI 边界风险（指针有效性/对齐/生命周期/线程安全）
- unsafe impl Send/Sync：并发内存模型风险
- ignored*result*：*let* = … 或 .ok() 等可能忽略错误

实现要点

- 邻域窗口与 SAFETY 注释检测：_window/_has_safety_comment_around 检测邻近 SAFETY 注释（支持中英文），降低置信度。
- 测试上下文检测：_in_test_context 检测 #[test]/cfg(test)/mod tests，适度降低严重度。
- 置信度到严重性映射：_severity_from_confidence（>=0.8→high, >=0.6→medium, <0.6→low）。

流程（PlantUML）

边界与容错

- 文件读取失败时返回空列表；规则匹配失败不影响其他规则；置信度始终落在合理区间。
- 避免对 unsafe impl 重复上报（由专门规则处理）。

5.7 cli（命令行协调者）

职责

- 使用 Typer 暴露子命令：`config`（配置管理）、`analyze`（LLM 聚类与验证分析）和 `scan`（直扫生成基线报告）。
- 提供参数解析与错误处理，统一输出错误告警与阶段性摘要。

关键行为

- init_env：初始化运行环境与欢迎提示
- 懒加载：避免未使用模块的硬依赖
- 错误与摘要：统一输出错误告警与阶段性摘要
- 回退策略：当 Agent 分析过程出错或无输出时，自动回退到直扫基线（direct_scan），确保至少输出基础扫描结果

参数说明

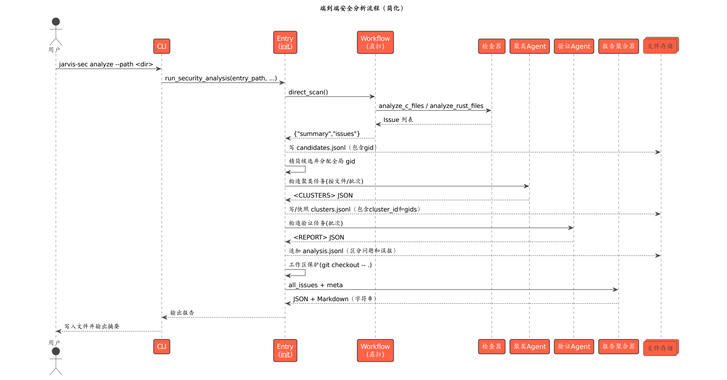
- –path/-p：待分析的根目录（必选）
- –llm-group/-g：使用的模型组（可选，仅对本次运行生效）
- –output/-o：最终Markdown报告输出路径（默认 ./report.md）
- –cluster-limit/-c：聚类每批最多处理的告警数（默认 50）

边界与容错

- 环境初始化失败不应阻塞 CLI 基础功能；报告写入失败打印错误但不中断流程。
- Agent 分析过程出错时回退到直扫基线，确保至少输出基础结果。

6. 模块间交互流程（端到端）

典型“直扫 → 聚类 → 验证 → 聚合”的端到端时序。



要点

- 每一阶段产物作为下一阶段输入，失败时提供容忍与继续策略；产物集中在 .jarvis/sec 便于断点续扫。
- Agent 全程只读；检测到工作区变更立即回退，保证“分析不破坏仓库状态”。
- 支持断点续扫：基于 candidates.jsonl、clusters.jsonl 和 analysis.jsonl 三个核心配置文件恢复状态，跳过已处理批次。

7. 配置与参数说明（概览）

CLI 子命令主要参数（源码为准）

- `config`
  - 用于管理安全分析配置
- `analyze`
  - –path/-p：待分析的根目录（必选）
  - –llm-group/-g：使用的模型组（可选，仅对本次运行生效）
  - –output/-o：最终 Markdown 报告输出路径（默认 `./report.md`）
  - –cluster-limit/-c：聚类每批最多处理的告警数（默认 50）
- `scan`
  - –path/-p：待扫描的根目录（必选）
  - –output/-o：扫描报告输出路径（默认 `./report.md`）

工作流参数

- languages：限定扫描的语言扩展（默认 [“c”, “cpp”, “h”, “hpp”, “rs”]）
- exclude_dirs：排除目录列表（默认 [“.git”, “build”, “out”, “target”, “third_party”, “vendor”]）
- report_file：增量报告文件路径（可选，默认 .jarvis/sec/agent_issues.jsonl）
- cluster_limit：聚类每批最多处理的告警数（默认 50）

8. 可靠性与容错设计

- 产物稳定与断点续扫：直扫与聚类/验证结果以 JSONL 持久化；通过 candidates.jsonl、clusters.jsonl 和 analysis.jsonl 三个核心配置文件支持断点续扫，便于中断后继续。
- 构建安全与回退：Agent 执行后如检测到文件被修改，则立即执行 git checkout – . 恢复，保证只读分析。
- 摘要解析容错：聚类与验证摘要严格限定在标记内（/），采用 JSON 解析（使用 json5，支持尾随逗号、注释等）并进行字段校验；失败时重试（最多 2 次）与告警。
- 工具白名单：Agent 仅能使用 read_code 与 execute_script，避免 rm/mv/cp/echo>、sed -i、git、patch、chmod/chown 等写操作。
- 目录与语言过滤：默认排除多类构建/第三方目录，降低误报与扫描耗时。
- 检查器容错：文件读取失败、正则匹配失败、解析异常等不影响主流程，返回空列表或跳过该文件。
- 断点续扫：通过 candidates.jsonl、clusters.jsonl 和 analysis.jsonl 三个核心配置文件自动推断恢复点，支持跳过已处理的候选、聚类和分析结果。

9. 扩展与二次开发建议

- 规则扩展：在 checkers 中扩展 C/C++ 与 Rust 规则库（缓冲区操作、整数溢出、跨语言 FFI 边界）与置信度模型。
- 聚类策略增强：优化“验证条件一致性”聚类提示词，使簇内条件更可执行；引入局部上下文读取策略以减少误聚。
- 验证提示词细化：根据类别生成专用验证模板（指针边界、长度/对齐、错误传播），提升确认质量。
- 报告聚合增强：在 report 中扩展评分维度与审阅视图（文件粒度统计、按验证条件聚合）。
- 断点与原子写：为快照与增量报告引入原子写与校验，提升长任务可靠性。
- 多语言支持：扩展检查器支持更多语言（Java、Python、Go 等）的安全问题检测。
- 置信度模型优化：基于历史验证结果调整规则置信度，减少误报率。
- 并行化：支持多文件/多批次的并行处理，提升大规模代码库的分析效率。

jarvis-c2rust 系统架构设计

本文档基于源码 `src/jarvis/jarvis_c2rust` 下的实现，对“C→Rust 迁移套件（jarvis-c2rust）”进行结构化架构说明，覆盖模块组成、模块关系、工作流程、以及各模块内部设计。面向本项目开发者和使用者。

参考风格：与本仓库现有架构文档一致，使用 PlantUML 以通俗术语呈现角色与流程，强调职责边界与可回退策略。

- **源码位置**：`src/jarvis/jarvis_c2rust/`
  - `__init__.py`：模块说明与导出（scanner、optimizer）
  - `cli.py`：Typer 命令行入口，提供 `config`（配置管理）、`run`（一键流水线，支持断点续跑）、`verify`（功能对齐验证）三个命令
  - `scanner.py`：libclang 驱动的 C/C++ 函数与类型扫描、引用图生成、转译顺序计算
  - `collector.py`：头文件函数名采集（使用 libclang 解析函数声明，由 config 命令内部调用）
  - `library_replacer.py`：基于 LLM 的库替代评估与剪枝（子树评估、断点续跑）
  - `llm_module_agent.py`：LLM 驱动的 Rust crate 结构规划与落盘（JSON 规划、模块声明、初始构建校验）
  - `transpiler.py`：函数级转译器与构建修复循环（模块规划、代码生成、构建校验、审查复核）
    - 相关支持模块：`constants.py`（常量定义）、`models.py`（数据模型）、`loaders.py`（数据加载器）、`utils.py`（工具函数）
  - `optimizer.py`：Rust 代码保守优化器（所有优化步骤均使用 CodeAgent 完成，包括 clippy 告警消除、unsafe 清

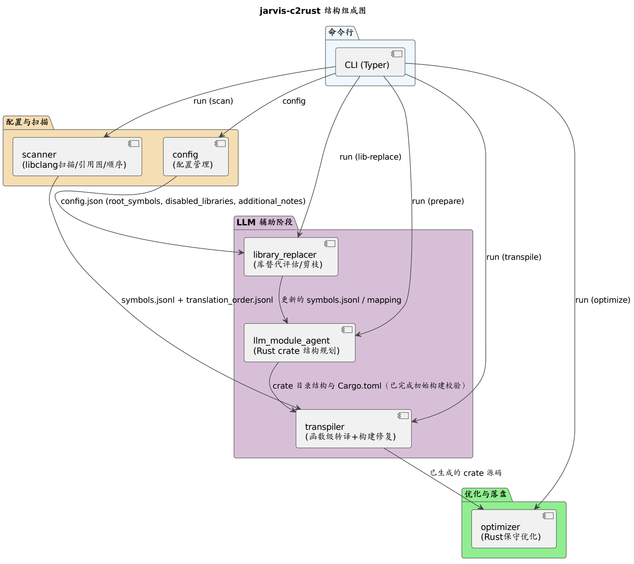
理、可见性优化、文档补充）
    - `verify.py`：功能对齐验证（检查转译是否完成、按文件分析 C 与 Rust 功能对齐性、支持迭代优化直至对齐或达到上限）
- **核心数据目录与产物**（默认路径 `<project_root>/.jarvis/c2rust/`）：
    - **符号表**：
        - `symbols.jsonl`：经过筛选/剪枝/替代后的主输入（统一符号表，包含函数与类型）
        - `symbols_raw.jsonl`：原始扫描数据（作为回退与比对依据）
        - `symbols_library_pruned.jsonl` / `symbols_prune.jsonl`：库替代阶段剪枝后的符号表（兼容别名）
    - **转译顺序**：
        - `translation_order.jsonl`：通用别名，由 `scanner.compute_translation_order_jsonl` 计算；按引用关系排序函数转译顺序
        - `translation_order_prune.jsonl`：库替代阶段基于剪枝表计算的顺序（兼容别名指向通用顺序）
    - **配置与状态**：
        - `config.json`：配置文件（root_symbols、disabled_libraries、additional_notes、enable_ffi_export_validation），由 config 命令管理
        - `run_state.json`：流水线状态文件（记录各阶段完成状态与时间戳），支持断点续跑
    - **库替代映射**：
        - `library_replacements.jsonl`：LLM 评估生成的库替代与剪枝结果（每行一个 JSON 对象）
        - `library_replacer_checkpoint.json`：库替代阶段的断点续跑记录（eval_counter/processed_roots/pruned_dynamic/selected_roots/timestamp/key）
    - **进度与映射**：
        - `progress.json`：转译或优化阶段的断点续跑记录（current/converted/metrics/converted_commits），每个已转换函数完成后记录对应的 commit id
        - `symbol_map.jsonl`：符号名映射（C→Rust，每行一条记录，支持同名/重载；兼容旧版 `symbol_map.json`）
        - `optimize_progress.json`：优化阶段的进度记录（processed 文件列表、steps_completed 步骤列表、step_commits 每个步骤的 commit id、fix_progress 每个修复的 commit id、last_commit 最后一步的 commit id）
        - `optimize_report.json`：优化阶段的统计报告（files_scanned/unsafe_removed/visibility_downgraded/docs_added 等）

1. 设计目标与总体思路

- **渐进式迁移流水线**：从“扫描数据基线”出发，逐步执行“库替代评估 → 模块规划 → 转译 → 优化”的流水线。每个阶段生成明确的中间产物，支持独立执行与调试。
- **明确产物与断点续跑**：每阶段产物写入 `.jarvis/c2rust`，形成可复现与可继续的中间结果；各阶段完成后自动记录状态到 `run_state.json`，支持流水线断点续跑；转译与优化阶段支持细粒度 `resume`（通过 `progress.json` 和 `optimize_progress.json` 记录进度，包括每个函数/步骤的 commit id，恢复时自动 reset 到对应的 commit id 确保代码状态一致）。
- **前置依赖检查**：各流程执行前自动检查前置依赖是否完成，防止跳过必要步骤；支持交互模式允许用户选择是否继续。
- **保守与可回退**：转译与优化过程具备构建检测与回退策略（如 `git_guard` 自动快照与回滚），尽可能保证最终 crate 可构建。所有修改均伴随 `cargo check` / `cargo test` 验证，失败立即回滚。
- **模块化与可替换**：扫描、评估、规划、转译、优化模块边界清晰，可独立使用或替换。数据格式统一为 JSONL，便于跨模块复用。
- **LLM 辅助决策**：在库替代评估、模块规划、代码生成、审查等关键环节使用 LLM 进行智能决策，支持指定模型组（`--llm-group`）以平衡成本与质量。
- **libclang 版本要求**：支持 libclang 16-21（含），通过环境变量 `CLANG_LIBRARY_FILE`、`LIBCLANG_PATH`、`LLVM_HOME` 配置库路径。

2. 模块组成（PlantUML）

下图从“命令行与流水线”视角呈现模块静态组成与依赖关系。



要点

- CLI 提供 `run` 命令执行完整流水线，支持断点续跑（根据 `run_state.json` 自动跳过已完成的阶段）。
- 数据产物在各阶段生成并复用，形成稳定的“迁移上下文”。

3. 核心产物与文件约定

- 统一符号表（JSONL）
    - symbols.jsonl：经过筛选/剪枝/替代后的主输入
    - symbols_raw.jsonl：原始扫描数据（作为回退与比对依据）
- 转译顺序
    - translation_order.jsonl：由 scanner.compute_translation_order_jsonl 计算；按引用关系排序函数转译顺序
- 配置
    - config.json：根函数名列表（root_symbols）、禁用库列表（disabled_libraries）、附加说明（additional_notes）、FFI 导出验证开关（enable_ffi_export_validation），由 config 命令管理
- 库替代映射
    - library_replacements.jsonl：LLM 评估生成的库替代与剪枝结果
- 进度与映射
    - progress.json：转译或优化阶段的断点续跑记录（current/converted/metrics/converted_commits），每个已转换函数完成后记录对应的 commit id，恢复时自动 reset 到对应的 commit id
    - symbol_map.jsonl：符号名映射（转译过程中的辅助）

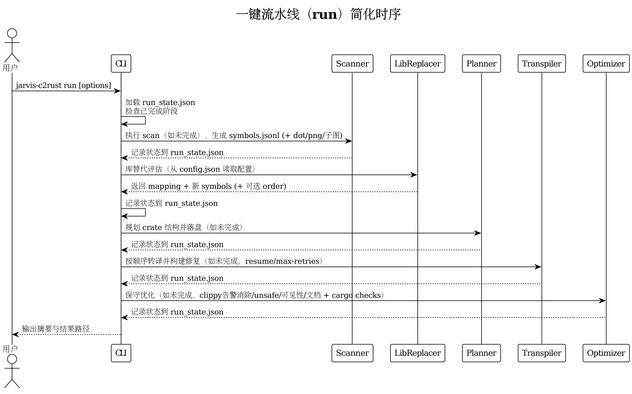
4. 命令与工作流程

命令行子命令（CLI）

- **config**：管理配置文件（`.jarvis/c2rust/config.json`），设置根符号列表、禁用库列表、附加说明和 FFI 导出验证
    - 参数：`--files <paths...>`（头文件或函数名列表文件，头文件自动提取函数名），`--root-list-syms <str>`（根符号列表内联，逗号分隔），`--disabled-libs <str>`（禁用库列表，逗号分隔），`--additional-notes <str>`（附加说明，将在所有 agent 的提示词中追加），`--enable-ffi-export-validation` / `--disable-ffi-export-validation`（启用/禁用 FFI 导出验证，默认禁用），`--show`（显示当前配置），`--clear`（清空配置）
    - 功能：从头文件（.h/.hh/.hpp/.hxx）自动提取函数名，或从文本文件逐行读取函数名；支持同时设置多个参数
- **run**：一键流水线（scan → lib-replace → prepare → transpile → optimize），支持断点续跑
    - 参数：`-g/--llm-group <name>`（LLM 相关阶段使用的模型组），`-m/--max-retries <N>`（transpile 构建/修复最大重试次数，0 表示不限制），`--interactive`（启用交互模式，默认非交互模式），`--reset`（重置状态，从头开始执行所有阶段）
    - 配置来源：从 `config.json` 读取 `root_symbols`、`disabled_libraries` 和 `additional_notes`（使用 `config` 命令预先设置）
    - 断点续跑：根据 `run_state.json` 自动跳过已完成的阶段，支持从任意阶段继续；各阶段完成后自动记录状态到 `run_state.json`
    - 阶段说明：
        - **scan**：执行 C/C++ 扫描，输出 `symbols.jsonl` / DOT 图（可生成 PNG）、子图等
        - **lib-replace**：对指定根列表进行库替代评估与剪枝（LLM-only 子树评估），从 `config.json` 读取配置；若未找到符号数据，自动执行扫描以生成数据
        - **prepare**：调用 LLM 规划 crate 结构并直接落盘，执行初始构建校验（`cargo check`）并进行最小修复
        - **transpile**：按 `translation_order.jsonl` 逐函数转译并进行构建修复，默认始终启用断点续跑（通过 `progress.json` 记录细粒度进度）
        - **optimize**：对生成的 crate 执行保守优化（所有优化步骤均使用 CodeAgent 完成，包括 clippy 告警消除、unsafe 清理、可见性优化、文档补充），采用默认优化配置，自动检测 crate 根目录；每个修复前执行 `cargo fmt` 格式化代码，每个 Agent 调用后执行 `cargo test` 验证，测试失败时自动回退到运行前的 commit
- **verify**：功能对齐验证（独立子命令，需在 `run` 完成后执行）
    - 参数：`-g/--llm-group <name>`（用于分析与优化的模型组），`-i/--max-iterations <N>`（最大迭代次数，默认 10），`--interactive`（启用交互模式）
    - 前置条件：要求 `run_state.json` 中 transpile 与 optimize 阶段均已完成，否则提示先执行 `jarvis-c2rust run`
    - 工作流程：检查转译状态 → 切换到目标 crate 目录 → 创建功能对齐分析任务（按文件拆分子任务）→ 使用 Agent 分析 C 与 Rust 代码的功能对齐性（函数签名、逻辑、错误处理、内存安全、数据结构布局等）→ 生成结构化报告（如 `alignment_report.md`）→ 若判定未对齐则调用 CodeAgent 基于报告优化代码并重新验证，直到对齐或达到最大迭代次数
    - 产物：crate 根目录下的 `alignment_report.md`（对齐分析报告）；验证结果以 JSON 形式在最终总结中输出（含 `is_aligned`、`summary` 等）

- 源码位置：`verify.py`（`check_transpile_completed`、`run_verify`、`_run_alignment_analysis`、`_run_optimization`）

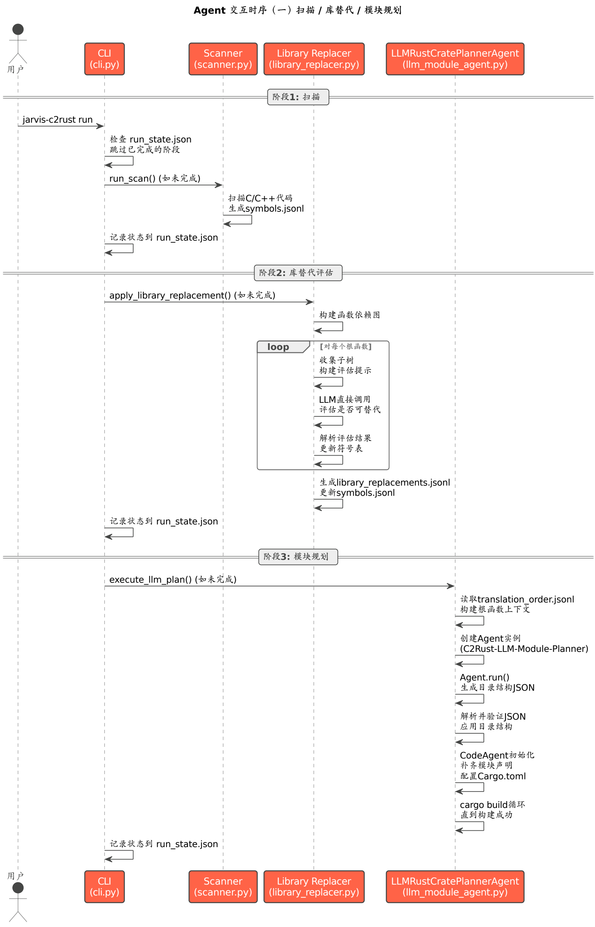
典型流水线（PlantUML）



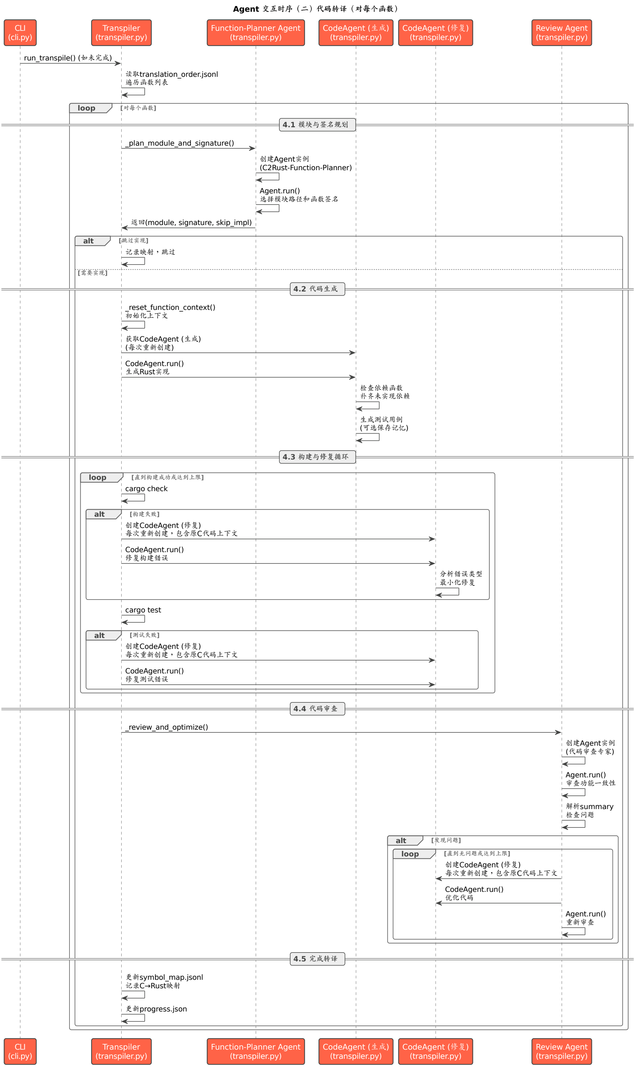
Agent 交互时序图：

以下三幅时序图分阶段展示 jarvis-c2rust 中 CLI、Scanner、Library Replacer、模块规划 Agent、Transpiler、CodeAgent（生成/修复）、Review Agent、Optimizer 及各类优化用 CodeAgent 的交互流程。

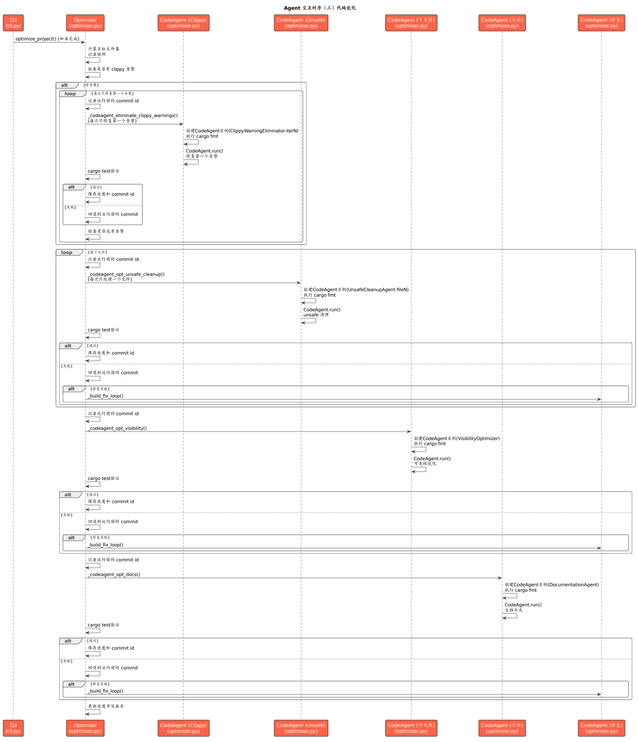
（一）扫描、库替代评估与模块规划



（二）代码转译（单函数流程）



（三）代码优化



时序图说明：

- **图（一）**：阶段1 扫描 → 阶段2 库替代评估 → 阶段3 模块规划。Scanner 扫描 C/C++ 生成符号表；Library Replacer 用 LLM 直接调用（非 Agent）评估子树是否可库替代；LLMRustCratePlannerAgent 规划 crate 目录结构并落盘，CodeAgent 初始化补齐模块与 Cargo.toml。
- **图（二）**：阶段4 代码转译（对每个函数）。Function-Planner Agent 选择模块与签名；CodeAgent (生成) 每次调用重新创建，生成 Rust 实现并补齐依赖与测试；CodeAgent (修复) 每次重新创建、带原 C 上下文，用于构建/测试修复；Review Agent 审查一致性，有问题时用 CodeAgent (修复) 迭代优化。
- **图（三）**：阶段5 代码优化。先按需用 CodeAgent 消除 clippy 告警（每次只修第一个，迭代）；再按文件用 CodeAgent 做 unsafe 清理、可见性优化、文档补充；每步前 `cargo fmt`，每步后 `cargo test`，失败则回退；修复失败时走 `_build_fix_loop()`。

Agent 复用策略：

- **代码生成 Agent**：每个函数使用独立的 CodeAgent（每次调用 get_generation_agent 重新创建），用于代码生成任务。
- **修复 Agent**：每次修复时重新创建 CodeAgent 实例，不复用，确保每次修复都有独立的上下文和状态。
- **修复 Agent 上下文增强**：修复 Agent 的上下文中自动包含原 C 实现代码，帮助 Agent 更好地理解原始实现意图，提高修复准确性。
- **上下文传递**：通过 `_compose_prompt_with_context()` 传递函数上下文；修复 Agent 调用时使用 `for_fix=True` 参数，自动添加原 C 代码上下文。
- **记忆功能**：代码生成 Agent 当前实现中未启用 `force_save_memory`；完成函数实现后可根据需要记录关键信息供后续修复与优化使用。

5. 模块内部设计

5.1 config（配置管理）

职责

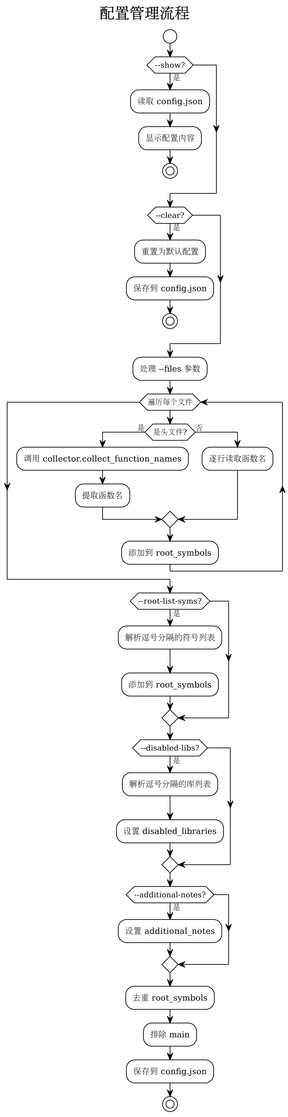
- 管理配置文件（`.jarvis/c2rust/config.json`），设置根符号列表（`root_symbols`）、禁用库列表（`disabled_libraries`）和附加说明（`additional_notes`）。
- 支持从头文件（.h/.hh/.hpp/.hxx）自动提取函数名，或从文本文件逐行读取函数名。
- 内部调用 `collector.collect_function_names` 处理头文件，确保与扫描器行为一致。
- 附加说明（`additional_notes`）将在所有 LLM Agent 的提示词中自动追加，用于传递用户自定义的要求和约束。

关键接口

- `config` 命令：管理配置文件
  - `--files <paths...>`：头文件或函数名列表文件
  - `--root-list-syms <str>`：根符号列表内联（逗号分隔）

- `--disabled-libs <str>`：禁用库列表（逗号分隔）
- `--additional-notes <str>`：附加说明（将在所有 agent 的提示词中追加）
- `--show`：显示当前配置
- `--clear`：清空配置

流程



技术细节

- **libclang 版本要求**：支持 libclang 16-21，复用 scanner 的版本检测与加载逻辑
- **编译参数处理**：优先使用 `compile_commands.json`，回退为最小参数（`-I <file.parent>` + 语言标志）
- **函数名提取策略**：优先使用 `qualified_name`（限定名），若不存在则使用 `name`（简单名）
- **容错机制**：解析失败的文件记录告警但不阻断其他文件；多次尝试（scan_file → 声明收集）以确保最大成功率
- **附加说明（additional_notes）**：
  - 配置格式：`config.json` 中的 `additional_notes` 字段为字符串类型，存储用户自定义的附加说明
  - 自动应用：所有与 LLM 对话的模块（transpiler、library_replacer、llm_module_agent、optimizer）都会自动读取并追加到提示词末尾
  - 追加格式：以 `【附加说明（用户自定义）】` 标题追加，确保 LLM 能够识别并遵循用户要求
  - 使用场景：可用于传递项目特定的编码规范、错误处理要求、类型设计约束等自定义要求

边界

- 非头文件后缀将被跳过（仅支持 .h/.hh/.hpp/.hxx）
- 解析失败记录告警但不阻断其他文件处理
- 输出文件使用 UTF-8 编码，每行一个函数名（支持 `#` 开头的注释行，但本模块不输出注释）

5.2 scanner（扫描与引用图）

职责

- 使用 libclang（版本 16-21）扫描 C/C++ 文件，收集函数信息（名称、参数、调用关系、语言等）与类型信息。
- 生成统一符号表（`symbols.jsonl` 和 `symbols_raw.jsonl`）、引用图 DOT/PNG、根函数识别与转译顺序计算。
- 提供 `run_scan` 一体化入口，支持 DOT 生成、子图导出等功能。

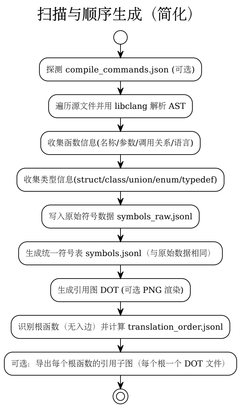
关键接口（源码参考）

- `_try_import_libclang()`：libclang 导入与版本验证（支持 16-21，通过环境变量配置路径）
- `find_compile_commands` / `load_compile_commands`：编译参数发现与加载
- `iter_source_files` / `scan_file` / `scan_directory`：文件遍历与扫描
- `generate_dot_from_db`：生成引用关系 DOT 文件（可渲染 PNG）
- `find_root_function_ids`：识别根函数（引用图中的无入边或策略定义的入口）
- `compute_translation_order_jsonl`：根据引用关系生成函数转译顺序（新格式：每行包含 `ids` 和 `items`）
- `export_root_subgraphs_to_dir`：导出每个根函数的引用子图

数据格式（symbols.jsonl） 每条记录（JSONL，每行一个 JSON 对象）包含：

- `id`（int）：符号唯一标识
- `category`（str）：“function” | “type”
- `name`（str）：简单名称
- `qualified_name`（str）：限定名称（如 `ns1::ns2::Class::method`）
- `signature`（str）：函数签名（类型可空）
- `return_type`（str）：函数返回类型（类型可空）
- `params`（list[dict]）：函数参数列表 `[{name, type}]`（类型可空）
- `kind`（str）：类型种类（struct/class/union/enum/typedef/type_alias，仅类型）
- `underlying_type`（str）：typedef/type_alias 的底层类型（仅类型）
- `ref`（list[str]）：统一的引用列表（被调用的函数或引用的类型）
- `file`（str）：源文件路径
- `start_line` / `start_col` / `end_line` / `end_col`（int）：位置信息
- `language`（str）：源语言（C/C++）
- `created_at` / `updated_at`（str）：时间戳

流程（PlantUML）



边界与容错

- **libclang 版本要求**：支持 libclang 16-21（含），通过环境变量 `CLANG_LIBRARY_FILE` 、 `LIBCLANG_PATH` 、 `LLVM_HOME` 配置库路径；版本不匹配时提供明确的修复建议
- **编译参数处理**：若 `compile_commands.json` 不存在，回退为基本解析参数；对复杂宏/编译选项场景保持告警
- **DOT 生成与 PNG 渲染**：为可选功能；PNG 渲染默认启用（需系统安装 Graphviz）
- **转译顺序计算**：基于引用关系构建依赖图，按拓扑排序生成顺序；若存在循环依赖，采用广度优先近似策略
- **数据格式兼容性**：仅支持新格式（每行包含 `ids` 和 `items` ），旧格式已移除支持

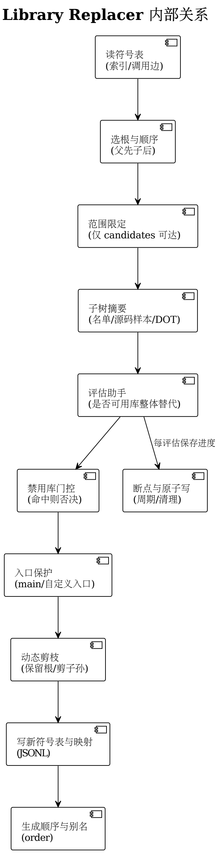
5.3 library_replacer（库替代评估）

系统核心：在扫描得到的函数依赖图上，按“根函数子树”进行评估，若整个子树可由一个或多个成熟 Rust 库整体替代，则以库占位替代该根的引用并剪除其子孙函数；保留类型记录不受影响。支持禁用库约束与断点续跑。

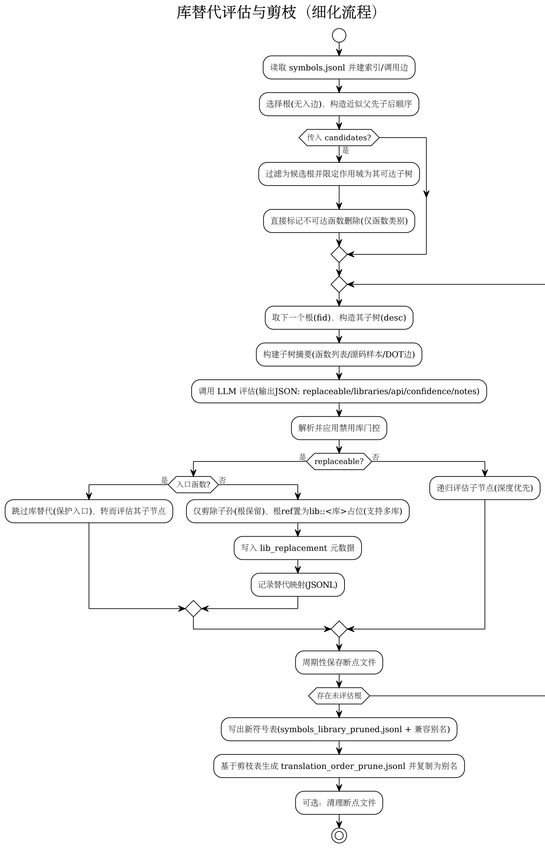
职责（精细拆解）

- 数据读取与图构建：
  - 读取 symbols.jsonl（统一符号表）；建立 id→记录、名称/限定名→id 索引
  - 仅针对函数类别构建调用边（id→id），形成邻接表
  - 识别根函数（无入边）；若无根，则回退为全量函数集合
- 根选择与评估顺序：
  - 默认从所有根函数开始，采用近似“父先子后”的顺序（广度遍历）
  - 支持 candidates（名称或限定名列表）：仅评估这些根，并限定作用域为其可达子树；不可达函数将直接删除（类型保留）
- 子树摘要与样本构建：
  - 对待评估根，收集其子树节点摘要（名称 | 签名），限制总条数
  - 提取部分代表性节点源码片段（根及最多两个直接子节点），限制行数
  - 构建子树边列表（caller -> callee）；边少时生成 DOT 文本辅助判断
- LLM 评估与解析：
  - 使用 PlatformRegistry 的普通平台与模型组（可选 llm_group），统一 system_prompt
  - 输出要求为 块内的 JSON（支持 JSON5 语法），或容忍 JSON 代码块；解析字段：replaceable、libraries、library（主库）、apis/api、confidence、notes
  - 评估约束条件：
    - 禁用库约束：若建议命中 disabled_libraries（大小写归一），强制判定不可替代，并在备注中追加告警
    - 直接使用约束：如果当前调用的函数无法使用 crate 直接提供的功能而需要封装或者改造，则认为不可替代（确保仅使用成熟库的标准 API，避免自定义封装层）
- 入口函数保护：
  - 默认跳过 main，不进行库替代（改为深入评估其子节点）
  - 支持通过环境变量配置多个入口名：JARVIS_C2RUST_DELAY_ENTRY_SYMBOLS / JARVIS_C2RUST_DELAY_ENTRIES / C2RUST_DELAY_ENTRIES（逗号/空白/分号分隔）
- 剪枝与映射生成：
  - 可替代时：仅剪除子孙函数（根保留），将根的 ref 设置为库占位（lib::，支持多库），并写入 lib_replacement 元数据（libraries/library/apis/api/confidence/notes/mode/updated_at）
  - 不可替代时：递归评估其子节点
  - 选中替代根的概要按 JSONL 写出到 library_replacements.jsonl
- 输出文件与别名：
  - 新符号表：symbols_library_pruned.jsonl（兼容输出 symbols_prune.jsonl），保留别名 symbols.jsonl 指向新表以统一后续流程
  - 转译顺序：translation_order_prune.jsonl（基于剪枝表计算），并复制为通用别名 translation_order.jsonl
- 断点续跑与原子写：
  - 断点文件：library_replacer_checkpoint.json（eval_counter/processed_roots/pruned_dynamic/selected_roots/timestamp/key）
  - 匹配关键键（symbols + library_name + llm_group + candidates + disabled_libraries + max_funcs）才恢复
  - 定期保存（checkpoint_interval），完成后可选清理（clear_checkpoint_on_done）；落盘使用原子写避免损坏
- 限流与稳健性：
  - max_funcs 用于限制评估根数量（调试/限流）
  - LLM 不可用或失败时，视为不可替代并继续评估子节点；解析失败容忍并警告

内部关系



评估与剪枝流程（细化）



参数与行为要点（与 CLI lib-replace 对应）

- db_path：符号表路径或其所在目录（自动解析至 .jarvis/c2rust/symbols.jsonl）
- library_name：指定库名（如 std/regex），评估时优先该库；解析结果支持多库组合
- llm_group：指定评估模型组；平台不可用时回退为“不可替代”
- candidates：仅评估这些根；作用域限制为其可达子树，不可达函数直接删除（类型保留）
- disabled_libraries：禁用库名列表（大小写不敏感），命中则强制不可替代
- resume / checkpoint_interval / clear_checkpoint_on_done：断点续跑相关配置
- max_funcs：评估根数量上限（限流/调试）
- out_symbols_path / out_mapping_path：输出路径（默认落在 .jarvis/c2rust）
- **additional_notes**：从 `config.json` 读取附加说明，自动追加到 LLM 评估提示词末尾，用于传递用户自定义的要求和约束

边界与策略

- 类型记录：任何剪枝不影响类型记录；仅函数类别参与剪枝
- 库占位：以 lib:: 表示库引用，占位符在后续转译与优化阶段作为上下文提示
- 入口保护：main 默认保护（可配置），确保后续转译仍能生成入口实现
- 解析容错：支持 JSON 格式（支持 JSON5 语法）；解析失败则不可替代并继续
- 原子写与报告：断点/输出文件使用原子写；在完成时输出统计摘要（选中替代根数/剪除函数数/新符号表与顺序路径）

5.4 llm_module_agent（crate 规划与落盘）

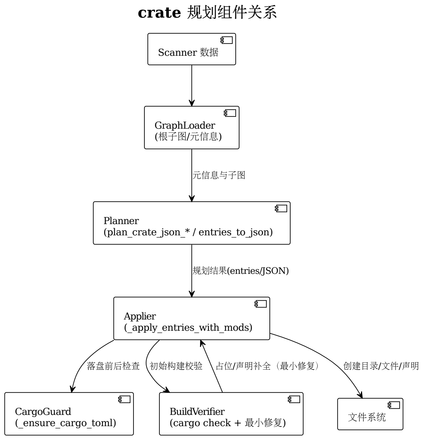
职责

- 以 LLM 为核心，基于引用子图与实体元信息规划 crate 模块结构（JSON），并将结构应用到磁盘。
- 确保 Cargo.toml 存在，模块声明完整（pub mod），并创建必要的目录与文件占位。
- 在创建初始工程结构后，执行构建校验（cargo check）并进行必要的最小修复/占位补全，确保初始工程可构建通过。
- **附加说明支持**：从 `config.json` 读取 `additional_notes`，自动追加到所有 LLM 提示词（系统提示、用户提示、总结提示）末尾，用于传递用户自定义的要求和约束

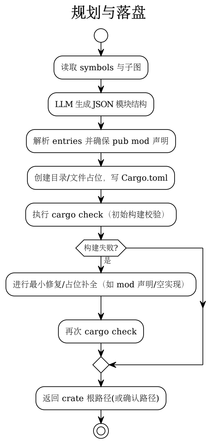
关键类型与函数（源码参考）

- _FnMeta / _GraphLoader：函数元信息与子图加载
- plan_crate_json_text / plan_crate_json_llm：生成 JSON 结构文本
- entries_to_json / _parse_project_json_entries*：JSON 与内部结构互转
- _ensure_pub_mod_declarations / _apply_entries_with_mods：确保模块声明与写盘
- _resolve_created_dir / _ensure_cargo_toml：目录定位与 Cargo.toml 保障
- execute_llm_plan(apply=True, llm_group=…)

组件关系（PlantUML）



流程



边界与容错

- JSON 解析失败时尝试 fallback 解析；声明缺失时进行自动补全。
- 写盘过程中异常提供告警并尽量保持幂等。

5.5 transpiler（转译与构建修复）

Git 变更信息管理（GitManager）：

- **职责**：管理转译过程中的 Git 操作，包括获取 git diff、记录 commit id、回退到指定 commit 等
- **核心方法**：
  - `get_git_diff(base_commit: Optional[str]) -> str`：获取从基准 commit 到当前工作区的 git diff
    - 如果 `base_commit` 为 `None`，使用 HEAD 作为基准
    - 如果 `base_commit` 不存在，回退到 HEAD
    - 如果仓库为空（无 HEAD），使用 `git diff` 获取工作区差异
    - 自动暂存新增文件（`git add -N`）以便获取完整的 diff，完成后重置暂存区
    - 返回 UTF-8 编码的 diff 内容，失败时返回空字符串
  - `get_crate_commit_hash() -> Optional[str]`：获取 crate 目录的当前 commit id
  - `reset_to_commit(commit_hash: str) -> bool`：回退到指定的 commit（硬重置 + 清理未跟踪文件）
- **Git Diff 长度限制工具**（`utils.truncate_git_diff_with_context_limit`）：
  - 功能：限制 git diff 的长度，避免上下文过大导致超出 LLM token 限制
  - 策略：
    - 优先使用 Agent 的剩余 token 数量计算可用空间（更准确）
    - 回退到 LLM 模型的输入窗口限制（通过 `llm_group` 参数获取）
    - 最后回退到默认限制（10000 字符）
  - Token 比例配置：
    - 构建修复阶段：30%（`token_ratio=0.3`），因为修复提示词本身已经很长
    - 审查阶段：50%（`token_ratio=0.5`），因为审查需要更多的上下文信息
  - 实现：如果 git diff 超出限制，截断并添加提示信息

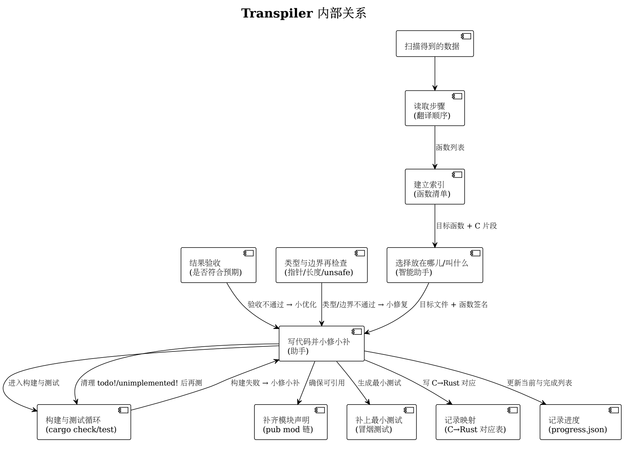
系统核心：Transpiler 负责按扫描顺序逐函数完成“模块与签名规划 → 代码生成 → 构建校验与最小修复 → 审查与类型边界复核 → 映射记录与占位清理”的完整闭环。

职责（精细拆解）

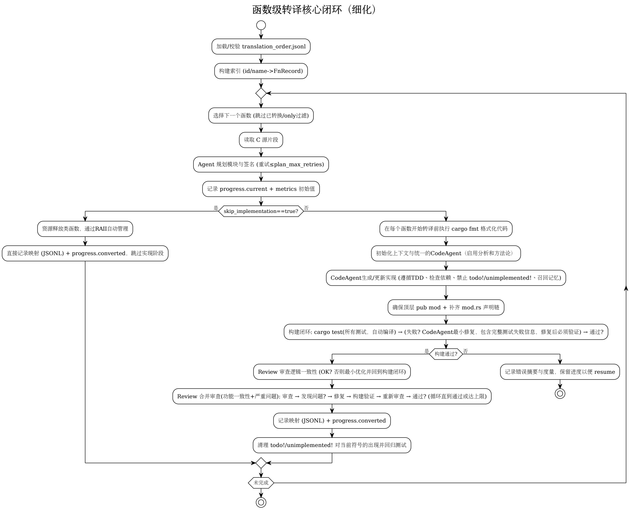
- 顺序与索引：
  - 确保并加载 translation_order.jsonl（_ensure_order_file/_iter_order_steps）
  - 构建自包含索引（id → FnRecord、name/qname → id）（_load_order_index）
- 模块与签名规划（Agent）：
  - 基于 C 源片段、调用者上下文（已转译/未转译）、crate 目录树，调用 Agent 选择目标模块文件与 Rust 函数签名（_plan_module_and_signature）
  - 对规划结果进行基本格式检查（字段存在性）与至多 plan_max_retries 次重试，失败回退至兜底模块与占位签名
  - **附加说明支持**：从 `config.json` 读取 `additional_notes`，自动追加到所有 Agent 提示词末尾（以 `【附加说明（用户自定义）】` 标题），用于传递用户自定义的要求和约束
  - **资源释放类函数跳过实现逻辑**：
    - 识别标准：在规划阶段，Agent 会识别资源释放类函数（如文件关闭 fclose、内存释放 free、句柄释放、锁释放 mutex_unlock 等），这些函数在 Rust 中通常通过 RAII（Resource Acquisition Is Initialization，Drop trait）自动管理资源
    - 跳过标记：如果函数被识别为资源释放类，Agent 会在规划结果的 JSON 中设置 `skip_implementation: true` 字段，并在 `notes` 字段中说明原因（如“通过 RAII 自动管理，无需显式实现”）
    - 跳过处理：当 `skip_implementation` 为 `true` 时，转译流程会：
      - 记录模块路径和函数签名到 progress.json 和 symbol_map.jsonl
      - 直接标记函数为已转换（添加到 progress['converted']），跳过代码生成、构建验证和审查阶段
      - 在 symbol_map.jsonl 中记录映射关系，但标注为“跳过实现”
    - 实现策略提示：在代码生成阶段，如果函数未被跳过但属于资源释放类，提示中会说明：
      - 如果函数签名需要保留（用于兼容性），可以提供空实现或仅返回成功状态
      - 在函数文档注释中明确说明：“此函数在 Rust 中通过 RAII（Drop trait）自动管理资源，无需显式调用。保留此函数仅用于 API 兼容性。”
      - 对于需要返回值的函数（如错误码），可以返回成功状态（如 `Ok(())` 或 `0`）
      - 测试可以非常简单（如仅验证函数可以调用而不崩溃），或可以跳过测试（在文档注释中说明原因）
- 进度记录：
  - 更新 progress.json 当前项（id/name/qname/file/位置/module/rust_signature/signature_hint/metrics）（_update_progress_current）
  - 支持 resume：跳过已完成函数（symbol_map.has_rec；progress['converted']）
- 上下文与复用：
  - 构建当前函数的上下文头部（全量/精简），在复用代码生成Agent/Review等Agent时拼接（_reset_function_context/_compose_prompt_with_context/_refresh_compact_context）
  - 保存当前函数的 C 代码，供修复 Agent 使用（_current_c_code）
- 代码生成 Agent（CodeAgent，可复用）：
  - 在单个函数生命周期内创建并复用同一个CodeAgent实例，用于代码生成任务（_get_generation_agent）
  - CodeAgent配置：启用方法论和分析功能（use_methodology=True, use_analysis=True），附加工具包括 read_symbols 和 methodology，用于提供更好的代码生成能力
- 修复 Agent（CodeAgent，每次重新创建）：
  - 每次修复时重新创建CodeAgent实例，不复用，确保每次修复都有独立的上下文和状态（_get_fix_agent）
  - CodeAgent配置：启用方法论和分析功能（use_methodology=True, use_analysis=True），附加工具包括 read_symbols 和 methodology，用于提供更好的修复能力
  - 上下文增强：修复 Agent 的上下文中自动包含原 C 实现代码，通过 `compose_prompt_with_context(prompt, for_fix=True)` 自动添加
  - 代码生成遵循TDD（测试驱动开发）方法：通过提示词指导Agent先写测试再写实现，确保生成的代码可测试且质量更高
  - 代码生成：在目标模块生成或更新实现，遵循“最小变更、禁止 todo!/unimplemented!、必要时补齐依赖实现”的约束（_codeagent_generate_impl）
  - **附加说明支持**：所有代码生成和修复提示词都会自动追加 `additional_notes`（从 `config.json` 读取），确保用户自定义要求在所有 Agent 对话中生效
- 记忆功能：代码生成 Agent 默认不强制保存记忆（force_save_memory=False），但提示词中会建议Agent在需要时使用 save_memory 工具记录关键信息；修复 Agent 每次重新创建，不依赖记忆功能
  - 记忆召回提示：在代码生成和修复阶段，强烈建议使用 retrieve_memory 工具召回已保存的函数实现记忆，这些记忆可能包含之前已实现的类似函数、设计决策、实现模式等有价值的信息，可以显著提高实现和修复效率
  - 依赖检查与实现要求：在实现或修复函数时，要求检查当前函数及其所有依赖函数是否已实现，对于未实现的依赖函数，需在本次一并补齐等价的Rust实现，禁止使用 todo!/unimplemented! 占位
  - **代码格式化**：在每个函数开始转译前（读取C函数源码之后、规划模块与签名之前）自动执行 `cargo fmt` 格式化代码，确保代码风格一致（_run_cargo_fmt）
- 模块可见性与声明链：
  - 确保 src/lib.rs 顶层 pub mod，补齐从目标模块文件向上的 mod.rs 声明链（_ensure_top_level_pub_mod/_ensure_mod_rs_decl/_ensure_mod_chain_for_module）
- 构建校验与最小修复（核心闭环）：
  - 直接执行 cargo test（运行所有测试，不区分项目结构）：`cargo test` 会自动编译并运行所有类型的测试（lib tests、bin tests、integration tests、doc tests 等），无需单独执行 cargo check
  - **Cargo Test 编译警告检测**：
    - 在 `cargo test` 通过后，检查编译和测试过程中的警告输出
    - 如果发现警告（如 `unused_mut`、`unused_variables`、`dead_code`、`unused_import` 等），进入警告修复阶段
    - 警告修复提示词包含完整的警告信息，要求修复所有警告
    - 修复后必须确保 `cargo test -- --nocapture` 能够通过（返回码为 0 且无警告输出）
  - 测试失败时：错误分类（missing_import/type_mismatch/visibility/borrow_checker/dependency_missing/module_not_found），使用修复 Agent（每次重新创建，包含原 C 代码上下文）进行最小修复并继续循环（_classify_rust_error/_cargo_build_loop）
  - 测试命令配置：使用 `cargo test -- --nocapture` 获取完整的测试失败信息，并设置30秒超时（timeout=30）防止测试卡死
  - 统一测试验证：使用 `cargo test` 运行所有类型的测试（lib tests、bin tests、integration tests、doc tests 等），不区分项目结构（lib/bin/mixed），确保所有测试都能被验证
  - 测试失败信息反馈：测试失败时使用 `cargo test -- --nocapture` 获取完整的测试失败信息（包括测试用例名称、断言失败位置、期望值与实际值、堆栈跟踪等），并通过专门的 `<TEST_FAILURE>` 标签传递给修复 Agent，明确区分测试失败和编译错误
  - **修复范围要求**：
    - **重要**：不仅要修复当前问题，如果修复过程中导致其他测试用例失败，也必须一并修复
    - 修复后必须运行 `cargo test -- --nocapture` 验证所有测试用例都能通过
    - 如果发现修复后某些原本通过的测试用例现在失败了，说明修复引入了回归问题，必须一并修复
    - 必须确保修复后所有测试用例（包括目标函数的测试和其他函数的测试）都能通过
    - 如果修复影响了其他函数或模块，需要检查并修复所有受影响的部分
    - 不要只修复当前问题，而忽略其他测试的失败
  - **自动修复步骤**：
    - **编译警告修复**：优先使用 `cargo fix --allow-dirty --allow-staged` 自动修复可以自动修复的编译警告，然后验证修复效果
    - **Clippy 警告修复**：优先使用 `cargo clippy --fix --allow-dirty --allow-staged -- -D warnings` 自动修复可以自动修复的 clippy 警告，然后验证修复效果
    - 如果自动修复后仍有警告，继续手动修复剩余的警告
  - **并发问题排查指南**：
    - 如果测试运行失败，但在修复时没有修改任何代码，重新运行测试却成功了，这可能是并发问题导致的
    - 可能的原因：
      - 测试之间存在竞态条件（race condition）
      - 共享资源（文件、网络、数据库等）的并发访问问题
      - 测试执行顺序依赖问题
    - 排查建议：
      - 多次运行测试确认问题是否稳定复现
      - 检查测试之间是否存在共享状态或资源
      - 检查测试是否依赖特定的执行顺序
      - 考虑添加同步机制或隔离测试环境
      - 如果问题不稳定，可能需要添加重试机制或调整测试策略
  - 修复后验证：修复完成后，必须使用 `execute_script` 工具执行 `cargo test -- --nocapture` 验证修复效果，确保命令执行成功（返回码为0）才说明修复成功
  - **原 C 代码作为修复上下文**：
    - 在构建修复阶段，修复 Agent 的上下文中自动包含原 C 实现代码，帮助 Agent 更好地理解原始实现意图
    - 获取方式：通过 `_reset_function_context()` 保存当前函数的 C 代码（`_current_c_code`），在创建修复 Agent 时传入
    - 添加方式：通过 `compose_prompt_with_context(prompt, for_fix=True)` 自动在修复提示词中添加原 C 代码上下文
    - 格式：原 C 代码以 `<C_SOURCE>` 标签包裹，包含在修复提示词的“【原始 C 实现代码（修复参考）】”部分
    - 用途：帮助修复 Agent 理解原始 C 实现的逻辑和意图，提高修复的准确性和一致性
  - **Git 变更信息作为修复上下文**：
    - 在构建修复阶段，修复提示词会自动包含从函数开始处理到当前的 git 变更信息（git diff）

        - 基准 commit：使用函数开始处理时记录的 commit id（`_current_function_start_commit`）作为基准，显示该函数处理过程中的所有变更
        - 获取方式：通过 `GitManager.get_git_diff()` 获取从基准 commit 到当前工作区的差异（包括未提交的修改）
        - 长度限制：使用 `truncate_git_diff_with_context_limit()` 函数限制 git diff 的长度，避免上下文过大
            - 优先使用 Agent 的剩余 token 数量计算可用空间（更准确）
            - 回退到 LLM 模型的输入窗口限制
            - 修复提示词使用 30% 的 token 比例（`token_ratio=0.3`），因为修复提示词本身已经很长
        - 格式：git diff 以 `<GIT_DIFF>` 标签包裹，包含在修复提示词的 "【Git 变更信息】" 部分
        - 用途：帮助修复 Agent 了解当前代码状态和已做的修改，识别之前修改引入的问题，确认意外修改的必要性
        - 容错：如果获取 git diff 失败（如非 git 仓库、commit 不存在等），不影响主流程，只记录警告
    - 每轮记录 test_attempts、impl_verified、last_build_error 等度量
    - **测试代码删除检测（基于事件订阅）**：
        - 实现机制：通过订阅 `BEFORE_TOOL_CALL` 和 `AFTER_TOOL_CALL` 事件，在每次工具调用前后进行细粒度检测
        - 工具调用前记录：在 `BEFORE_TOOL_CALL` 事件处理器中记录当前 commit id，存储在 `_agent_before_commits` 字典中（以 Agent 实例 ID 为键）
        - 工具调用后检测：在 `AFTER_TOOL_CALL` 事件处理器中，仅对编辑文件的工具调用（`edit_file`）进行检测
        - 检测逻辑：使用 `detect_test_deletion` 函数检测 diff 中是否包含删除的 `#[test]` 或 `#[cfg(test)]` 标记
        - LLM 判断：如果检测到删除，使用 `ask_llm_about_test_deletion` 询问 LLM 判断删除是否合理（考虑测试代码是否被移动到其他位置、是否为重复测试等）
        - 自动回退：如果 LLM 认为删除不合理，立即回退到工具调用前的 commit，并在 Agent 的 session 中添加提示告知修改被撤销
        - 覆盖范围：所有创建的 Agent（CodeAgent、Review Agent、Planner Agent）都会自动订阅这两个事件，确保所有代码修改操作都受到保护
        - 优势：相比在 `agent.run()` 完成后检测，事件订阅机制能够更及时地发现问题并回退，避免问题累积
    - Commit ID 记录与失败回退机制：
        - 在处理每个函数前，记录 crate 目录的当前 commit id（`_get_crate_commit_hash`），存储在 `_current_function_start_commit` 中
        - 跟踪连续修复失败次数（`_consecutive_fix_failures`）：每次修复失败后递增，修复成功或测试通过时重置为 0
        - 当连续修复失败达到 10 次时，自动回退到函数开始时的 commit（`_reset_to_commit`）：
            - 使用 `git reset --hard` 硬重置到记录的 commit
            - 使用 `git clean -fd` 清理未跟踪的文件
            - 回退成功后，`_cargo_build_loop` 返回 `None` 表示需要重新开始
        - 重新开始机制：检测到需要重新开始时（`ok is None`），重新执行函数处理流程（生成实现 → 构建修复循环），最多重新开始 10 次
        - 重新开始时，重新记录 commit id 并重置连续失败计数，确保每次重新开始都有新的基准点
        - 若重新开始次数达到上限（10 次），停止处理该函数并保留当前状态，便于后续 resume
- 审查与复核：
    - Review Agent 合并审查功能一致性和严重问题（_review_and_optimize）：
        - 功能一致性检查：核心输入输出、主要功能逻辑是否与 C 实现一致；允许 Rust 实现修复 C 代码中的安全漏洞或使用不同的类型设计、错误处理方式、资源管理方式等，只要功能一致即可
        - 严重问题检查：明显的空指针解引用、明显的越界访问等可能导致功能错误的问题
        - 不检查类型匹配、指针可变性、边界检查细节、资源释放细节等技术细节（除非会导致功能错误）
    - **Git 变更信息作为审查上下文**：
        - 在代码审查阶段，审查提示词会自动包含从函数开始处理到当前的 git 变更信息（git diff）
        - 基准 commit：使用函数开始处理时记录的 commit id（`current_function_start_commit_getter()`）作为基准
        - 获取方式：通过 `GitManager.get_git_diff()` 获取从基准 commit 到当前工作区的差异（包括未提交的修改）
        - 长度限制：使用 `truncate_git_diff_with_context_limit()` 函数限制 git diff 的长度
            - 审查阶段使用 50% 的 token 比例（`token_ratio=0.5`），因为审查阶段需要更多的上下文信息
            - 优先使用 Agent 的剩余 token 数量，回退到 LLM 模型的输入窗口限制
        - 格式：git diff 包含在审查提示词中，帮助审查 Agent 了解代码变更历史
        - 状态跟踪：记录 diff 获取状态（success/no_change/error/no_start_commit/no_git_diff_func），便于调试和监控
        - 容错：如果获取 git diff 失败，不影响主流程，只记录警告
    - **审查修复阶段的上下文增强**：
        - 当审查发现问题需要修复时，使用修复 Agent（每次重新创建，包含原 C 代码上下文）进行优化
        - 修复提示词包含 git 变更信息：使用与构建修复阶段相同的机制和 token 比例（30%）
        - 修复提示词包含原 C 代码：通过 `compose_prompt_with_context(prompt, for_fix=True)` 自动添加原 C 实现代码
        - 帮助修复 Agent 结合审查问题、原 C 代码和 git 变更信息，有针对性地修复代码
    - 审查采用循环验证机制：发现问题 → 修复 → 构建验证 → 重新审查，最多迭代 review_max_iterations 次（0 表示无限重试）
    - **附加说明支持**：审查和优化阶段的提示词也会自动追加 `additional_notes`，确保用户自定义要求在整个审查流程中生效
- 映射与占位清理：
    - 记录 C→Rust 符号映射到 JSONL（支持同名/重载），更新 progress["converted"]（_mark_converted，_SymbolMapJsonl）
    - 清理 crate 源码中对当前符号的 todo!("sym")/unimplemented!("sym") 占位，替换为真实调用并回归测试（_resolve_pending_todos_for_symbol）
- 初始工程自洽：
    - 在未执行 prepare 的情况下，兜底确保最小 Cargo.toml 与 src/lib.rs 存在（transpile开头的初始化逻辑）

组件关系（Transpiler 子系统）



核心流程



边界与策略（与源码一致）

- 数据与顺序：
    - translation_order.jsonl 必须存在且包含有效步骤，若为空/无效则基于 symbols.jsonl 重新计算，一直校验通过为止
    - 仅支持新格式（每行含 ids/items），旧格式忽略

- 规划与生成：
  - 模块路径必须位于 crate/src/ 下，禁止指向 src/main.rs，文件后缀为 .rs 或 mod.rs
  - 签名需包含 fn 名称，函数接口设计应遵循 Rust 最佳实践，不需要兼容 C 的数据类型；禁止使用 `extern "C"`；优先使用 Rust 原生类型和惯用法；参数个数与顺序可以保持与 C 一致，但类型设计应优先考虑 Rust 的惯用法和安全性
- 构建闭环：
  - 直接执行 cargo test（运行所有测试，不区分项目结构）：`cargo test` 会自动编译并运行所有类型的测试，无需单独执行 cargo check
  - 测试命令配置：使用 `cargo test -- --nocapture` 获取完整的测试失败信息，并设置30秒超时（timeout=30）防止测试卡死
  - test 阶段最多迭代 test_max_retries 次（0 表示无限重试），check_max_retries 参数保留用于兼容性，但实际不再使用
  - 统一测试验证：使用 `cargo test` 运行所有类型的测试（lib tests、bin tests、integration tests、doc tests 等），不区分项目结构，确保所有测试都能被验证
  - 测试失败时使用 `cargo test -- --nocapture` 获取完整测试失败信息（测试用例名称、断言失败位置、期望值与实际值、堆栈跟踪等），通过 `<TEST_FAILURE>` 标签传递给CodeAgent，明确区分测试失败和编译错误
  - 修复后验证：修复完成后，必须使用 `execute_script` 工具执行 `cargo test -- --nocapture` 验证修复效果，确保命令执行成功（返回码为0）才说明修复成功
  - 每轮修复均保持最小改动：修正声明/依赖、检查并补齐未实现被调函数（包括依赖的依赖）、精确 use 导入、避免通配，必要时更新 Cargo.toml
  - 修复时要求检查当前函数及其所有依赖函数是否已实现，对于未实现的依赖函数需一并补齐等价的Rust实现
  - 记忆召回：修复时强烈建议使用 retrieve_memory 工具召回已保存的函数实现记忆，参考已有的实现模式、设计决策等
  - Commit ID 记录与失败回退机制：
    - 函数开始前记录 commit id：在处理每个函数前，调用 `_get_crate_commit_hash()` 获取并记录 crate 目录的当前 commit id，存储在 `_current_function_start_commit` 中
    - 连续失败跟踪：使用 `_consecutive_fix_failures` 计数器跟踪连续修复失败的次数，每次修复失败后递增，修复成功或测试通过时重置为 0
    - 自动回退触发：当连续修复失败达到 10 次时，自动触发回退机制：
      - 调用 `_reset_to_commit()` 回退到函数开始时的 commit
      - 使用 `git reset --hard <commit_hash>` 硬重置工作区
      - 使用 `git clean -fd` 清理未跟踪的文件和目录
      - 回退成功后，`_cargo_build_loop()` 返回 `None` 表示需要重新开始处理该函数
    - 重新开始处理：检测到需要重新开始时（`ok is None`），重新执行函数处理流程：
      - 重新记录 commit id（回退后的新基准点）
      - 重置连续失败计数
      - 重新生成实现并进入构建修复循环
      - 最多重新开始 10 次，超过后停止处理该函数并保留状态
    - 实现位置：
      - Commit ID 获取：`transpiler.py::_get_crate_commit_hash()`
      - 回退操作：`transpiler.py::_reset_to_commit()`
      - 失败跟踪：`transpiler.py::_cargo_build_loop()` 中的 `_consecutive_fix_failures` 计数
      - 重新开始逻辑：`transpiler.py::transpile()` 中的 `function_retry_count` 循环
- 审查与复核：
  - Review Agent 合并审查功能一致性和严重问题（`_review_and_optimize`）：
    - 功能一致性检查：核心输入输出、主要功能逻辑是否与 C 实现一致；允许 Rust 实现修复 C 代码中的安全漏洞或使用不同的类型设计、错误处理方式、资源管理方式等，只要功能一致即可
    - 严重问题检查：明显的空指针解引用、明显的越界访问等可能导致功能错误的问题
    - 不检查类型匹配、指针可变性、边界检查细节、资源释放细节等技术细节（除非会导致功能错误）
  - 审查采用循环验证机制：发现问题 → 修复 → 构建验证 → 重新审查，最多迭代 review_max_iterations 次（0 表示无限重试）
  - 审查失败不阻塞主流程，通过最小修复回到构建闭环，直到通过或达上限
- 进度与映射：
  - progress.json 持续更新 current/metrics 与 converted 集合，支持断点续跑
  - 每个函数转换完成后记录对应的 commit id 到 `converted_commits` 字段，恢复时自动 reset 到最后一个已转换函数的 commit id，确保代码状态一致
  - SymbolMap JSONL 每函数写入一条映射（带源位置），支持同名函数/重载区分
- 占位清理：
  - 针对 todo!(“symbol”)/unimplemented!(“symbol”) 的出现文件逐一最小修复为真实调用，并回归测试
- 兜底初始化：
  - 在未运行 prepare 时，兜底生成最小 Cargo.toml 与 src/lib.rs，以确保后续流程可运行

5.6 optimizer（保守优化器）

系统核心：对已生成的 Rust crate 进行“保守、可回退”的质量提升，所有优化步骤均使用 CodeAgent 完成。分步执行并在每步后用构建测试验证（本模块统一使用 cargo test 作为验证手段）。失败时尝试最小修复；如仍失败且开启 git_guard，则自动回滚到步骤前快照。

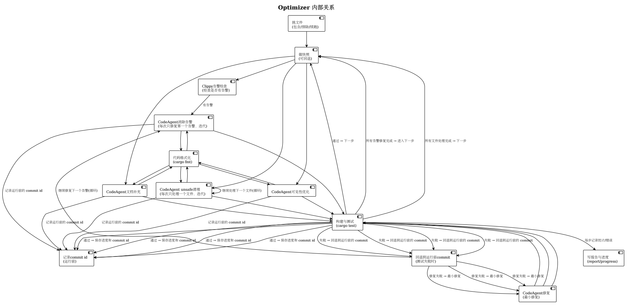
职责（精细拆解）

- 文件选择与进度管理：
  - 依据 include/exclude、max_files、resume 从 crate 中挑选本批次要处理的 .rs 文件
  - 使用 .jarvis/c2rust/optimize_progress.json 记录 processed 列表、steps_completed 步骤列表、step_commits 每个步骤的 commit id，支持断点续跑与重置
  - 每个步骤完成后自动记录当前 commit id，恢复时自动 reset 到最后一个已完成步骤的 commit id，确保代码状态一致
- 快照与回滚（git_guard）：
  - 每个优化步骤前记录当前 HEAD 快照；步骤后若构建仍失败且无法修复，自动 reset –hard 回快照
- 1. Clippy 告警消除（CodeAgent，可选）：
  - 检查是否有 clippy 告警，如果有则使用 CodeAgent 消除告警
  - **迭代修复策略**：每次只修复第一个告警，然后重新检查告警，迭代修复直到没有告警
  - 每次修复前：记录运行前的 commit id，执行 `cargo fmt` 格式化代码
  - 每次修复后：执行 `cargo test` 验证，通过后保存进度和 commit id，失败时回退到运行前的 commit
  - 如果没有告警，则跳过此步骤
  - Agent 名字：`ClippyWarningEliminator-iter{iteration}`
  - **测试代码删除检测（基于事件订阅）**：所有 CodeAgent 实例都会自动订阅 `BEFORE_TOOL_CALL` 和 `AFTER_TOOL_CALL` 事件，在每次工具调用后立即检测测试代码删除，如果检测到问题则立即回退
- 1. unsafe 清理（CodeAgent）：
  - **迭代处理策略**：每次只处理一个文件，迭代处理所有目标文件
  - 使用 CodeAgent 识别并移除该文件中不必要的 `unsafe { ... }` 包裹
  - 若必须使用 unsafe，缩小 unsafe 块的范围，并在紧邻位置添加 `/// SAFETY: ...` 文档注释说明理由
  - 每个文件处理前：记录运行前的 commit id，执行 `cargo fmt` 格式化代码
  - 每个文件处理后：执行 `cargo test` 验证，通过后保存进度和 commit id，失败时回退到运行前的 commit
  - Agent 名字：`UnsafeCleanupAgent-file{file_idx}`
- 1. 可见性最小化（CodeAgent）：
  - 使用 CodeAgent 将 `pub fn` 降为 `pub(crate) fn`（如果函数仅在 crate 内部使用）
  - 保持对外接口（跨 crate 使用的接口，如 lib.rs 中的顶层导出）为 `pub`
  - 运行前：记录运行前的 commit id，执行 `cargo fmt` 格式化代码
  - 运行后：执行 `cargo test` 验证，通过后保存进度和 commit id，失败时回退到运行前的 commit
  - Agent 名字：`VisibilityOptimizer`
- 1. 文档补充（CodeAgent）：
  - 使用 CodeAgent 为缺少模块级文档的文件添加 `//! ...` 模块文档注释
  - 为缺少函数文档的公共函数添加 `/// ...` 文档注释（可以是占位注释或简要说明）
  - 运行前：记录运行前的 commit id，执行 `cargo fmt` 格式化代码
  - 运行后：执行 `cargo test` 验证，通过后保存进度和 commit id，失败时回退到运行前的 commit
  - Agent 名字：`DocumentationAgent`
- **附加说明支持**：从 `config.json` 读取 `additional_notes`，自动追加到所有 CodeAgent 优化和构建修复提示词末尾，确保用户自定义要求在整个优化流程中生效

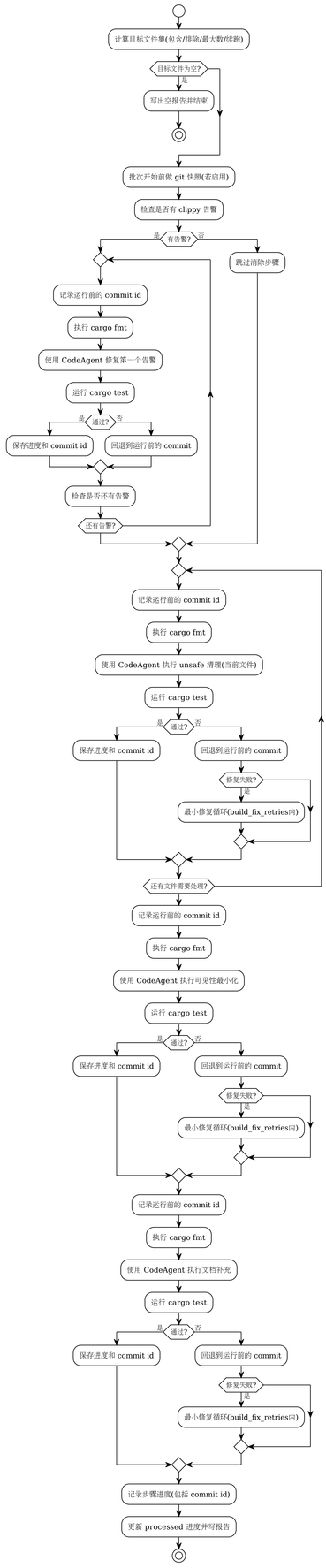
关键数据与度量

- 报告：.jarvis/c2rust/optimize_report.json（files_scanned、unsafe_removed/annotated、visibility_downgraded、docs_added、cargo_checks、errors）
- 进度：.jarvis/c2rust/optimize_progress.json（processed 文件列表、steps_completed 步骤列表、step_commits 每个步骤的 commit id、last_commit 最后一步的 commit id，posix 相对路径）
- 验证预算：max_checks 限制 cargo test 次数；超出预算将提前停止修复
- 断点恢复机制：每个步骤完成后记录 commit id，恢复时自动检测代码状态，如果不一致则 reset 到对应的 commit id，确保代码状态与进度记录一致

内部关系


Optimizer 内部关系
输入文件
(文件列表/排除/限制)
解析依赖
拓扑排序
Clippy 警告收集
(cargo clippy 输出)
处理第
CodeAgent 初始分析
(理解代码结构
识别可优化点)
文件级迭代
(Outer Loop)
迭代 i = 1 ~ N
所有可整理项完成 → 进入下一步
所有文件处理完成 → 下一步
迭代迭代前的 commit id
使用修复下一个警告
CodeAgent 执行修复
CodeAgent unsafe 清理
(移除不必要的 unsafe
缩小 unsafe 块范围)
使用完成下一个文件(相同)
CodeAgent 文档补充
(为模块/函数
添加文档注释)
构建 / 测试
(cargo test)
迭代 i = 第首次迭代前的 commit
迭代 i = 迭代第(i-1)次的 commit
迭代 i = 迭代第(i-1)次的 commit
迭代 i = 迭代第(i-1)次

优化主流程（按步骤验证，失败可修复/回滚）



参数与行为要点（与 CLI optimize 对应）

- `crate_dir`：自动检测（当前目录/Cargo.toml/同级 `<cwd>_rs` /目录下首个 Cargo.toml）
- `enable_*`：三类优化步骤开关（unsafe/visibility/doc），均为布尔值，默认全部启用；所有优化步骤均使用 CodeAgent 完成
- `max_checks`：限制 `cargo test` 次数预算；为 0 表示不限；超出预算将提前停止修复
- `dry_run`：仅统计潜在修改，不写盘；仍统计 `files_scanned` 及可能的计数增量
- `include/exclude`：大项目分批优化；路径相对 crate 根，支持通配（如 `src/**/*.rs`）
- `max_files`：本次最多处理的文件数（0 表示不限），用于大项目分批优化
- `resume/reset_progress`：跳过已处理文件/清空 `processed` 并重跑
- `build_fix_retries`：构建失败时的最小修复重试次数（CodeAgent），默认 3
- `git_guard`：步骤失败后自动回滚到步骤前快照（每个优化步骤前记录 HEAD，失败时 `git reset --hard`）

边界与策略

- **验证方式**：统一使用 `cargo test -q`（而非 `cargo check`），确保最小测试也能通过；预算超限则提前终止修复
- **回滚机制**：所有文本改动均以最小修改为目标；失败立即回滚本处修改（或整步回滚）。若启用 `git_guard`，步骤失败且修复耗尽时自动 `git reset --hard` 回快照
- **CodeAgent 修改范围**：严格限制在本批次文件范围内（除非确有必要），避免扩大影响面
- **公开 API 保守性**：可见性仅在不破坏外部使用时缩小；跨 crate 接口必须保留为 `pub`
- **断点续跑**：使用 `optimize_progress.json` 记录 `processed` 文件列表（posix 相对路径）、`steps_completed` 步骤列表、`step_commits` 每个步骤的 commit id、`fix_progress` 每个修复的 commit id，支持跳过已处理文件；恢复时自动检测代码状态，如果不一致则 reset 到最后一个已完成步骤的 commit id，确保代码状态与进度记录一致
- **报告输出**：优化结果写入 `optimize_report.json`，包含统计摘要（files_scanned/unsafe_removed/unsafe_annotated/visibility_downgraded/docs_added/cargo_checks/errors）
- **代码格式化**：每个 CodeAgent 修复前执行 `cargo fmt` 格式化代码，确保代码格式一致
- **所有优化使用 CodeAgent**：所有优化步骤（clippy 告警消除、unsafe 清理、可见性优化、文档补充）均使用 CodeAgent 完成，确保智能、准确且可回退
- **迭代修复策略**：
  - Clippy 告警修复：每次只修复第一个告警，迭代修复直到没有告警
  - Unsafe 清理：每次只处理一个文件，迭代处理所有目标文件
- **测试验证与回退**：每个 Agent 调用后立即执行 `cargo test` 验证，通过后保存进度和 commit id，失败时自动回退到运行前的 commit，确保代码始终处于可编译、可测试的状态
- **Agent 命名**：每个 Agent 都有唯一的名字，便于在日志和调试中识别（如 ClippyWarningEliminator-iterN、UnsafeCleanupAgent-fileN、VisibilityOptimizer、DocumentationAgent、BuildFixAgent-iterN）
- **测试代码删除检测（基于事件订阅）**：
  - 实现机制：所有 CodeAgent 实例在创建时自动订阅 `BEFORE_TOOL_CALL` 和 `AFTER_TOOL_CALL` 事件
  - 工具调用前记录：在 `BEFORE_TOOL_CALL` 事件处理器中记录当前 commit id，存储在 `_agent_before_commits` 字典中
  - 工具调用后检测：在 `AFTER_TOOL_CALL` 事件处理器中，仅对编辑文件的工具调用进行检测，使用 `de-`

`tect_test_deletion` 检测是否删除了测试代码标记
    - LLM 判断与回退：如果检测到删除，询问 LLM 判断是否合理；若不合理则立即回退到工具调用前的 commit
    - 覆盖范围：所有优化步骤中的 CodeAgent（Clippy 告警消除、unsafe 清理、可见性优化、文档补充、构建修复）都受到保护
    - 优势：相比在 `agent.run()` 完成后检测，事件订阅机制能够更及时地发现问题并回退，避免问题累积，确保测试代码不会被意外删除

5.7 CLI（命令行协调者）

职责

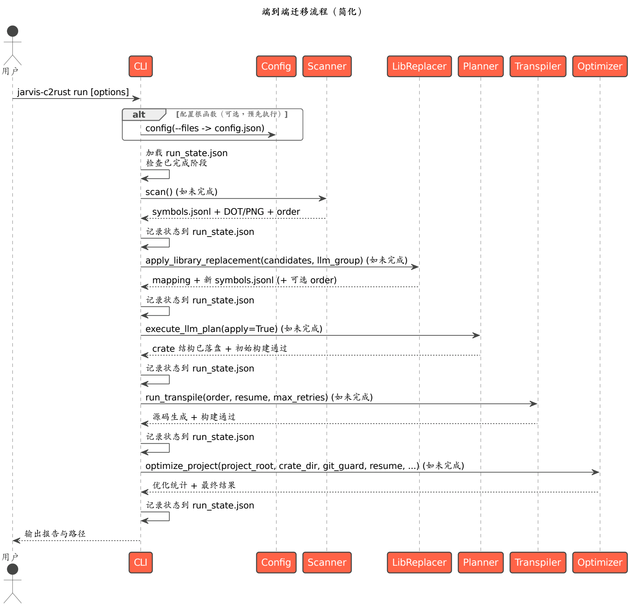
- 使用 Typer 暴露命令：`config`（配置管理）、`run`（一键流水线，支持断点续跑）和 `verify`（功能对齐验证）。
- 在 `run` 中统一 orchestrate 完整流水线（scan → lib-replace → prepare → transpile → optimize），根据 `run_state.json` 自动跳过已完成的阶段；`verify` 作为独立命令在 `run` 完成后执行功能对齐验证。

关键行为

- init_env：初始化运行环境与欢迎提示
- 懒加载：避免未使用模块的硬依赖
- 断点续跑：根据 `run_state.json` 记录各阶段完成状态，支持从任意阶段继续执行
- 错误与摘要：统一输出错误告警与阶段性摘要（mapping/order/优化统计等）

6. 模块间交互流程（端到端）

典型“先收集根→扫描→库替代→规划→转译→优化”的端到端时序。



要点

- 每一阶段产物作为下一阶段输入，失败时提供可回退策略（告警并保留当前产物，避免污染）。
- run 对参数进行约束与默认处理，提升整体可用性与确定性。

7. 配置与参数说明（概览）

CLI 命令主要参数（源码为准）

- **config**：管理配置文件
    - `--files`（头文件或函数名列表文件）、`--root-list-syms`（根符号列表内联）、`--disabled-libs`（禁用库列表）、`--additional-notes`（附加说明，将在所有 agent 的提示词中追加）、`--enable-ffi-export-validation` / `--disable-ffi-export-validation`、`--show`（显示配置）、`--clear`（清空配置）
- **run**：一键流水线，支持断点续跑
    - `-g/--llm-group`（LLM 相关阶段使用的模型组）、`-m/--max-retries`（transpile 构建/修复最大重试次数，0 表示不限制）、`--interactive`（启用交互模式，默认非交互模式）、`--reset`（重置状态，从头开始执行所有阶段）
    - 配置来源：从 `config.json` 读取 `root_symbols`、`disabled_libraries` 和 `additional_notes`（使用 `config` 命令预先设置）
    - 断点续跑：根据 `run_state.json` 自动跳过已完成的阶段，支持从任意阶段继续
- **verify**：功能对齐验证
    - `-g/--llm-group`（用于分析与优化的模型组）、`-i/--max-iterations`（最大迭代次数，默认 10）、`--interactive`（启用交互模式）
    - 前置条件：需先完成 `run`，否则会提示先执行 `jarvis-c2rust run`

8. 可靠性与容错设计

- 产物稳定与断点续跑：阶段产物集中在 .jarvis/c2rust，便于中断后继续（resume/进度文件）。
- 构建失败修复：转译与优化阶段提供最小修复循环（次数可控），失败时清晰告警并回退。
- git 保护（optimizer）：启用 git_guard 时在修复耗尽后自动 reset 到快照，避免破坏仓库状态。
- 懒加载与独立运行：CLI 避免不必要依赖；各模块可独立运行与调试。
- 参数约束与提示：run 对参数提供约束提示，减少误用；lib-replace 对根集合来源进行校验与去重。

9. 扩展与二次开发建议

- 规则与扫描扩展：在 scanner 中扩展类型与函数信息采集策略（宏处理、更多语言支持），完善顺序计算与根识别。
- 评估策略扩展：library_replacer 中引入更多库策略与禁用集；对剪枝策略进行更精细控制（保留/替代的条件）。
- 结构规划增强：llm_module_agent 引入更多约束（命名规范、模块边界规则），增强 JSON 校验与可视化。
- 转译器增强：transpiler 中扩展语义映射与构建修复启发式，优化符号映射与跨文件依赖处理。
- 优化器细化：optimizer 中对 unsafe 清理与可见性优化提供更细粒度规则，采用迭代修复策略（Clippy 告警每次只修复第一个，unsafe 清理每次只处理一个文件），每个 Agent 调用后立即验证，失败时自动回退到运行前的 commit。
- CLI 生态：新增 analyze/export 等子命令，将各阶段摘要与统计导出为独立报告或图形化展示。

附：参考源码关键入口

- scanner.run_scan / compute_translation_order_jsonl / generate_dot_from_db / export_root_subgraphs_to_dir
- collector.collect_function_names
- library_replacer.apply_library_replacement
- llm_module_agent.execute_llm_plan / plan_crate_json_text / apply_project_structure_from_json
- transpiler.run_transpile
- optimizer.optimize_project
- cli 命令：config（配置管理）、run（一键流水线，支持断点续跑）、verify（功能对齐验证）

核心算法与策略

代码安全问题检测算法设计

本节基于 src/jarvis/jarvis_sec 下的实现，对“四阶段流水线”（启发式扫描 → 聚类 → 分析 → 报告）的安全问题检测算法进行结构化说明，覆盖总体流程、核心启发式规则、聚类与验证策略、证据与评分、误报控制、复杂度与可扩展性等。

四阶段流水线概述

系统采用“四阶段流水线”设计：

1. **阶段1：启发式扫描**：通过纯 Python + 启发式检查器提供“可复现、可离线”的安全问题直扫基线，输出候选问题列表（candidates.jsonl，兼容旧版 heuristic_issues.jsonl）。
2. **阶段2：聚类**：将候选问题按文件分组，使用单Agent对每个文件内的告警进行“验证条件一致性”聚类，生成聚类批次（clusters.jsonl，兼容旧版 cluster_report.jsonl）。
3. **阶段3：分析**：将每个聚类批次交由单Agent执行“只读验证”，确认是否存在真实安全风险，输出验证确认问题（analysis.jsonl）。
4. **阶段4：报告**：通过报告聚合器将所有确认问题聚合为 JSON + Markdown 报告，包含统计概览与详细条目。

一、阶段1：启发式扫描算法

总体流程与数据流

- 文件发现与过滤（workflow._iter_source_files）
    - 递归枚举目标仓库源文件
    - 包含扩展名：.c/.cpp/.h/.hpp/.rs
    - 默认排除目录：.git、build、out、target、third_party、vendor（任一祖先匹配即排除）
    - 以相对 entry_path 的相对路径输出，便于跨平台与报告呈现
- 语言分派与检查器
    - C/C++：checkers.c_checker.analyze_files（导出为 analyze_c_files）
    - Rust：checkers.rust_checker.analyze_rust_files
- 产物：candidates.jsonl（结构化候选问题列表，兼容 heuristic_issues.jsonl）

C/C++ 启发式检测算法（checkers/c_checker.py）

1. 预处理（降低误报，保持行号稳定）

- _strip_if0_blocks：跳过 #if 0 主体，保留 #else 与行数
- _remove_comments_preserve_strings：移除注释（//、/**/），保留字符串/字符字面量与换行，维持列/行稳定
- _mask_strings_preserve_len：将字符串/字符字面量内容掩蔽为空格，保留引号与换行，便于“通用 API 匹配”避免把字面量当作代码
- 使用策略：
  - 需解析字面量的规则（如格式串、scanf 宽度）使用原始行（clean_text）
  - 通用 API/关键字匹配、复杂语义线索使用掩蔽行（masked_text）

1. 规则分类与要点（部分示例）

- 不安全/边界类
  - unsafe_api：strcpy/strcat/gets/sprintf/vsprintf 等（跳过头文件函数原型）
  - buffer_overflow：memcpy/memmove/strncpy/strncat 等；若第三参明显为 sizeof(非指针解引用) 且不含 strlen，则认定更安全用法并跳过；若出现 strlen/sizeof(*ptr) 则提高风险
  - strncpy/strncat 未保证 NUL 结尾（邻近窗口无 '\0' 处理提示风险）
  - scanf %s 未限宽（忽略 %*s、GNU %ms/%m[…] 等更安全用法）
  - alloc_size_overflow：malloc(size 含乘法且未显式使用 sizeof)
  - alloca_unbounded/VLA：非常量/未界定的栈分配
- 内存管理类
  - realloc_overwrite：realloc 直接覆写原指针（建议临时变量承接）
  - alloc_no_null_check：malloc/calloc/new 后未见 NULL 检查即使用
  - use_after_free_suspect：free(var) 后窗口内检测到解引用且未重新赋值/置空
  - double_free/free_non_heap：重复释放或对非堆内存（&var/字符串字面量）调用 free
  - possible_null_deref：p-> 或 *p 解引用附近无 NULL 判定（带上下文过滤乘法误报）
  - wild_pointer_deref：未初始化指针声明后在赋值前被解引用
- 错误处理与并发类
  - unchecked_io：I/O/系统调用可能未检查返回值（尝试识别赋值后对变量的判断以降低误报）
  - pthread_ret_unchecked：pthread 常见接口返回值未检查
  - cond_wait_no_loop：pthread_cond_wait 未置于 while 调词循环
  - deadlock_patterns：双重加锁、锁顺序反转、缺失解锁怀疑（启发式）
  - thread_leak_no_join：创建线程后未见 join/detach
- 输入/权限/敏感信息与网络时间等
  - format_string：printf/snprintf/fprintf 等格式串参数非字面量（允许本地化包装与回看变量字面量赋值）
  - command_exec：system/exec* 非字面量参数（回看变量是否被赋值为字面量）
  - getenv_unchecked：从环境变量读取后未见显式校验
  - rand_insecure：安全敏感上下文使用 rand/srand
  - strtok_nonreentrant：非重入线程不安全
  - open_permissive_perms：open(…, O_CREAT, 0666/0777/…) 权限过宽；fopen 在敏感上下文写模式提示
  - inet_legacy/time_api_not_threadsafe：过期/非线程安全接口使用

1. 置信度与严重度

- 置信度 confidence ∈ [0,1]：按命中模式 + 临近窗口信号加减（如存在边界检查/NULL 检查/字面量赋值/SAFETY 注释等）
- 严重度 severity：_severity_from_confidence(conf, base) 阈值映射
  - conf ≥ 0.8 → high；≥ 0.6 → medium；否则 low
- 证据 evidence：截断 200 字符的命中行（剔除缩进与制表符）

Rust 启发式检测算法（checkers/rust_checker.py）

- unsafe_usage：unsafe 代码与关键不安全接口（mem::transmute、MaybeUninit/assume_init、原始指针）
- 错误处理：unwrap/expect 与忽略结果（let _ = / .ok()）等）
- FFI 与并发：extern “C”、unsafe impl Send/Sync
- 置信度与降噪：
  - SAFETY 注释附近降低置信度（_has_safety_comment_around）
  - 测试上下文（#[test] 或 mod tests）降低置信度
  - 严重度映射与 C/C++ 类似（high/medium/low）
- 证据与行号同上

二、阶段2：聚类算法

聚类策略与流程

- 候选精简与全局编号：
  - 将候选精简为 compact_candidates（language/category/pattern/file/line/evidence/confidence/severity）
  - 为每条分配全局 gid（1..N），用于跨批次/跨文件统一编号与跟踪
- 按文件分组：
  - 将候选按文件分组，每个文件内按 cluster_limit 分批（默认 50）
- 验证条件一致性聚类：
  - 构造聚类 Agent，以“验证条件一致性”为准聚合相近告警
  - 摘要输出格式：仅在 … 内输出 JSON 数组（支持 json5 语法）：
    - verification: 字符串（本簇验证条件）
    - gids: 整数数组（属于该簇的全局编号）
- 断点恢复：
  - 支持断点恢复：若 clusters.jsonl 存在，优先复用已有聚类结果
- 产物：clusters.jsonl（按文件/批次写入聚类结果）

聚类容错机制

- 聚类摘要解析失败时重试（至多 2 次）；失败批次仍标记进度并继续
- 聚类快照写入失败不阻断主流程，只打印或记录事件

三、阶段3：分析算法

验证策略与流程

- 构造验证批次：
  - 基于聚类结果构造验证批次（每个批次包含同一验证条件的候选）
- 构造验证 Agent：
  - 工具限制：read_code/execute_script/save_memory/retrieve_memory（记忆工具）；禁止写操作
  - **必须进行调用路径推导**：
    - 对于每个候选问题，必须明确推导从可控输入到缺陷代码的完整调用路径。
    - 调用路径推导必须包括：识别可控输入来源、追踪数据流、识别调用链、分析每个调用点的校验情况、确认触发条件。
    - 必须向上追溯所有可能的调用者，使用工具查找函数的调用者，检查每个调用者是否对输入进行了校验。
  - 摘要输出格式：仅在 … 内输出 JSON 数组（支持 json5 语法），每个元素：
    - gid: int（全局编号，>=1）
    - has_risk: bool
    - preconditions: string（触发漏洞的前置条件，当 has_risk=true 时必需）
    - trigger_path: string（漏洞的触发路径，必须包含完整的调用路径推导，不能省略或简化。必须明确说明从可控输入到缺陷代码的完整调用链，以及每个调用点的校验情况。如果无法推导出完整的调用路径，应该判定为误报（has_risk: false），当 has_risk=true 时必需）
    - consequences: string（漏洞被触发后可能导致的后果，当 has_risk=true 时必需）
    - suggestions: string（修复或缓解该漏洞的建议，当 has_risk=true 时必需）
- 解析与校验：
  - 解析严格校验字段与类型；成功则将“确认风险”的条目增量写入 .jarvis/sec/analysis.jsonl
  - 支持重试：摘要解析失败或关键字段缺失时，最多重试 2 次
- 工作区保护：
  - 每次运行 Agent 后检测 git 工作区是否有变更；如有通过 git checkout – . 恢复，记录 meta（workspace_restore）
- 进度追踪：
  - 记录 batch_status 与 task_status 事件，并将每个候选标记为已处理
  - 基于进度文件跳过已完成的候选（通过 candidate_signature 匹配）
- 产物：analysis.jsonl（验证确认问题列表）

验证容错机制

- 验证摘要解析失败时重试（至多 2 次）；失败批次仍标记进度并继续
- 严禁 Agent 写操作；通过工具白名单（read_code/execute_script）与只读约束保证安全
- 工作区保护：若检测到文件被修改，立即恢复，确保分析只读

四、阶段4：报告生成算法（report.py）

数据归一化与评分

- 统一结构化 Issue（types.Issue）：language、category、pattern、file、line、evidence、description、suggestion、confidence、severity
- 归一化与稳定 ID
  - _normalize_issue 将 Issue 或 dict 统一为 dict，并补默认值
  - 稳定 ID：id = PREFIX + sha1(file:line:category:pattern) 前 6 位；C 前缀为 C，Rust 为 R
- 评分规则
  - 严重度权重：_SEVERITY_WEIGHT = {“high”:3.0, “medium”:2.0, “low”:1.0}
  - 分数：score = round(confidence × severity_weight, 2)
- Summary 聚合
  - total/by_language/by_category/top_risk_files
  - Top 风险文件按累计 score 降序，更稳定可解释
  - 预设类别顺序 _CATEGORY_ORDER 用于 Markdown 展示；不在该序列的类别仍出现在 JSON 的 by_category 中
- Markdown 渲染
  - format_markdown_report：输出统计概览与逐条问题详情（含评分），附改进建议段落
  - build_json_and_markdown：一次性输出 JSON + Markdown，便于评测
- 产物：最终报告（JSON + Markdown 文本字符串）

五、误报控制与边界

- 误报控制要点
  - 注释移除 + 字符串掩蔽，避免把注释/字面量作为证据
  - 头文件原型过滤（unsafe_api）
  - 边界信号回看：长度上界/NULL 检查/条件判断/变量字面量赋值等
  - 复杂语义采用滑窗与多信号合成（UAF、死锁、线程生命周期等）
  - Rust 中 SAFETY 注释与测试上下文降低置信度
- 边界与取舍
  - 属启发式检测，部分规则需人工确认；报告中通过 confidence/severity/score 提示优先级
  - Markdown 概览按预设类别输出，新增类别以 JSON 为准
  - 直扫默认逐文件正则，保留 rg 加速接口；大仓库可结合目录排除降低复杂度

六、复杂度与性能

阶段1（启发式扫描）

- 文件遍历 O(N_files)，行级规则总体 O(Σlines)
- 预处理按线性时间实现，字符串掩蔽与注释移除保持行号稳定
- 可按需启用目录排除/语言过滤；预留 rg 批搜索接口（批大小 200）

阶段2（聚类）

- 按文件分组 O(N_candidates)，聚类 Agent 调用按批次串行
- 聚类快照写入支持断点恢复，避免重复计算

阶段3（分析）

- 验证 Agent 调用按批次串行，每个批次独立验证
- 工作区保护开销可忽略（git status + checkout）

阶段4（报告）

- 聚合与评分 O(N_issues)，Markdown 渲染 O(N_issues)

总体性能

- 四阶段串行执行，总耗时主要取决于阶段1（文件扫描）与阶段3（Agent 验证）
- 支持断点续扫，中断后可恢复至任意阶段继续执行

七、可扩展性

规则库扩展

- 规则库可在 c_checker/rust_checker 中按模块化函数扩展；优先保证：
  - 先预处理 → 再匹配 → 滑窗复核 → 动态调节置信度
  - 返回最小可用结构化 Issue，描述与建议清晰可操作

聚类策略扩展

- 优化“验证条件一致性”聚类提示词，使簇内条件更可执行
- 引入局部上下文读取策略以减少误聚

验证策略扩展

- 根据类别生成专用验证模板（指针边界、长度/对齐、错误传播），提升确认质量

报告扩展

- 聚合与报告通过 report 模块统一收敛；新增类别只需规则产出即可被 JSON 记录，Markdown 可通过 _CATEGORY_ORDER 配置显示顺序
- 扩展评分维度与审阅视图（文件粒度统计、按验证条件聚合）

断点与可靠性

- 为快照与增量报告引入原子写与校验，提升长任务可靠性
- 支持多文件/多批次的并行处理（未来扩展）

C/C++到Rust转换策略和决策树

本节基于 src/jarvis/jarvis_c2rust 下的实现（scanner、library_replacer、llm_module_agent、transpiler、optimizer），梳理从“C/C++ 函数/子图候选”到“Rust 实现/库替代”的完整策略与决策树，覆盖路径选择、签名与类型映射、FFI 边界与错误语义、构建修复闭环与优化回退。

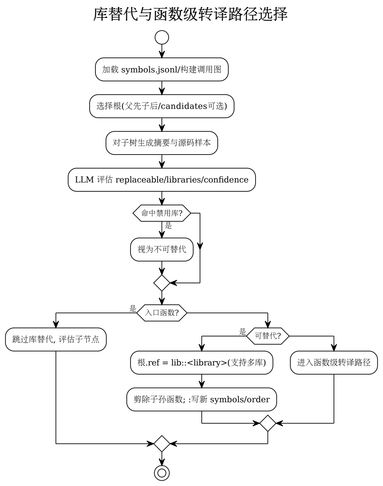
一、总体策略与阶段职责

- **扫描与顺序（scanner.py）**
  - 产物：`symbols.jsonl`（统一符号表，包含函数与类型）、`symbols_raw.jsonl`（原始数据）、`translation_order.jsonl`（函数转译顺序，新格式：每行包含 `ids` 和 `items`）、子图/DOT/PNG（可选）
  - 用途：为“库替代评估”和“函数级转译”提供准确的函数/类型与引用关系基线
  - 技术要点：使用 libclang 16-21 解析 AST，构建调用图（id→id 邻接表），按拓扑排序生成转译顺序
- **库替代评估与剪枝（library_replacer.py.apply_library_replacement）**
  - 目标：若某“根函数的可达子树”可整体由成熟 Rust 库 API 组合替代，则保留根为库占位 `lib::<library>`（支持多库组合），剪除其子孙函数（仅函数类别，类型保留）
  - 保护与门控：
    - `main` 等入口默认不替代（可通过环境变量 `JARVIS_C2RUST_DELAY_ENTRY_SYMBOLS` 配置多个入口名）
    - 命中 `disabled_libraries`（大小写不敏感）强制判定不可替代
    - 直接使用约束：如果当前调用的函数无法使用 crate 直接提供的功能而需要封装或者改造，则认为不可替代（确保仅使用成熟库的标准 API，避免自定义封装层）
    - 支持断点续跑（`library_replacer_checkpoint.json`）与原子写
  - 评估策略：LLM 评估子树摘要（函数列表/源码样本/DOT 边），输出 JSON 格式（`replaceable` / `libraries` / `library` / `apis` / `confidence` / `notes`，支持 JSON5 语法）；评估提示词中明确要求：如果无法使用 crate 直接提供的功能而需要封装或改造，则判定为不可替代
  - 产物：新符号表（`symbols_library_pruned.jsonl` + 兼容别名 `symbols.jsonl`）、替代映射（`library_replacements.jsonl`）、转译顺序（`translation_order_prune.jsonl` + 别名 `translation_order.jsonl`）
- **模块规划与落盘（llm_module_agent.py.execute_llm_plan）**
  - 目标：以引用子图为上下文，通过 LLM 生成 crate 模块结构（JSON），并写盘（创建目录/文件/`pub mod` 声明），确保初始 `cargo check` 通过（最小修复）
  - 技术要点：基于根函数子图构建上下文（仅函数名列表，不包含签名/路径等），LLM 输出 JSON 格式（支持 JSON5 语法），解析后确保 `pub mod` 声明链完整，创建 `Cargo.toml`（若不存在）
- **函数级转译（transpiler.py.run_transpile）**
  - 目标：按 `translation_order.jsonl` 逐函数完成“模块与签名规划 → 代码生成 → 构建修复闭环 → 审查与类型边界复核 → 符号映射”
  - 技术要点：
    - 支持断点续跑（`progress.json` 记录 `current` / `converted`）
    - 支持 `--only` 过滤（仅转译指定函数）
    - 符号映射使用 JSONL 格式（`symbol_map.jsonl`），支持同名/重载（兼容旧版 `symbol_map.json`）
    - 构建验证：先 `cargo check`，通过后 `cargo test`
    - 错误分类：`missing_import` / `type_mismatch` / `visibility` / `borrow_checker` / `dependency_missing` / `module_not_found`
    - 模块化设计：`transpiler.py` 已拆分为多个支持模块以提高可维护性（`constants.py`、`models.py`、`loaders.py`、`utils.py`）
- **保守优化（optimizer.py.optimize_project）**
  - 目标：在保持可构建前提下“最小”提升质量（clippy 告警消除、unsafe 清理、可见性缩小、文档补齐），失败可回滚
  - 技术要点：
    - 所有优化步骤均使用 CodeAgent 完成，确保智能、准确且可回退
    - 检查 clippy 告警并使用 CodeAgent 消除（如果没有告警则跳过）
    - Clippy 告警修复：每次只修复第一个告警，迭代修复直到没有告警
    - Unsafe 清理：每次只处理一个文件，迭代处理所有目标文件
    - 每个 CodeAgent 修复前执行 `cargo fmt` 格式化代码
    - 每个 Agent 调用后立即执行 `cargo test` 验证，通过后保存进度和 commit id，失败时自动回退到运行前的 commit
    - 验证统一使用 `cargo test -q`（而非 `cargo check`）
    - 支持 `git_guard`（步骤前快照，失败时 `git reset --hard`）
    - 支持断点续跑（`optimize_progress.json` 记录 `processed` 文件列表、`steps_completed` 步骤列表、`step_commits` 每个步骤的 commit id、`fix_progress` 每个修复的 commit id，恢复时自动 reset 到对应的 commit id）
    - 分批优化：支持 `include` / `exclude` / `max_files` 控制处理范围
    - 每个 Agent 都有唯一的名字，便于在日志和调试中识别

二、宏观路径选择（库替代 vs 函数级转译）

- **决策依据（library_replacer）**
  - `replaceable`：LLM 评估该根的可达子树是否可由“指定库”（`library_name` 参数）或“多库组合”（`libraries` 列表）整体替代
  - **禁用库门控**：若 LLM 建议命中 `disabled_libraries`（大小写归一），强制判定不可替代，并在备注中追加告警
  - **直接使用约束**：如果当前调用的函数无法使用 crate 直接提供的功能而需要封装或者改造，则认为不可替代（确保仅使用成熟库的标准 API，避免自定义封装层）
  - **入口保护**：`main` 默认跳过库替代（改为深入评估其子节点），可通过环境变量配置多个入口名：
    - `JARVIS_C2RUST_DELAY_ENTRY_SYMBOLS` / `JARVIS_C2RUST_DELAY_ENTRIES` / `C2RUST_DELAY_ENTRIES`（逗号/空白/分号分隔）
  - **候选根限制**：若提供 `candidates`（名称或限定名列表），仅评估这些根，作用域限定为其可达子树；不可达函数将直接删除（仅函数类别，类型保留）
  - **评估顺序**：默认从所有根函数开始，采用近似“父先子后”的顺序（广度遍历）；若不存在无入边节点，回退为全量函数集合
- **行为与产物**
  - **可替代**：
    - 保留根（不删除），将根的 `ref` 设置为库占位（`lib::<library>`，支持多库组合）
    - 剪除其所有子孙函数（仅函数类别，类型记录不受影响）
    - 写入 `lib_replacement` 元数据（`libraries` / `library` / `apis` / `api` / `confidence` / `notes` / `mode` / `updated_at`）
    - 写出新符号表（`symbols_library_pruned.jsonl` + 兼容别名 `symbols.jsonl`）
    - 基于剪枝表计算转译顺序（`translation_order_prune.jsonl` + 别名 `translation_order.jsonl`）
    - 记录替代映射到 `library_replacements.jsonl`（每行一个 JSON 对象）
  - **不可替代**：保持原图，递归评估其子节点（深度优先），进入函数级转译路径

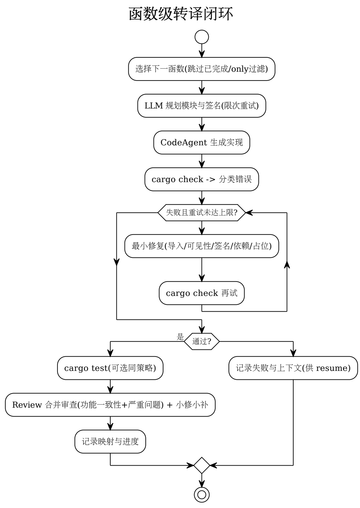
PlantUML：路径选择（简化）



三、函数级转译闭环策略（transpiler）

- **模块与签名规划**
  - **依据**：C 源片段（按起止行读取）、调用上下文（已转译/未转译：若已转译则提供 Rust 模块与符号，否则提供原 C 位置信息）、crate 目录树（`_dir_tree` 格式化输出）
  - **检查机制**：对规划结果进行基本格式检查（字段存在性），不做硬编码规则校验，完全依赖 LLM Agent 的智能决策
  - **重试机制**：至多 `plan_max_retries` 次重试（默认 5），失败回退至兜底模块与占位签名
- **代码生成与最小修复**
  - **生成策略**：
    - 使用 CodeAgent 生成实现，遵循“最小变更、禁止 `todo!` / `unimplemented!` 、必要时补齐依赖实现”的约束
    - 确保模块可见性与声明链：`src/lib.rs` 顶层 `pub mod` ，补齐从目标模块文件向上的 `mod.rs` 声明链
  - **构建验证**：
    - 先执行 `cargo check` （更快），通过后再执行 `cargo test`
    - 失败时按分类标签进行聚焦修复：`missing_import` / `type_mismatch` / `visibility` / `borrow_checker` / `dependency_missing` / `module_not_found` 等
    - 每轮修复均保持最小改动：修正声明/依赖、补齐未实现被调函数、精确 `use` 导入（避免通配）、必要时更新 `Cargo.toml`
- **审查与复核**
  - **Review Agent 合并审查**（`_review_and_optimize`）：
    - **功能一致性检查**：核心输入输出、主要功能逻辑是否与 C 实现一致；允许 Rust 实现修复 C 代码中的安全漏洞或使用不同的类型设计、错误处理方式、资源管理方式等，只要功能一致即可
    - **严重问题检查**：明显的空指针解引用、明显的越界访问等可能导致功能错误的问题
    - 不检查类型匹配、指针可变性、边界检查细节、资源释放细节等技术细节（除非会导致功能错误）
  - 审查采用循环验证机制：发现问题 → 使用修复 Agent（每次重新创建，包含原 C 代码上下文）修复 → 构建验证 → 重新审查，最多迭代 `review_max_iterations` 次（0 表示无限重试），直到所有问题修复并通过审查或达到上限
- **进度与映射**
  - **progress.json**：记录 `current`（当前函数信息：id/name/qname/file/位置/module/rust_signature/signature_hint/metrics）与 `converted` （已转换函数 ID 集合），支持断点续跑
  - **symbol_map.jsonl**：记录 C→Rust 符号映射（每行一条记录，支持同名/重载），字段包括：
    - `c_name` / `c_qname` ： C 函数名/限定名
    - `c_file` / `start_line` / `end_line` ： 源文件位置
    - `module` ： Rust 模块路径（`src/xxx.rs` 或 `src/xxx/mod.rs` ）
    - `rust_symbol` ： Rust 函数名
    - `updated_at` ： 时间戳
  - **占位清理**：清理 crate 源码中对当前符号的 `todo!("sym")` / `unimplemented!("sym")` 占位，替换为真实调用并回归测试（`_resolve_pending_todos_for_symbol` ）

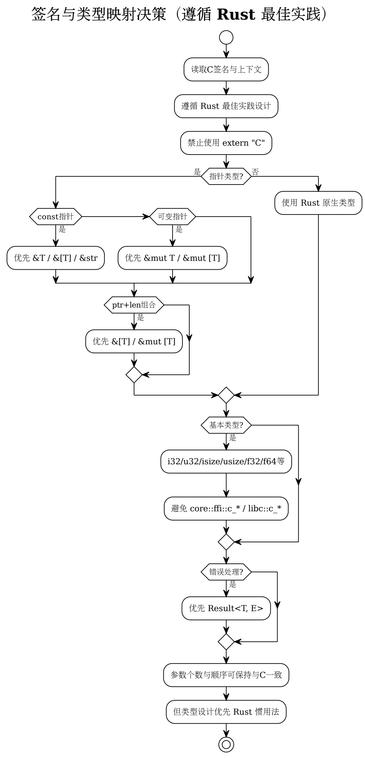
PlantUML：函数级转译闭环（简化）



四、签名与类型映射决策树（遵循 Rust 最佳实践）

- **原则**：函数接口设计应遵循 Rust 最佳实践，不需要兼容 C 的数据类型；禁止使用 `extern "C"` ；优先使用 Rust 原生类型和惯用法，而不是 C 风格类型
- 指针与切片
  - 输入 const 指针：优先使用 &T / &[T]；仅在必要时使用原始指针 *const T
  - 输入可变指针：优先使用 &mut T / &mut [T]；仅在必要时使用原始指针 *mut T
- 指针+长度（ptr,len）
  - 首选 &[T]/&mut [T]；仅在特殊场景下保留原始指针
- 字符串与字节
  - const char* NUL 终止：优先使用 &str（需校验 UTF-8）；必要时使用 &CStr
  - 可变 char* 或需要拥有：优先使用 String；必要时使用 CString / Vec
- 基本类型
  - 优先使用 i32/u32/i64/u64/isize/usize/f32/f64 等原生 Rust 类型，而不是 core::ffi::c_*或 *libc::c_*
- 所有权与生命周期
  - malloc/calloc/realloc/free → Box/Vec/Arc 等 RAII 容器；realloc 覆盖风险由容器规避
  - 输出参数/双指针：优先返回值或 Result<T,E>，必要时 &mut Option 传回
- 结构体/枚举/联合
  - 优先使用 Rust 原生结构体；仅在必要时使用 #[repr(C)]；非等价抽象以 From/Into 实现转换；联合 union 需 unsafe 访问或以枚举+判别值建模
- 错误语义
  - 优先考虑使用 Result<T,E> 而非 C 风格的错误码；如适用，在 Rust API 层统一使用 Result
- 并发
  - pthread_* → std::thread/std::sync 映射；优先使用 Rust 原生并发抽象

PlantUML：签名与类型映射（简化）



五、Rust 最佳实践与安全设计

- **原则**：函数接口设计应遵循 Rust 最佳实践，不需要兼容 C 的数据类型；禁止使用 `extern "C"`；函数应使用标准的 Rust 调用约定
- 类型设计优先考虑 Rust 的惯用法、安全性和可读性
  - 优先使用 Rust 原生类型（i32/u32/usize、&[T]/&mut [T]、String、Result<T, E> 等）
  - 避免使用 C 风格类型（core::ffi::c_*、*libc::c_*），禁止使用 `extern "C"`
  - 仅在必要时使用原始指针或 #[repr(C)]，并在使用处进行适当的安全检查
- 错误处理：优先考虑使用 Result<T, E> 而非 C 风格的错误码
- 安全保证：以 SAFETY 注释记录前置条件（与 rust_checker 中的约定一致）

六、构建修复闭环与度量

- **验证顺序**：
  - `cargo check` → 通过后 `cargo test`（确保最小测试也能通过）
  - 每轮记录 `check_attempts`、`test_attempts`、`impl_verified`、`last_build_error` 等度量到 `progress.json` 的 `current.metrics` 字段
  - check 阶段最多迭代 `check_max_retries` 次（0 表示不限制），test 阶段最多迭代 `test_max_retries` 次（0 表示不限制），超出上限则记录失败并继续下一函数（支持断点续跑）
- **错误分类与修复策略**（`_classify_rust_error`）：
  - `missing_import`：精确 `use` 导入（避免通配），必要时添加模块路径
  - `type_mismatch`：类型对齐，指针可变性一致（`*const T` ↔ `&T`，`*mut T` ↔ `&mut T`）
  - `visibility`：补齐 `pub mod` 声明链，确保从目标模块到 `src/lib.rs` 的路径可访问
  - `borrow_checker`：调整生命周期、借用规则，必要时使用 `Arc` / `Rc` 共享所有权
  - `dependency_missing`：最小增补 `Cargo.toml` 依赖（精确版本号）
  - `module_not_found`：创建缺失模块文件或修正模块路径
- **修复动作（最小化原则）**：
  - `use` 导入精确到符号（避免 `use xxx::*`）
  - 补齐 `pub mod` 声明链（从目标模块向上到 `src/lib.rs`）
  - 签名仅修复（ptr+len→slice、指针可变性一致、返回类型与错误语义）
  - 缺失实现最小占位（仅占位，不实现完整逻辑）
  - 必要时最小增补 `Cargo.toml` 依赖（精确版本，避免过度依赖）
- **断点续跑机制**：
  - 通过 `progress.json` 的 `converted` 集合跳过已转换函数
  - 通过 `symbol_map.jsonl` 的 `has_rec` 方法判断函数是否已转译（基于源位置唯一性）
  - 每个函数转换完成后记录对应的 commit id 到 `converted_commits` 字段
  - 恢复时自动检测代码状态，如果不一致则 reset 到最后一个已转换函数的 commit id，确保代码状态与进度记录一致
  - 支持 `--resume`（默认启用）和 `--only` 过滤（仅转译指定函数）

七、优化与回退（optimizer）

- **Clippy 告警消除（CodeAgent，可选）**：
  - 检查是否有 clippy 告警，如果有则使用 CodeAgent 消除告警（如果没有告警则跳过此步骤）
  - **迭代修复策略**：每次只修复第一个告警，然后重新检查告警，迭代修复直到没有告警
  - 每次修复前：记录运行前的 commit id，执行 `cargo fmt` 格式化代码
  - 每次修复后：执行 `cargo test` 验证，通过后保存进度和 commit id，失败时回退到运行前的 commit
  - Agent 名字：`ClippyWarningEliminator-iter{iteration}`
- **unsafe 最小化（CodeAgent）**：
  - **迭代处理策略**：每次只处理一个文件，迭代处理所有目标文件
  - 使用 CodeAgent 识别并移除该文件中不必要的 `unsafe { ... }` 包裹
  - 若必须使用 unsafe，缩小 unsafe 块的范围，并在紧邻位置添加 `/// SAFETY: ...` 文档注释说明理由
  - 每个文件处理前：记录运行前的 commit id，执行 `cargo fmt` 格式化代码
  - 每个文件处理后：执行 `cargo test` 验证，通过后保存进度和 commit id，失败时回退到运行前的 commit
  - 记录 `unsafe_removed`（成功移除数）和 `unsafe_annotated`（失败回滚并补充注释数）
  - Agent 名字：`UnsafeCleanupAgent-file{file_idx}`
- **可见性缩小（CodeAgent）**：
  - 使用 CodeAgent 将 `pub fn` 降为 `pub(crate) fn`（如果函数仅在 crate 内部使用）
  - 保持对外接口（跨 crate 使用的接口，如 lib.rs 中的顶层导出）为 `pub`
  - 修复前执行 `cargo fmt` 格式化代码
  - 变更后执行 `cargo test -q` 验证
  - 若失败使用 CodeAgent 进行最小修复
  - 记录 `visibility_downgraded`（成功降级数）
  - 对公开 API 保持保守：可见性仅在不破坏外部使用时缩小；跨 crate 接口必须保留为 `pub`
- **文档补齐（CodeAgent）**：
  - 使用 CodeAgent 为缺少模块级文档的文件添加 `//! ...` 模块文档注释
  - 为缺少函数文档的公共函数添加 `/// ...` 文档注释（可以是占位注释或简要说明）
  - 修复前执行 `cargo fmt` 格式化代码
  - 记录 `docs_added`（补充文档数）
- **git_guard 回退机制**：
  - 每个优化步骤前记录当前 HEAD 快照（`_git_head_commit`）
  - 步骤后若构建仍失败且无法修复（`build_fix_retries` 耗尽），自动 `git reset --hard` 回快照（`_git_reset_hard`）
  - 默认启用，可通过 `--no-git-guard` 禁用
- **分批优化与断点续跑**：
  - 支持 `include` / `exclude` / `max_files` 控制处理范围（大项目分批优化）
  - 使用 `optimize_progress.json` 记录 `processed` 文件列表（posix 相对路径）、`steps_completed` 步骤列表、`step_commits` 每个步骤的 commit id，支持跳过已处理文件
  - 每个步骤完成后自动记录当前 commit id，恢复时自动检测代码状态，如果不一致则 reset 到最后一个已完成步骤的 commit id，确保代码状态与进度记录一致
  - 支持 `--resume`（默认启用）和 `--reset-progress`（重置进度）
- **验证预算**：
  - `max_checks` 限制 `cargo test` 次数；超出预算将提前停止修复
  - 为 0 表示不限；在统计中记录 `cargo_checks`（实际执行次数）

八、落地建议（与实现约束一致）

- 优先库替代：当子树可由成熟库覆盖时，优先选用，减少手写 unsafe 与维护面
- Rust 最佳实践：函数接口设计应遵循 Rust 最佳实践，不需要兼容 C 的数据类型；禁止使用 `extern "C"`；优先使用 Rust 原生类型和惯用法
- 所有权清晰：以 RAII 容器取代手动 malloc/free；在接口处一次性转换
- 错误处理：优先考虑使用 Result<T, E> 而非 C 风格的错误码
- 构建先行：任何修复/优化后均以 cargo check/test 通过为准

unsafe使用决策机制

目标与原则

- **最小面积原则**：仅在无法通过安全抽象达成语义时使用 unsafe，且将不安全操作限制在最小作用域内（minimal unsafe block）。优先使用 `&` / `&mut`、slice、RAII 容器（`Box` / `Vec` / `Arc`）取代原始指针/手动内存管理。
- **可证明性**：每处 unsafe 必须具备清晰的前置条件与不变式，并以 `/// SAFETY: ...` 注释进行说明（与 `rust_checker` 规则一致）。注释应包含“假设/前置条件”和“为何成立”的简要证明。
- **Rust 最佳实践优先**：函数接口设计应遵循 Rust 最佳实践，不需要兼容 C 的数据类型；禁止使用 `extern "C"`；优先使用 Rust 原生类型和惯用法。仅在必要时使用 unsafe，并将 unsafe 范围最小化。在 unsafe 边界处进行参数校验（非空/长度/对齐/别名/生命周期），内部转为安全抽象（`&` / `&mut` / `&[T]` / `&str` 等）。

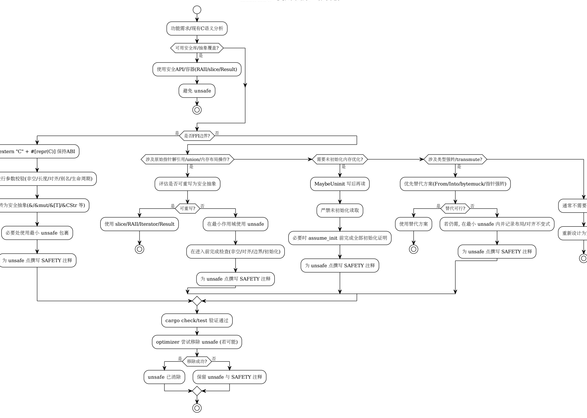
- **可回退与可验证**：通过构建与测试闭环验证（`cargo check` / `cargo test`），能去除则去除；失败则回滚并补全 SAFETY 注释（optimizer 行为）。optimizer 使用 CodeAgent 自动尝试移除 unsafe 块，若移除导致构建失败则回滚，同时为该处生成/补全 `/// SAFETY: ...` 注释。

与实现的对应关系

- **transpiler（转译阶段）**
  - **代码生成策略**：签名与模块规划后使用代码生成 Agent（可复用）生成实现，尽量以 `&` / `&mut`、slice、RAII 容器（`Box` / `Vec` / `Arc`）取代原始指针/手动内存管理。遵循“最小变更、禁止 `todo!` / `unimplemented!`”的约束。
  - **修复策略**：构建和测试失败时使用修复 Agent（每次重新创建，包含原 C 代码上下文）进行修复，确保每次修复都有独立的上下文和状态，提高修复准确性。
  - **Review Agent 合并审查**（`_review_and_optimize`）：
    - 合并检查功能一致性和严重问题（如明显的空指针解引用、明显的越界访问等）
    - 不检查类型匹配、指针可变性、边界检查细节等技术细节（除非会导致功能错误）
    - 采用循环验证机制：发现问题 → 使用修复 Agent（每次重新创建，包含原 C 代码上下文）最小修复 → 构建验证 → 重新审查，最多迭代 `review_max_iterations` 次（0 表示无限重试），循环直到通过或达上限
- **optimizer（优化阶段）**
  - **Clippy 告警消除（CodeAgent，可选）**：
    - 检查是否有 clippy 告警，如果有则使用 CodeAgent 消除告警（如果没有告警则跳过）
    - 迭代修复策略：每次只修复第一个告警，然后重新检查告警，迭代修复直到没有告警
    - 每次修复前：记录运行前的 commit id，执行 `cargo fmt` 格式化代码
    - 每次修复后：执行 `cargo test` 验证，通过后保存进度和 commit id，失败时回退到运行前的 commit
  - **unsafe 最小化流程（CodeAgent）**：
    - 迭代处理策略：每次只处理一个文件，迭代处理所有目标文件
    - 使用 CodeAgent 识别并移除该文件中不必要的 `unsafe { ... }` 包裹
    - 每个文件处理前：记录运行前的 commit id，执行 `cargo fmt` 格式化代码
    - 每个文件处理后：执行 `cargo test` 验证，通过后保存进度和 commit id，失败时回退到运行前的 commit
    - 记录统计：`unsafe_removed`（成功移除数）和 `unsafe_annotated`（失败回滚并补充注释数）
  - **所有优化步骤均使用 CodeAgent**：clippy 告警消除、unsafe 清理、可见性优化、文档补充均使用 CodeAgent 完成，每个步骤修复前执行 `cargo fmt` 格式化代码
  - **测试验证与回退机制**：每个 Agent 调用后立即执行 `cargo test` 验证，通过后保存进度和 commit id，失败时自动回退到运行前的 commit，确保代码始终处于可编译、可测试的状态
  - **Agent 命名**：每个 Agent 都有唯一的名字（如 ClippyWarningEliminator-iterN、UnsafeCleanupAgent-fileN、VisibilityOptimizer、DocumentationAgent、BuildFixAgent-iterN），便于在日志和调试中识别
- **jarvis-sec rust_checker（静态检查）**
  - 对以下规则进行提示：
    - `unsafe` 关键字使用
    - 原始指针（`*const T` / `*mut T`）解引用
    - `transmute` 类型强转
    - `MaybeUninit` / `assume_init` 未初始化内存操作
  - **置信度调整**：若附近存在 `SAFETY` 注释或测试上下文，置信度下降（视为已审核/已验证）

决策树（何时需要 unsafe）

unsafe 使用决策（简化）



决策要点：

1. **优先安全抽象**：评估是否可重写为安全抽象（slice/RAII/Iterator/Result），确需不安全时再按决策树收敛
2. **FFI 边界收敛**：优先将 unsafe 收敛在 FFI 边界层，对外提供安全包装 API
3. **最小作用域**：只在指针解引用/布局敏感段落使用 unsafe；将校验与转换放在 unsafe 外侧
4. **验证闭环**：每次改动后执行 `cargo check/test`；optimizer 使用 CodeAgent 自动尝试移除 unsafe 并回退失败，形成自动化收敛

不变式与检查清单（用于 SAFETY 注释与自检）

- **指针与别名**（Review 合并审查重点）
  - 指针非空（在解引用前已做 `ptr.is_null()` 检查）
  - 可变引用/指针不存在别名（独占可变，不可变可多重）
  - 指向内存对齐且有效（对齐检查：`ptr as usize % align_of::<T>() == 0`）
  - 指针可变性一致（`*const T` ↔ `&T`，`*mut T` ↔ `&mut T`）
- **生命周期与所有权**
  - 引用在其生命周期内有效（不引用已释放内存）
  - 返回的引用/切片不引用临时或已释放内存（避免悬空指针）
  - 跨 FFI 不悬空（确保 FFI 调用期间内存有效）
  - 所有权清晰（以 RAII 容器取代手动 `malloc` / `free`）
- **边界与初始化**（Review 合并审查重点）
  - 对 `(ptr,len)` 组合已做上界检查（`ptr.add(len)` 不越界）
  - 读取前已完成初始化（避免读取未初始化内存）
  - 使用 `MaybeUninit` 时“先写后读”，不提前 `drop`
  - 数组/切片边界检查（索引 `i` 满足 `0 <= i < len`）
- **布局与对齐**（FFI 边界要求）
  - `transmute` / `union` /FFI 结构体满足 `#[repr(C)]` 要求
  - 类型大小/对齐与目标一致（`size_of::<T>()` 和 `align_of::<T>()`）
  - 字节序考虑（大端/小端，跨平台兼容）
- **错误与异常安全**
  - 错误路径保持资源与锁一致释放（RAII 或 `drop` 保证）
  - `panic` 不破坏全局不变式（避免在 `unsafe` 块中 `panic`）
  - `Result` / `Option` 表示异常分支，避免 `unwrap`（在 FFI 边界层统一转换）
- **并发与可发送性**
  - 避免手写 `unsafe impl Send/Sync`；若必须，证明线程安全的内部不变式
  - 优先使用 `std::sync` 构件（`Mutex` / `RwLock` / `Arc` / `Atomic` 等）
  - 避免数据竞争（确保同步原语正确使用）

SAFETY 注释模板（optimizer 自动生成格式）：

```
/// SAFETY: <前置条件说明>
/// - <不变式1>; <证明要点>
/// - <不变式2>; <证明要点>
/// 原因: <为何需要 unsafe 的简要说明>
unsafe { ... }
```

落地策略

- **先安全后不安全**：
  - 评估是否可重写为安全抽象（slice/RAII/Iterator/Result）
  - 确需不安全时再按决策树收敛
  - transpiler 在代码生成阶段尽量以 `&` / `&mut`、slice、RAII 容器取代原始指针/手动内存
- **作用域收敛**：
  - 只在指针解引用/布局敏感段落使用 unsafe
  - 将校验与转换放在 unsafe 外侧（在进入 `unsafe` 块前完成所有检查）
  - 最小化 unsafe 块的范围（仅包裹必要的不安全操作）
- **注释即契约**：
  - 每处 unsafe 前添加 `/// SAFETY: ...` 说明，包含“假设/前置条件”和“为何成立”的简要证明
  - Review 合并审查强制检查 SAFETY 注释完整性，缺失则补充
  - optimizer 使用 CodeAgent 移除 unsafe，失败时自动补充 SAFETY 注释
- **验证闭环**：
  - 每次改动后执行 `cargo check` / `cargo test`（确保最小测试也能通过）
  - optimizer 使用 CodeAgent 自动尝试移除 unsafe 并回退失败，形成自动化收敛
  - 记录统计：`unsafe_removed`（成功移除数）和 `unsafe_annotated`（失败回滚并补充注释数）

与工具链配合

- **rust_checker（jarvis-sec）**：
  - 用于发现潜在不当 unsafe 用法与原始指针/类型强转等风险点
  - 规则覆盖：`unsafe` /原始指针/ `transmute` / `MaybeUninit` / `assume_init` 等
  - 置信度调整：若附近存在 SAFETY 注释或测试上下文，置信度下降（视为已审核/已验证）
- **transpiler/Review 合并审查（转译阶段）**：
  - 在生成阶段：CodeAgent 尽量生成安全抽象，避免不必要的 unsafe
  - 在复核阶段：Review Agent 合并审查功能一致性和严重问题，强制补齐不变式与边界检查
  - 检查重点：功能一致性、严重问题（空指针解引用、越界访问等）、FFI 不变式与 SAFETY 注释
- **optimizer（优化阶段）**：
  - 使用 CodeAgent 自动尝试缩小或移除 unsafe 面积（识别并移除不必要的 `unsafe { ... }` 包裹）
  - 验证方式：`cargo test -q`（确保最小测试通过）
  - 失败处理：回滚该处修改，并自动补充 SAFETY 注释实现“可回退”
  - 统计记录：`unsafe_removed` / `unsafe_annotated` 写入 `optimize_report.json`

渐进式代码演进路径规划

目标

- 在不打断业务与交付的前提下，分阶段将存量 C/C++ 代码演进为“更安全、更可维护”的形态，优先实现安全加固与可观测性增强，再逐步引入 Rust 化与库替代。
- 明确每个阶段的输入/输出、门禁指标与回退策略，保证每步可验证、可回退、可度量。

阶段分解与门禁

- 阶段0：安全基线评估（jarvis-sec）
  - 输入：项目根目录
  - 动作：workflow.direct_scan → report.build_json_and_markdown
  - 产出：issues.json(+md)、summary（by_language/by_category/top_risk_files）
  - 门禁：留存基线分数（score_sum/issue_count），作为后续阶段“安全分数改进量”的对比基线
- 阶段1：C/C++ 安全加固（最小修复）
  - 目标：修复高优先问题（unsafe_api、buffer_overflow、memory_mgmt、error_handling）
  - 方法：CodeAgent 驱动小步 edit_file（优先对 top_risk_files）；静态脚本验证（execute_script + rg/clang-tidy 可选）
  - 门禁：构建不退化、report.score_sum 显著下降（例如 ≥15%），关键类别计数下降
  - 回退：基于 git 提交与可逆补丁；失败不影响整体推进
- 阶段2：结构化与可测试化（可选）
  - 目标：拆分巨函数/巨文件、补充 smoke tests，明确模块边界与头文件契约
  - 方法：CodeAgent 生成最小单测占位；对 I/O 与错误路径添加断言
  - 门禁：构建通过；新增测试通过；report 问题不恶化
- 阶段3：FFI 边界安全包装
  - 目标：在 C 层引入“安全包装层”，将 (ptr,len)/返回码/errno 明确为契约
  - 方法：添加边界检查与统一错误转换；为未来 Rust 化准备可替换边界
  - 门禁：边界层单测通过；error-handling 类问题减少
- 阶段4：库替代评估与剪枝（library_replacer）
  - 目标：可由成熟 Rust 库整体替代的子树优先替代，减少自实现面积
  - 方法：apply_library_replacement（支持 candidates/disabled_libraries/resume）
  - 门禁：替代后 symbols/order 产物有效；后续转译规模减少
  - 回退：保持原始 symbols.jsonl 与 mapping，失败即放弃替代继续转译路径
- 阶段5：渐进式 Rust 转译（transpiler）
  - 目标：按 translation_order 逐函数转译；每次小步提交、可断点续跑
  - 方法：模块与签名规划 → CodeAgent 生成 → cargo check/test 闭环 → Review 合并审查 → 符号映射
  - 门禁：每批次构建/测试通过，进度与映射持续更新；report 中 unsafe_usage 与 memory_mgmt 类风险在 Rust 新增代码中接近 0
  - 回退：progress.json 断点；单函数级别失败可跳过/重试
- 阶段6：保守优化与面向外部接口收口（optimizer）
  - 目标：unsafe 最小化、重复消除、可见性缩小、文档补齐；对外接口稳定
  - 方法：optimizer.optimize_project（git_guard 可回滚）
  - 门禁：cargo test 全通过；unsafe_removed/annotated 指标提升；可见性收口不破坏 API
- 阶段7：替换调用与退役 C 代码（持续）
  - 目标：逐步用 Rust 安全封装替代 C 调用；最终退役对应 C 模块
  - 方法：构建脚本/链接配置切换；保持 FFI 兼容期；渐进删除旧实现
  - 门禁：覆盖率与性能不退化；关键路径延迟可接受；bug 回归为 0

任务挑选与优先级

- 以 report.summary.top_risk_files 为优先队列，穿插处理“高危类别”与“代价低收益高”的问题（如 format_string/scanf 未限宽、realloc 覆盖原指针）。
- 转译阶段优先选择“调用图下游（被调）→ 上游（调用者）”的顺序，减少接口震荡；利用 translation_order.jsonl 已排序步骤。
- 对可由库替代的子树优先替代（减少自实现与 unsafe），其余进入转译。

指标体系与度量

- 安全分数改进：Δscore_sum、各类别 Δcount（high/medium/low）
- 构建/测试：cargo check/test 成功率、重试次数、build_attempts 分布
- Rust 化进度：symbol_map.jsonl 映射条目数/占比、转译函数数/总数
- unsafe 面积：optimizer.unsafe_removed/annotated；rust_checker 中 unsafe_usage 的计数变化
- 可见性：pub→pub(crate) 成功数；文档补齐数（docs_added）
- 性能/资源（可选）：关键路径基准延迟/内存

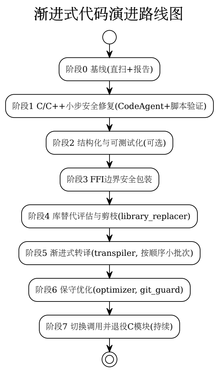
回退与可靠性

- git_guard：优化阶段失败自动 reset 到步骤前快照
- 原子写：library_replacer 与产物写入使用原子写，避免部分写入损坏
- 断点续跑：transpiler.progress.json、optimizer.optimize_progress.json 持续记录进度
- 失败隔离：单函数/单文件失败不阻断全局流程，进入下一项

自动化编排建议

- 将阶段 0/1/4/5/6 编排为可重入流水线：
  - 安全基线 → 小步安全修复 → 库替代评估 → 渐进转译 → 保守优化
- 非交互运行：
  - jarvis-sec: agent 命令（失败时自动回退到 direct_scan）
  - c2rust: scan → lib-replace → prepare → transpile → optimize
- 中间产物版本化并入仓（.jarvis/c2rust/* 与 issues 基线），支持跨分支协作与回归对比

PlantUML：演进路线图（简化）



门禁判定（示例策略）

- 若 report.score_sum 降幅 < X% 或 high 严重度问题仍 > N，继续阶段1修复
- 若 library_replacer 替代覆盖率 < Y%，放宽 candidates 或进入阶段5转译
- 若 cargo check/test 重试超过阈值，暂停本批次，返回阶段1/3补强边界与结构

与现有实现的映射

- 安全评估与修复：jarvis_sec.workflow/report + CodeAgent 小步补丁
- 库替代与剪枝：jarvis_c2rust.library_replacer（resume/disabled_libraries）
- 转译与闭环：jarvis_c2rust.transpiler（progress/symbol_map/Review 合并审查）
- 优化与回滚：jarvis_c2rust.optimizer（unsafe/可见性/文档 + git_guard）